My Project

Date 2026-10-29

Table of Contents

---

1. Aero_data/Parallel Runs/steven_merge_aero.py
2. Aero_data/fix_aero_data.py
3. compare_performance_lq.py
4. design_optimization.py
5. flight_performance_metrics.py
6. pitch_integrator_nonlinear.py
7. pitch_integrator_nonlinear_compare.py
8. plot_aero_data_sweeps.py
9. plot_gramian_eigenvectors.py
10. process_plot_data.py
11. utils/compute_inf_reach_set.py
12. utils/gramian_metrics.py
13. utils/interpolatory_model.py
14. utils/my_rk4.py
15. utils/nonlinear_model.py
16. utils/plotters.py
17. utils/progress_bar.py
18. utils/reach_set_data_read.py
19. utils/reach_set_functions.py
20. utils/read_hdf5.py
21. utils/ref_trackers.py
22. utils/solve_rde.py
23. utils/transient_errors.py
24. utils/trim_aircraft.py
25. utils/volume_smoothness/volume_smoothness.py

Aero_data/Parallel Runs/steven_merge_aero.py

```

import numpy as np

N_CHUNKS = 10

merged = {}
for c in range(N_CHUNKS):
    d = np.load(f'Aero_data/Parallel Runs/aero_data{c}.npz')
    for k in d.files:
        if k in ('ML','PL','CHSL','WHSL','ERL','WSL','THSL','WDL'):
            merged[k] = d[k]
        else:
            merged[k] = merged.get(k, np.zeros_like(d[k])) + d[k]
np.savez_compressed('Aero_data/aero_data_merged.npz', **merged)
```

Aero_data/fix_aero_data.py

```
# Fix Aero Data

import numpy as np
import matplotlib.pyplot as plt
from textwrap import wrap

saveit = 1


aero_arrays = np.load('Aero_data/aero_data_merged_forces_fixed.npz')



PL = aero_arrays['PL']
WHSL = aero_arrays['WHSL']
CHSL = aero_arrays['CHSL']
ERL = aero_arrays['ERL']
ML = aero_arrays['ML']
X_v = aero_arrays['X_v']
X_alpha = aero_arrays['X_alpha']
X_deltae = aero_arrays['X_deltae']
M_alpha = aero_arrays['M_alpha']
M_v = aero_arrays['M_v']
M_deltae = aero_arrays['M_deltae']
Z_alpha = aero_arrays['Z_alpha']
Z_v = aero_arrays['Z_v']
Z_deltae = aero_arrays['Z_deltae']
Fx = aero_arrays['Fx_mat']
Fz = aero_arrays['Fz_mat']
M = aero_arrays['M_mat']
CL = aero_arrays['CL_mat']
Di = aero_arrays['Di']
L = aero_arrays['L']
Cdi = aero_arrays['Cdi_mat']

```

```
WSL = aero_arrays['WSL'] # wing sweep list
THSL = aero_arrays['THSL'] # transition half span list
WDL = aero_arrays['WDL'] # wing dihedral list


weight = 331522 # lb
weight = weight * 4.44822 # convert to N
# weight = 1645907.20892072 # N, estimate on 9/30
mass = weight / 9.81 # kg
Iy = 3759014 # slug ft^2
Iy = Iy * 14.5939 * 0.3048**2 # kg m^2

cg_loc = 14


''' Fix the force rotation '''
def fix_forces(fx,fz,alpha):
    # Take in the incorrectly rotated fx, fz
    # fxo is the reconstruction of the panel method fx, fz
    # fxp is the properly rotated one

    fzo = (fz + fx*np.sin(alpha))/np.cos(alpha)
    fxo = (fx - fzo*np.sin(alpha))/np.cos(alpha)

    fxp = fxo*np.cos(alpha) + fzo*np.sin(alpha)
    fzp = -fxo*np.sin(alpha) + fzo*np.cos(alpha)

    return fxp,fzp


# Fx,Fz = fix_forces(Fx,Fz,PL)
# X_v,Z_v = fix_forces(X_v,Z_v,PL)
# X_alpha,Z_alpha = fix_forces(X_alpha,Z_alpha,PL)
# X_deltae,Z_deltae = fix_forces(X_deltae,Z_deltae,PL)

'''Fix moments and derivatives'''

M= M - cg_loc*Fz * np.cos(PL) - cg_loc*Fx*np.sin(PL)     # This works.

M_v = (Iy)* M_v - cg_loc * np.cos(PL)*Z_v*mass - cg_loc*np.sin(PL)*X_v*mas
    s
M_v = M_v/Iy

M_alpha = (Iy) * M_alpha - cg_loc * np.cos(PL)*Z_alpha * mass - cg_loc*np.s
    in(PL)*X_alpha * mass + cg_loc * np.sin(PL)*Fz - cg_loc*np.cos(PL)*F
    x   # not that close ish, but maybe good enough?
M_alpha = M_alpha / (Iy)

M_deltae = Iy* M_deltae - cg_loc * np.cos(PL)*Z_deltae*mass - cg_loc*np.si
    n(PL)*X_deltae*mass
M_deltae = M_deltae/Iy

array_to_save = {
    'X_v': X_v,
    'Z_v': Z_v,
    'M_v': M_v,
    'X_alpha': X_alpha,
    'Z_alpha': Z_alpha,
```

```
    'M_alpha': M_alpha,
    'X_deltae': X_deltae,
    'Z_deltae': Z_deltae,
    'M_deltae': M_deltae,
    'CL_mat': CL,
    'Fx_mat': Fx,
    'Fz_mat': Fz,
    'M_mat': M,
    'Cdi_mat': Cdi,
    'ML': ML,
    'PL': PL,
    'CHSL': CHSL,
    'WHSL': WHSL,
    'ERL': ERL,
    'WSL': WSL,
    'THSL': THSL,
    'WDL': WDL,
    'Di': Di,
    'L': L
}
if saveit:
    np.savez_compressed(f'Aero_data/aero_merged_cg_{cg_loc}.npz', **array_
    to_save)
```

---

compare_performance_lq.py

```
# This code will take the optimized and initial system matrices from the
# design optimization problem and compare the two linear systems in terms
    of
# how well they can complete reference tracking/stabilization tasks. These
     will
# be evaluated in terms of required control effort and tracking error.

from design_optimization import *
import numpy as np
from scipy.integrate import trapezoid
from utils.read_hdf5 import extract_dataset_from_results
from utils.ref_trackers import lqr_ref_track
from utils.plotters import linear_sim_plots

#%% Build system matrices, cost matrices, time horizon

def simulate_lq(design, case, show_error, do_meas_noise=False, do_proc_noi
    se=False,more_data = False):
    if more_data:
        aero_data_file = 'Aero_data/aero_merged_cg_17.npz'
    else:
        aero_data_file = 'Aero_data/aero_7dim2.npz'
    interp_dict = aero_interps(aero_data_file)

    # # Flight point
    V0 = 200 # m/s
    Tf = 5

    if more_data:
```

```
27          chs, whs, ws, ths, wd = design
28          alpha0, er0, deltaT0, pitch0, _ = compute_trim_eom(np.asarray([V0,c
        hs,whs,ws,ths,wd]), aero_data_file,climb_angle=0)
29      else:
30          chs, whs = design
31          alpha0, er0, deltaT0, pitch0, _ = compute_trim_eom(np.asarray([V0,c
        hs,whs]), aero_data_file,climb_angle=0)
32
33      flight_point = np.array([V0, alpha0, er0])
34      # print(f'Optimized trimmed alpha: {np.rad2deg(alpha0)}, deltae: {np.r
        ad2deg(er0)}, deltaT: {deltaT0}, pitch: {np.rad2deg(pitch0)}')
35      trim_point = np.array([V0, alpha0, 0, pitch0])
36
37      # First construct A,B matrices
38      full_flight_point = np.concatenate((flight_point[0:2], design[0:2], fl
        ight_point[2:],design[2:]))
39      A,B = build_AB(interp_dict,full_flight_point)
40
41      # Compute input bounds
42      thrust_ub = 1-deltaT0
43      thrust_lb = -deltaT0
44      elev_ub = np.deg2rad(30) - er0
45      elev_lb = np.deg2rad(-30) - er0
46
47      u_upper = np.array([thrust_ub, elev_ub])
48      u_lower = np.array([thrust_lb, elev_lb])
49
50      # Simulate the system's LQ tracking
51      t_opt, x_opt, xref_opt, u_opt, error_opt = lqr_ref_track(A,B,case,Tf=T
        f,u_ub = u_upper,u_lb=u_lower, do_meas_noise=do_meas_noise, do_pro
        c_noise=do_proc_noise)
52
53      return t_opt, x_opt, u_opt, np.tile(trim_point.reshape(-1,1),t_opt.siz
        e), error_opt
54
55  if __name__ == "__main__":
56
57
58      cases = ['pitch','velocity']
59      # cases = ['pitch']
60      error = False
61      saveit = False
62      do_meas_noise = False
63      do_proc_noise = False  # This one seems to make for more interesting r
        esults. Might need to debug the other case if desired.
64
65
66      opt_path1 = 'Output/5_dec_vars/N6_volmax'
67      opt_path2 = 'Output/5_dec_vars/N6_115max'
68      opt_path3 = 'Output/5_dec_vars/N6_volmax_115con'
69
70      models = [opt_path1, opt_path2, opt_path3]
71
72      sim_data = {}
73
74      # Initial case
75      chs = 5
76      whs = 12
```

```
    ws = 35
    ths = 4.9
    wd = 10
    design = [chs,whs,ws,ths,wd]

    for case in cases:
        t,x,u,tp, error_init = simulate_lq(design,case,error,do_meas_noise,do_proc_noise,True)

        u_l2_norm_init = np.sqrt(trapezoid(np.sum(u**2,axis=0),t))
        elev_l2_norm_init = np.sqrt(trapezoid(u[1,:]**2,t))
        thrust_l2_norm_init = np.sqrt(trapezoid(u[0,:]**2,t))

        sim_data[f'initial_{case}'] = {
            't': t,
            'x': x,
            'u': u,
            'tp': tp,
            'style': 'dashed',
            'color': 'orange',
            'name': 'initial',
            'ul2':u_l2_norm_init,
            'elev_l2':elev_l2_norm_init,
            'state_l2':error_init
        }

    for file in models:
        for case in cases:
            output_hdf5_file = f"{file}/opt_vars.hdf5"
            design = extract_dataset_from_results(output_hdf5_file, "x")
            model_type = file.split('/')[-1][3:]

            t,x,u,tp, error_opt = simulate_lq(design,case,error,do_meas_noise,do_proc_noise,True)
            u_l2_norm_opt = np.sqrt(trapezoid(np.sum(u**2,axis=0),t))
            elev_l2_norm_opt = np.sqrt(trapezoid(u[1,:]**2,t))
            thrust_l2_norm_opt = np.sqrt(trapezoid(u[0,:]**2,t))

            if model_type == 'volmax':
                key = 'vm'
                color = 'red'
            elif model_type == '115max':
                key = 'dm'
                color = 'blue'
            elif model_type == 'volmax_115con':
                key = 'vmdc'
                color = 'green'
            elif model_type == 'inf_set':
                key = model_type
                color = 'salmon'
            elif model_type == 'gramian_eigs':
                key = model_type
                color = 'teal'

            sim_data[f'{model_type}_{case}'] = {
                't': t,
                'x': x,
                'u': u,
```

```
                'tp': tp,
                'style': 'solid',
                'color': color,
                'name':key,
                'ul2': u_l2_norm_opt,
                'elev_l2':elev_l2_norm_opt,
                'state_l2':error_opt
            }
            print('-----------------------------------------------')
            print(f'Showing {model_type} for case {case}')
            error_init = sim_data[f'initial_{case}']['state_l2']
            u_l2_init = sim_data[f'initial_{case}']['ul2']
            elev_l2_init = sim_data[f'initial_{case}']['elev_l2']
            print(f'Optimized tracking error: {error_opt:.4f}, initial tracking error: {error_init:.4f}, change of: {(error_init-error_opt)/error_init*100:.2f}%')
            print(f"Opt control signal: {u_l2_norm_opt:.4f}, init control signal: {u_l2_init:.4f},          change of: {(u_l2_init-u_l2_norm_opt)/u_l2_init*100:.2f}%")
            print(f"Opt elev signal: {elev_l2_norm_opt:.4f}, init: {elev_l2_init:.4f},                    change of: {(elev_l2_init - elev_l2_norm_opt)/elev_l2_init*100:.2f}%")
    for case in cases:
        linear_sim_plots(sim_data,['initial','volmax','115max','volmax_115con'],
                         case,saveit)


```

design_optimization.py

```
from scipy.linalg import expm
from pyslsqp import optimize
from pyslsqp.postprocessing import print_dict_as_table
from utils.interpolatory_model import *
import numpy as np
import csv
from utils.compute_inf_reach_set import compute_inf_reach_set_sphere
from utils.reach_set_functions import compute_hull
from functools import partial
from utils.read_hdf5 import extract_dataset_from_results
from utils.trim_aircraft import compute_trim_eom
from scipy.optimize import linprog
import time
import pandas as pd

from utils.gramian_metrics import *

''' This script solves the actual design optimization problem. It loads aircraft parameters from a CSV file and on each iteration,
builds the A,B matrices while computing aerodynamic force/moment coefficients as a function of physical/operating parameters.'''

def load_parameters(csv_file_path):
    """
```

```
    Reads a CSV file containing parameter names and values, ignoring the first row (header),
    and loads the parameters into a dictionary.
    """
    parameters = {}
    with open(csv_file_path, mode="r") as csv_file:
        reader = csv.reader(csv_file)
        next(reader)  # Skip the header row
        for row in reader:
            param_name = row[0]
            param_value = float(row[1])  # Convert the value to float
            parameters[param_name] = param_value
    return parameters

def volume_obj(x, flight_point):
    ''' This objective computes the negative of the volume of the p-norm reach set for design variable x. Negative so we
    can maximize it.'''

    hull = set_hull(x,flight_point)
    # hull = set_rate_hull(x, flight_point)
    print(f"Design variables: {x}")
    vol = hull.volume
    print(f"Volume of convex hull: {vol}")
    return -vol  # Negative for maximization

def scaled_volume_obj(x, flight_point):
    ''' This objective just computes volume_obj, but scales it up to make optimization easier'''
    hull_vol = volume_obj(x, flight_point)
    return hull_vol * 1e2

def lift_drag(x, desired_vel):
    '''Computes lift over drag value from interpolator.'''
    data_file = 'Aero_data/aero_7dim2.npz'
    interp_dict = aero_interps(data_file)
    chs = x[0]
    whs = x[1]
    alpha0, deltae0, deltaT0, pitch0 = compute_trim_eom(
        np.asarray([desired_vel, chs, whs]), data_file, climb_angle=0
    )
    flight_point = np.array([desired_vel, alpha0, deltae0])
    full_flight_point = np.concatenate((flight_point[0:2], x, flight_point[2:]))

    lift = interp_dict['CL'](full_flight_point)
    drag = interp_dict['Cdi'](full_flight_point)
    print(f'lift:{lift}')
    print(f'drag:{drag}')
    c = -lift/drag
    print(c)
    return c[0]

def set_hull(x,desired_vel, already_trimmed = False, trim_data = None,save_set = False):
    ''' This function computes the A,B matrices of the system, computes the p-norm reach set, and returns the convex hull of the points.
    It will also save the data from the p-set computation into Outputs/reach_data.csv if desired
```

```
75
76      Input: x: array of design variables, where x[0] = center halfspan of b
        wb, x[1] = wing halfspan of bwb,
77      flight_point is the trim [V, pitch, er]. We will add chs and whs into
        this array.
78      But maybe now flight_point is just going to become velocity.
79
80      Output: reachable set hull: scipy.spatial.ConvexHull object'''
81
82      if already_trimmed:
83          tp, flight_point, data_file = trim_data
84      else:
85          tp, flight_point, data_file, _cost = design_trim(x,desired_vel)
86      deltaT0 = tp[2]
87      deltae0 = tp[1]
88      alpha0 = tp[0]
89      pitch0 = tp[3]
90
91      interp_dict = aero_interps(data_file)
92
93      # print(f'Trimmed alpha: {np.rad2deg(alpha0)}, deltae: {np.rad2deg(del
        tae0)}, deltaT: {deltaT0}, pitch: {np.rad2deg(pitch0)}')
94
95      # Compute input bounds
96      thrust_ub = 1-deltaT0
97      thrust_lb = -deltaT0
98      elev_ub = np.deg2rad(30) - deltae0
99      elev_lb = np.deg2rad(-30) - deltae0
100
101     u_upper = np.array([thrust_ub, elev_ub])
102     u_lower = np.array([thrust_lb, elev_lb])
103
104     # flight_point = np.array([desired_vel, alpha0, deltae0])
105
106     full_flight_point = np.concatenate((np.array([desired_vel, alpha0]), x[0:2], n
        p.array([deltae0]),x[2:]))
107     A,B = build_AB(interp_dict,full_flight_point)
108
109     # If we want to see the A,B matrices
110     # print(f'A: {A}')
111     # print(f'B: {B}')
112
113     tf = 1
114
115     N_res = 6     # Resolution for the costate sampling
116
117     # With this, actually compute the set
118     control_cost, final_positions = compute_inf_reach_set_sphere(A,B,tf,N_
        res,u_ub = u_upper, u_lb = u_lower, \
119                                                                 save_data=s
        ave_set,filename='Output/bwb_inf_data.csv')
120
121     # Next, take the convex hull of the points in final_positions, computi
        ng inclusion based on control cost.
122     hull = compute_hull(final_positions)
123
124     return hull
125
```

```python
def set_rate_hull(x, desired_vel):
    ''' This function computes the A,B matrices of the system, computes the rate-limited reach set,
    and returns the convex hull of the points.
    It will also save the data from the computation into Outputs/reach_data.csv if desired

    Input: x: array of design variables, where x[0] = center halfspan of bwb, x[1] = wing halfspan of bwb,
    desired_vel is the desired velocity we will trim it at.

    Output: reachable set hull: scipy.spatial.ConvexHull object'''

    # First construct A,B matrices
    tp, flight_point, datat_file = design_trim(x,desired_vel)
    deltaT0 = tp[2]
    deltae0 = tp[1]
    alpha0 = tp[0]
    pitch0 = tp[3]

    print(f'Trimmed alpha: {np.rad2deg(alpha0)}, deltae: {np.rad2deg(deltae0)}, deltaT: {deltaT0}, pitch: {np.rad2deg(pitch0)}')

    interp_dict = aero_interps(data_file)
    # Compute input bounds
    thrust_ub = 1-deltaT0
    thrust_lb = -deltaT0
    elev_ub = np.deg2rad(30) - deltae0
    elev_lb = np.deg2rad(-30) - deltae0

    u_upper = np.array([thrust_ub, elev_ub])
    u_lower = np.array([thrust_lb, elev_lb])

    v_upper = np.array([1, 1])
    v_lower = np.array([-1, -1])

    flight_point = np.array([desired_vel, alpha0, deltae0])

    full_flight_point = np.concatenate((flight_point[0:2], x, flight_point[2:]))
    A,B = build_AB(interp_dict,full_flight_point)

    # If we want to see the A,B matrices
    print(f'A: {A}')
    print(f'B: {B}')

    tf = 1

    N_res = 6     # Resolution for the costate sampling

    ubnds = np.block([u_upper.reshape(-1,1),u_lower.reshape(-1,1)])
    start_time = time.perf_counter()
    pts_list = lnprg_reach(A,B,ubnds, np.array([1, -1]), tf, 0.01, N_res ** 4, True, 'Output/rate_sets/reach_set')

    end_time = time.perf_counter()
    elapsed = end_time - start_time
    print(f'Took {elapsed} seconds')
    cc = np.zeros(N_res**4)
```

```
179     hull = compute_hull(cc, pts_list.T, cost_threshold=1)
180 
181     return hull
182 
183 def lnprg_reach(A,B,ubnds,udotbnds,tf,dt,sweep_size, save=False, savename='l
    nprg_traj_data'):
184     '''This function will use linear programming to solve the rate+magnitu
    de bounded reachability problem for a LTI system with initial cond
    ition from the origin. Input bounds are assumed to be held for all
     inputs.
185     Inputs:
186     A, B:    LTI matrices
187     ubnds:    input magnitude [[max1, min1],
188                                [max2, min2],
189                                [max3, min3],...]
190     udotbnds: rate bounds [max, min]
191     tf, dt:   final time and time step size for discretization
192     sweep_size: How many points to pull around the boundary
193     '''
194 
195     n = A.shape[0]
196     m = B.shape[1]
197 
198     # Set up parameters and matrices for LP
199     Abar = expm(A*dt)
200     Bbar = -np.linalg.inv(A) @ (np.eye(n) - Abar) @ B
201 
202     N = int((tf-0)/dt)
203 
204     Ahat = np.concatenate((np.concatenate((np.zeros((n,N*n)),np.kron(np.ey
    e(N),Abar)),axis = 0),
205                            np.zeros((N*n + n, n))), axis=1)
206     Bhat = np.block([[np.zeros((n,m*(N)))],
207                      [np.kron(np.eye(N),Bbar)]])
208 
209     # This picks out the inputs
210     IU = np.concatenate((np.zeros((m*N, n*(N+1))), np.eye(m*N)),axis=1)
211 
212     # Picks out x_N
213     IN = np.concatenate((np.zeros((n,n*N)),np.eye(n),np.zeros((n,m*N))),ax
    is=1)
214 
215     # Picks last N-1 inputs and first N-1 inputs.
216     I1 = np.concatenate((np.zeros((m*(N-1),m)),np.eye(m*(N-1))),axis=1)
217     I2 = np.concatenate((np.eye(m*(N-1)),np.zeros((m*(N-1),m))),axis=1)
218 
219     I3 = np.concatenate((1/dt * (I1-I2), -1/dt * (I1-I2)),axis=0)
220 
221     # Now set up matrices for linear program run
222     # equality constraint
223     Aeq = np.block([[np.eye(Ahat.shape[0]) - Ahat,-Bhat],
224                     [np.zeros((m,n*(N+1))),np.block([[np.eye(m), np.zeros((m,m*(N-1)))]])
225     beq = np.zeros((Aeq.shape[0]))
226 
227     # inequality constraint
228     Aineq = I3 @ IU
229     bineq1 = np.full((m*(N-1)),udotbnds[0])
230     bineq2 = np.full((m*(N-1)),-udotbnds[1])
```

```
231     bineq = np.concatenate((bineq1,bineq2),axis=0)
232 
233     # bineq1 = np.zeros((m*(N-1)))
234     # bineq2 = bineq1
235     # for i in range(N-1):
236     #     bineq1[i*m:(i+1)*m] = -udotbnds[:,0]
237     #     bineq2[i*m:(i+1)*m] = udotbnds[:,1]
238     # bineq = np.concatenate((bineq1,bineq2),axis=0)
239 
240     # variable bounds
241     # lb = np.concatenate((np.full((n*(N+1)),None),np.full((m*(N)),ubnds[1
        ])),axis=0)
242     # ub = np.concatenate((np.full((n*(N+1)),None),np.full((m*(N)),ubnds[0
        ])),axis=0)
243 
244     xbounds = [(None, None)] * (n*(N+1))
245     # ubounds = [(ubnds[1], ubnds[0])] * (m * N)
246     ubounds_single = [(ubnds[i,1],ubnds[i,0]) for i in range(m)]
247     ubounds = ubounds_single * N
248     bounds = xbounds + ubounds
249 
250     def solve_lp(lam):
251         res = linprog(-IN.T@lam, A_eq=Aeq, b_eq=beq, A_ub = Aineq, b_ub = b
        ineq, bounds=bounds)
252         return res
253 
254     theta = np.deg2rad(np.linspace(0,360,sweep_size))
255 
256     pt_list = np.zeros((n,sweep_size))
257 
258     # fig, ax = plt.subplots()
259 
260     traj_list = np.zeros((sweep_size,n,N+1))
261     u_list = np.zeros((sweep_size,m,N))
262 
263     rng = np.random.default_rng(seed=50)
264     lam_samples = rng.standard_normal((sweep_size, n))
265     lam_samples /= np.linalg.norm(lam_samples, axis=1, keepdims=True)
266 
267     for i in range(sweep_size):
268         result = solve_lp(lam_samples[i])
269 
270         sol = result.x
271         n_x = n * (N + 1)
272 
273         x = sol[:n_x].reshape(N + 1, n).T   # shape (n, N+1), column k is
        x_k
274         u = sol[n_x:].reshape(N, m).T       # shape (m, N),   column k is
        u_k
275 
276         # ax.scatter(x[0,-1],x[1,-1],color='green')
277         pt_list[:,i] = x[:,-1]
278 
279         traj_list[i] = x
280         u_list[i] = u
281 
282     if save:
283         np.savez_compressed(f'{savename}.npz',x=traj_list,u=u_list)
```

```
284
285     if save:
286         # Create dynamic column names based on state dimension
287         data = {}
288
289         # Add costate initial conditions
290         print(lam_samples.shape)
291         print(sweep_size)
292         for i in range(n):
293             data[f"LX{i+1}"] = lam_samples[:,i]
294
295         # Add final positions
296         for i in range(n):
297             data[f"final_x{i+1}"] = pt_list[i, :]
298
299         # data['control_cost'] = np.ones(lam_samples.shape[0])
300
301         df = pd.DataFrame(data)
302
303         # Save to CSV
304         df.to_csv(f'{savename}.csv', index=False)
305
306         print(f"\nData saved to {savename}")
307
308     return pt_list
309
310
311 def compute_axis_extent(x,v,desired_vel, already_trimmed = False, trim_dat
    a = None):
312     ''' This constraint computes the length of the p-norm reach set projec
    ted along a given direction.
313     Initial length along [0,0,0,1] is...
314
315     Going to also try checking along [0,1,0,0]...'''
316     hull = set_hull(x,desired_vel, already_trimmed = already_trimmed, trim
    _data = trim_data)
317
318     # Define axis for projection
319     axis = v
320     # ensure unit vector
321     axis = axis/np.linalg.norm(axis)
322
323     hull_points = hull.points[hull.vertices]
324     projections = hull_points @ axis
325
326     # Compute min and max position of projections along axis
327     proj_min, proj_max = projections.min(), projections.max()
328
329     # Compute the actual vectors in case of plot
330     line_points = np.concatenate((proj_min * axis, proj_max * axis),axis=1) #
    Saves min and max as [x_min, x_max]
331     extent = proj_max - proj_min
332     print(f"Design variables: {x}")
333     print(f"Extent along direction {v}: {extent}")
334     return extent
335
336 def direction_constraint(x,v,extent0,desired_vel):
337     ''' This is a wrapper for compute_axis_extent to be used as a constrai
```

```
    nt in the optimization.
338     Projects onto fourth axis [0,0,0,1]'''
339     extent = compute_axis_extent(x,v,desired_vel)
340     con = extent - 1.5 * extent0
341     print(f'Direction Constraint: {con}')
342     return np.array([con]) # Want to increase extent by at least 50%.
343 
344 def direction_obj(x,v,desired_vel):
345     print(f"Design variables: {x}")
346     extent = compute_axis_extent(x,v,desired_vel)
347     return -(extent) # Want to increase extent.
348 
349 def design_trim(x,desired_vel,climb_angle=0, give_cost = False):
350     chs = x[0]
351     whs = x[1]
352     if x.size < 3:
353         data_file = 'Aero_data/aero_7dim2.npz'
354         alpha0, deltae0, deltaT0, pitch0, cost = compute_trim_eom(np.asarr
    ay([desired_vel,chs,whs]), data_file,climb_angle=climb_angle)
355     else:
356         data_file = 'Aero_data/aero_merged_cg_17.npz'     # MAKE SURE TO P
    ASS THE CORRECT NUMBER OF FLIGHT_POINTS TO compute_trim_eom WHEN U
    SING THIS
357         wsl = x[2]
358         thsl = x[3]
359         wdl = x[4]
360 
361         alpha0, deltae0, deltaT0, pitch0, cost = compute_trim_eom(np.asarr
    ay([desired_vel,chs,whs,wsl,thsl,wdl]), data_file,climb_angle=clim
    b_angle)
362 
363     flight_point = np.concatenate((np.array([desired_vel, alpha0]), x[0:2], n
    p.array([deltae0]),x[2:]))
364 
365     return np.array([alpha0, deltae0, deltaT0, pitch0]), flight_point, dat
    a_file, cost
366 
367 def static_stability(x,desired_vel, already_trimmed = False, trim_data = N
    one):
368     if already_trimmed:
369         tp, flight_point, data_file = trim_data
370     else:
371         tp, flight_point, data_file, _cost = design_trim(x,desired_vel)
372     alpha0 = tp[0]
373 
374     interp_dict = aero_interps(data_file)
375     M_alpha = interp_dict['malpha'](flight_point)
376     Z_alpha = interp_dict['zalpha'](flight_point)
377     # stat_margin = M_alpha/(Z_alpha * np.cos(alpha0))
378     stat_margin = M_alpha/(Z_alpha)
379     print(f'Static Margin:{stat_margin}')
380     return stat_margin
381 
382 def collective_constraints(x,desired_vel,v = None, extent0 = None,extent_b
    ool = True, stab_bool = True, trim_bool = True, desired_sm = 0):
383 
384     cons = np.array([])
385     tp, flight_point, data_file, cost = design_trim(x,desired_vel,give_cos
```

```
    t = trim_bool)
386
387     if extent_bool:
388         extent = compute_axis_extent(x,v,desired_vel, already_trimmed = Tr
    ue, trim_data = (tp, flight_point, data_file))
389         cons = np.block([cons, extent - 1.275 * extent0])
390     if stab_bool:
391         stability = static_stability(x,desired_vel,already_trimmed = True, t
    rim_data = (tp, flight_point, data_file))
392         cons = np.block([cons, stability.item() - desired_sm])
393     if trim_bool:
394         thrust_force = 96000 # lb
395         maxT = thrust_force * 4.44822 * 1.5 # convert to N
396         weight = 331522 # lb
397         weight = weight * 4.44822 # convert to N
398         mass = weight / 9.81  # kg
399         g = 9.81
400
401         interp_dict = aero_interps(data_file)
402         M = interp_dict['M']
403         Di = interp_dict['Di']
404         L = interp_dict['L']
405
406         deltaT = tp[2]
407         alpha = tp[0]
408         theta = tp[3]
409         # full_flight_point = np.concatenate((flight_point[0:1],tp[0:1],fl
    ight_point[1:3],tp[1:2],flight_point[3:]))
410         full_flight_point = flight_point
411         Ft = deltaT * maxT
412
413         # Velocity constraint
414         cons = np.block([ Ft/mass * np.cos(alpha) - Di(full_flight_point)/m
    ass - g * np.sin(theta - alpha), cons])
415
416         # AOA constraint
417         cons = np.block([(-Ft/(mass * V0) * np.sin(alpha) - L(full_flight_
    point)/(mass * V0) + g/V0 * np.cos(theta - alpha) ) * 1e3, cons])
418
419         # Q constraint
420         cons = np.block([M(full_flight_point) * 1e-5, cons])
421
422         # Gamma constraint
423         climb_angle = 0
424         # cons = np.block([theta - alpha - np.deg2rad(climb_angle), cons])

425     print(cons)
426
427     return cons
428
429 if __name__ == '__main__':
430     '''Now, the design variable is only going to be center-halfspan and wi
    ng-halfspan.
431     we can use partial to pass the other arguments to building A,B.
432     At each iteration, we will build A,B, compute the Lp norm reach set, c
    ompute its volume, and then optimize.
433
434     flight_point will contain [alt, V, pitch, er], the full one should con
```

```
    tain [alt, V, pitch, CHS, WHS, er].
435 chs and whs were separated out because they are the design variables t
    hat we will alter, the others will remain fixed for now.
436 '''
437
438 # This dictates if we have additional decision variables or not.
439
440 tb = False
441 sb = True
442 angle = 115
443 stat_margin = 0
444
445 chs = 5 # m
446 whs = 12 # m
447 ws = 35
448 ths = 4.9
449 wd = 10
450 x0 = np.array([chs,whs,ws,ths,wd])
451
452 V0 = 200 # m/s
453
454 flight_params = V0
455
456 # Axis on which to project reachable set during some optimization prob
    lems
457 v = np.array([[0,0,np.cos(np.deg2rad(angle)),np.sin(np.deg2rad(angle))]]).T
458
459 init_extent = compute_axis_extent(x0,v,flight_params)
460
461 '''Determination of which eigenvalues we care about from the Gramian.'
    ''
462 # vals, vecs = np.linalg.eig(gram)
463 # print(vecs)
464 # for i in range(4):
465 #     print(vecs[:,i])
466 #     plt.scatter(vecs[2,i],vecs[3,i],label=f'{i}')
467 #     angle = np.rad2deg(np.arccos(np.dot(vecs[:,i],np.asarray([0,0,0,
    1]))))
468 #     print(angle)
469 # plt.grid()
470 # plt.legend()
471 # plt.show()
472
473 # hull = gramian_hull(x0,flight_params,tf=1,savepts=True)
474 # fig2, ax2 = plot_projected_boundary(np.linalg.inv(gram), 1, ax=ax, l
    abel='Second',color='red',draw_analytic=False)
475
476
477
478 ''' Partial functions for objective and constraint functions '''
479 # fixed_Lp_objective = partial(volume_obj,flight_point=V0)
480 fixed_scaled_volume_obj = partial(scaled_volume_obj,flight_point=V0)
481 # fixed_malpha = partial(static_stability, desired_vel = V0)
482
483 fixed_direction_obj = partial(direction_obj,v=v,desired_vel=V0)
484 fixed_stab_con = partial(collective_constraints,desired_vel = V0, exte
    nt_bool = False, stab_bool = sb,trim_bool = tb, desired_sm = stat_
    margin)
```

```
485     fixed_direction_con = partial(direction_constraint,v=v,extent0=init_ex
        tent, desired_vel=V0)
486     fixed_direction_stab_con = partial(collective_constraints,v=v,extent0=i
        nit_extent,desired_vel=V0, extent_bool = True, stab_bool = sb, tri
        m_bool = tb, desired_sm = stat_margin)
487     fixed_gram_direction_obj = partial(gramian_direction_obj, v=v, desired
        _vel = V0 )
488     fixed_gram_eig_obj = partial(gramian_eig_obj, desired_vel=V0, eigs=np.a
        rray([2,3]) )
489     # fixed_lift_drag = partial(lift_drag, desired_vel = V0)
490
491     # Set up optimization
492     num_eqcon = 3 if tb else 0  # Number of equality constraints. We have
        4 from trim. 3 if not using climb angle.
493     x_low = np.array([3, 10, 15, 3, 10])
494     x_upp = np.array([7, 20, 50, 8, 12])
495
496
497     '''This will purely maximize the volume of reach set.'''
498     # results = optimize(x0, obj=fixed_scaled_volume_obj, meq = num_eqcon,
         con=fixed_stab_con, xl=x_low, xu = x_upp, \
499     #                    visualize = True, save_figname="Output/optimizati
        on_progress.png", summary_filename="Output/design_optimization.out
        ", \
500     #                     save_itr = 'major', save_filename = 'Output/opt_
        vars.hdf5')
501     # print_dict_as_table(results)
502
503     '''Optimize some direction w/o direction constraint'''
504     results = optimize(x0, obj=fixed_direction_obj, meq = num_eqcon, con=f
        ixed_stab_con, xl=x_low, xu = x_upp, \
505                        visualize = True, save_figname="Output/optimizatio
        n_progress.png", summary_filename="Output/design_optimization.out", \
506                          save_itr = 'major', save_filename = 'Output/op
        t_vars.hdf5')
507     print_dict_as_table(results)
508
509     '''Optimized for volume with direction constraint'''
510     # results = optimize(x0, obj=fixed_scaled_volume_obj, meq = num_eqcon,
         con=fixed_direction_stab_con, xl=x_low, xu = x_upp, \
511     #                    visualize = True, save_figname="Output/optimizat
        ion_progress.png", summary_filename="Output/design_optimization.ou
        t", \
512     #                          save_itr = 'major', save_filename = 'Output/
        opt_vars.hdf5', finite_diff_abs_step=np.array([1e-3, 1e-3, 1e-3, 1
        e-3, 1e-3]), \
513     #                              acc = 1e-4)
514     # print_dict_as_table(results)
515
516     ''' ----------------- others ------------------------- '''
517
518     # '''Optimize some eigenvalue of the reachability Gramian'''
519     # results = optimize(x0, obj=fixed_gram_eig_obj, meq = num_eqcon, con=
        fixed_stab_con, xl=x_low, xu = x_upp, \
520     #                    visualize = True, save_figname="Output/optimizat
        ion_progress.png", summary_filename="Output/design_optimization.ou
        t", \
521     #                          save_itr = 'major', save_filename = 'Output/
```

```
    opt_vars.hdf5')
    # print_dict_as_table(results)

    output_hdf5_file = "Output/opt_vars.hdf5"
    x0 = extract_dataset_from_results(output_hdf5_file, "x")
    hull = set_hull(x0,V0,save_set = True)
```

flight_performance_metrics.py

```
''' Computing other aircraft metrics'''

from utils.interpolatory_model import *
import numpy as np
from utils.read_hdf5 import extract_dataset_from_results
from utils.trim_aircraft import compute_trim_eom

names = ['init','volmax','115max','volmax_115con']
for i in names:
    print(i)
    if i == 'init':
        x = np.array([5,12,35,4.9,10])
    else:
        output_hdf5_file = "Output/5_dec_vars/N6_" + i + "/opt_vars.hdf5"
        x = extract_dataset_from_results(output_hdf5_file, "x")
    print(x)
    chs = x[0]
    whs = x[1]
    desired_vel = 200
    data_file = 'Aero_data/aero_merged_cg_17.npz'     # MAKE SURE TO PASS THE CORRECT NUMBER OF FLIGHT_POINTS TO compute_trim_eom WHEN USING  THIS
    wsl = x[2]
    thsl = x[3]
    wdl = x[4]

    alpha0, deltae0, deltaT0, pitch0, _ = compute_trim_eom(np.asarray([desired_vel,chs,whs,wsl,thsl,wdl]), data_file,climb_angle=0)

    interp_dict = aero_interps(data_file)
    # print(f'Trimmed alpha: {np.rad2deg(alpha0)}, deltae: {np.rad2deg(deltae0)}, deltaT: {deltaT0}, pitch: {np.rad2deg(pitch0)}')
    flight_point = np.array([desired_vel, alpha0, deltae0])
    full_flight_point = np.concatenate((flight_point[0:2], x[0:2], flight_point[2:],x[2:]))
    L = interp_dict['L'](full_flight_point)
    D = interp_dict['Di'](full_flight_point)

    print(f"Aero efficiency: {L/D}")
    print(f"Power required at 200 m/s: {D * desired_vel/1e6}")

    # Computing speed at which minimum drag acts

    # def obj(x):
    #     # For speed x, trim for level flight, then compute Cl^1.5 / Cdi
    #     # The speed at which this quantity is max is minimum drag
```

```
42
43      #     alpha0, deltae0, deltaT0, pitch0, _ = compute_trim_eom(np.asarra
        y([x[0],chs,whs]), data_file,climb_angle=0, warn_trim=False)
44
45      #     full_flight_point = np.asarray([x[0], alpha0, chs, whs, deltae0]
        )
46      #     Cdi = interp_dict['Cdi'](full_flight_point).item()
47      #     Cl = interp_dict['CL'](full_flight_point).item()
48      #     quantity = Cl**3/Cdi**2
49      #     if np.iscomplexobj(quantity):
50      #         raise RuntimeError(f"complex objective at x={x}, J={quantity
        }. Cdi = {Cdi}, CL = {Cl}")
51      #     return -quantity
```

pitch_integrator_nonlinear.py

```
 1  # Pitch integrator nonlinear controller
 2
 3  import numpy as np
 4  from utils.nonlinear_model import nonlinear_track_pitch
 5  from utils.read_hdf5 import extract_dataset_from_results
 6  from utils.plotters import error_bars, niceplot_3_models_alt
 7  from utils.transient_errors import transient_errors
 8
 9  if __name__ == "__main__":
10      # Test the nonlinear model by integrating it forward in time.
11
12      savefig = False
13      make_plots = True
14
15      angle = 115
16
17      opt_path1 = 'Output/5_dec_vars/N6_volmax'
18      opt_path2 = 'Output/5_dec_vars/N6_115max'
19      opt_path3 = 'Output/5_dec_vars/N6_volmax_115con'
20      models = [opt_path1, opt_path2, opt_path3]
21
22      speed1 = 190
23      climb_angle1 = 10
24      speed2 = 210
25      climb_angle2 = 0
26
27      # Single consolidated dictionary
28      sim_data = {}
29
30      data_file = 'Aero_data/aero_merged_cg_17.npz'
31      tf = 20
32      st = 10
33
34      # Initial
35      model_type = 'initial'
36      chs = 5  # center halfspan in m
37      whs = 12  # wing halfspan in m
38      ws = 35
39      ths = 4.9
```

```
    wd = 10
    design = np.array([chs,whs,ws,ths,wd])

    t, x, u_hist, tp_hist = nonlinear_track_pitch(design, np.array([speed1, climb_angle1]),
                                                  np.array([speed2, climb_angle2]), data_file, tf, switch_time=st)

    sim_data['initial'] = {
        't': t,
        'x': x,
        'u': u_hist,
        'tp': tp_hist,
        'style': 'dashed',
        'color': 'orange',
        'name': 'initial'
    }

    sim_data['ref'] = {
        't':t,
        'x':tp_hist,
        'style':'dotted',
        'color':'purple'
    }

    for opt_path in models:

        model_type = opt_path.split('/')[-1][3:]
        output_hdf5_file = f"{opt_path}/opt_vars.hdf5"
        x_opt = extract_dataset_from_results(output_hdf5_file, "x")

        print("Below, model type is:")
        print(model_type)
        print(x_opt)
        t, x, u_hist, tp_hist = nonlinear_track_pitch(x_opt, np.array([speed1, climb_angle1]),
                                                      np.array([speed2, climb_angle2]), data_file, tf, switch_time=st)

        if model_type == f'volmax_sm2':
            key = 'vm'
            color = 'red'
        elif model_type == f'{angle}max_sm2':
            key = 'dm'
            color = 'blue'
        elif model_type == f'volmax_{angle}con':
            key = 'vmdc'
            color = 'green'

        sim_data[model_type] = {
            't': t,
            'x': x,
            'u': u_hist,
            'tp': tp_hist,
            'style': 'solid',
            'color': color,
            'name':key
        }
```

```

    transient_errors(sim_data, switch_time = st)

    # error_bars(sim_data,save=savefig)
    if make_plots:
        print(sim_data.keys())
        # niceplot_3_models_alt(sim_data, ['initial',f'volmax',f'{angle}max',f'volmax_{angle}con'], save=savefig)
        niceplot_3_models_alt(sim_data, ['initial',f'volmax_sm2',f'{angle}max_sm2'], save=savefig)
```

pitch_integrator_nonlinear_compare.py

```
# Pitch integrator nonlinear controller

import numpy as np
from utils.nonlinear_model import nonlinear_track_pitch
from utils.read_hdf5 import extract_dataset_from_results
from utils.plotters import niceplot_3_models_alt
from utils.transient_errors import transient_errors

if __name__ == "__main__":
    # Test the nonlinear model by integrating it forward in time.

    savefig = False
    make_plots = True

    angle = 115

    opt_path1 = 'Output/5_dec_vars/Gramian_comparison/inf_set'
    opt_path2 = 'Output/5_dec_vars/Gramian_comparison/gramian_eigs'
    models = [opt_path1, opt_path2]

    speed1 = 190
    climb_angle1 = 10
    speed2 = 210
    climb_angle2 = 0

    # Single consolidated dictionary
    sim_data = {}

    data_file = 'Aero_data/aero_merged_cg_17.npz'
    tf = 20
    st = 10

    # Initial
    model_type = 'initial'
    chs = 5  # center halfspan in m
    whs = 12  # wing halfspan in m
    ws = 35
    ths = 4.9
```

```
39      wd = 10
40      design = np.array([chs,whs,ws,ths,wd])
41
42      t, x, u_hist, tp_hist = nonlinear_track_pitch(design, np.array([speed1, c
        limb_angle1]),
43                                                    np.array([speed2, climb
        _angle2]), data_file, tf, switch_time=st)
44
45      sim_data['initial'] = {
46          't': t,
47          'x': x,
48          'u': u_hist,
49          'tp': tp_hist,
50          'style': 'dashed',
51          'color': 'orange',
52          'name': 'initial'
53      }
54
55      sim_data['ref'] = {
56          't':t,
57          'x':tp_hist,
58          'style':'dotted',
59          'color':'purple'
60      }
61
62      for opt_path in models:
63
64          model_type = opt_path.split('/')[-1]
65          output_hdf5_file = f"{opt_path}/opt_vars.hdf5"
66          x_opt = extract_dataset_from_results(output_hdf5_file, "x")
67
68          print("Below, model type is:")
69          print(model_type)
70          print(x_opt)
71          t, x, u_hist, tp_hist = nonlinear_track_pitch(x_opt, np.array([spe
        ed1, climb_angle1]),
72                                                        np.array([speed2, c
        limb_angle2]), data_file, tf, switch_time=st)
73
74          if model_type == 'inf_set':
75              key = model_type
76              color = 'salmon'
77          elif model_type == 'gramian_eigs':
78              key = model_type
79              color = 'teal'
80
81          sim_data[model_type] = {
82              't': t,
83              'x': x,
84              'u': u_hist,
85              'tp': tp_hist,
86              'style': 'solid',
87              'color': color,
88              'name':key
89          }
90
91      transient_errors(sim_data, switch_time = st)
92
```

```
    if make_plots:
        niceplot_3_models_alt(sim_data,['inf_set','gramian_eigs'],save=savefig,savename='nonlinear_gramian',compare=True)
```

plot_aero_data_sweeps.py

```
import numpy as np
import matplotlib.pyplot as plt
from textwrap import wrap

saveit = 0

aero_arrays = np.load('Aero_data/aero_merged_cg_17.npz')

PL = aero_arrays['PL']
WHSL = aero_arrays['WHSL']
CHSL = aero_arrays['CHSL']
ERL = aero_arrays['ERL']
ML = aero_arrays['ML']
X_v = aero_arrays['X_v']
X_alpha = aero_arrays['X_alpha']
X_deltae = aero_arrays['X_deltae']
M_alpha = aero_arrays['M_alpha']
M_v = aero_arrays['M_v']
M_deltae = aero_arrays['M_deltae']
Z_alpha = aero_arrays['Z_alpha']
Z_v = aero_arrays['Z_v']
Z_deltae = aero_arrays['Z_deltae']
Fx = aero_arrays['Fx_mat']
Fz = aero_arrays['Fz_mat']
M = aero_arrays['M_mat']
CL = aero_arrays['CL_mat']
Di = aero_arrays['Di']
L = aero_arrays['L']
# Cdi = aero_arrays['Cdi_mat']
WSL = aero_arrays['WSL'] # wing sweep list
THSL = aero_arrays['THSL'] # transition half span list
WDL = aero_arrays['WDL'] # wing dihedral list

# --- Fix other dimensions (if more than 2D) ---
shape = PL.shape
nparams = len(shape)
ndims = shape[0]

# Index to pick out the slice of data we want to inspect. By default, slice(None) in any axis will slice nothing (take all).
# We want to replace these with constant indices corresponding to the parameters we hold constant.
indices = [0] * nparams

#%%
'''Based on the order of the meshgrid, the indices to vary will follow this order.
Setting sweep_pair to list the names of the variables we want to vary, array_order tells us the indices that need to be
```

```
46  set to slice(None). array_names tells us the corresponding matrix for when
        we plot the data later'''
47
48  array_order = {"speed":0,
49           "pitch":1,
50           "center halfspan":2,
51           "wing halfspan":3,
52           "elevator rotation":4,
53           "wing sweep":5,
54           "trans halfspan":6,
55           "wing dihedral":7}
56  array_names = array_order.copy()
57  array_names["speed"] = ML
58  array_names["pitch"] = PL
59  array_names["center halfspan"] = CHSL
60  array_names["wing halfspan"] = WHSL
61  array_names["elevator rotation"] = ERL
62  array_names["wing sweep"] = WSL
63  array_names["trans halfspan"] = THSL
64  array_names["wing dihedral"] = WDL
65
66  # if randomize set to False, then all constant parameters will just take t
        heir minimum value (essentially, set their index to 0)
67  randomize = False
68  sweep_pair = np.array(["pitch","elevator rotation"]) # The two parameters
        we want to vary in the 2D plot
69
70  # indices = np.zeros(ndims)
71  for i in sweep_pair:
72      indices[array_order[i]] = slice(None)
73
74  # If we choose to, this will randomize what value each constant parameter
        is set to.
75  if randomize:
76      for i in range(nparams-1):
77          if indices[i] == 0:
78              indices[i] = np.random.randint(0,ndims-1)
79      indices[-1] = np.random.randint(0,2)
80
81  # speed to 200 m/s
82  indices[0] = 2
83
84  indices[5] = 2 # wing sweep 32.5 deg
85  indices[6] = 2 # ths 5.5 m
86  indices[7] = 0 # wing dih 10 deg
87
88  #chs
89  indices[2] = 2
90
91  # whs
92  indices[3] = 1
93
94  indices = tuple(indices)
95  print(f'Indices: {indices}')
96
97  #%% Plot the data
98  X = np.rad2deg(array_names[sweep_pair[0]][indices])
99  Y = np.rad2deg(array_names[sweep_pair[1]][indices])
```

```
100 
101 Z = Fx[indices]/1000
102 output_label = '$F_x$'
103 
104 rho = M_alpha * -Z_deltae / (M_deltae * (Di - Z_alpha))
105 
106 plt.rcParams.update({
107     "text.usetex": True,
108     "font.family": "serif",
109     "font.serif": ["Computer Modern Roman"],
110     'font.size': 15,
111 })
112 
113 plt.figure(figsize=(8, 6))
114 im = plt.pcolormesh(X, Y, Z, shading='auto', cmap='viridis')
115 
116 if saveit:
117     plt.ylabel('Elevator rotation angle, $\delta_e$ [deg]')
118     plt.xlabel('Pitch angle, $\\theta$ [deg]')
119     plt.colorbar(im, label='Forces in $x$-axis, $F_x$ [kN]')
120     plt.savefig('aero_sample.png', dpi=300)
121 else:
122     plt.xlabel(sweep_pair[0].title())
123     plt.ylabel(sweep_pair[1].title())
124     plt.title("\n".join(wrap(f'{output_label} as function of {sweep_pair[0]}
        and {sweep_pair[1]}')))
125     plt.colorbar(im, label=f'{output_label} [kN]')
126 
127 plt.tight_layout()
128 plt.show()
```

plot_gramian_eigenvectors.py

```
  1 """Plot the eigenvectors of a 4x4 reachability Gramian.
  2 
  3 Eigenvectors are projected onto the first two state axes (V, alpha) and th
        e
  4 last two state axes (q, theta), matching the state ordering used throughou
        t
  5 this repo (see utils/interpolatory_model.py:build_AB).
  6 """
  7 
  8 import numpy as np
  9 import matplotlib.pyplot as plt
 10 from utils.read_hdf5 import extract_dataset_from_results
 11 from utils.gramian_metrics import gramian
 12 
 13 def gramian_eigs(gram):
 14     """Compute eigenvalues/eigenvectors of a symmetric 4x4 Gramian, sorted
         descending."""
 15     gram = np.asarray(gram, dtype=float)
 16     if gram.shape != (4, 4):
 17         raise ValueError(f"Expected a 4x4 Gramian, got shape {gram.shape}")
 18     if not np.allclose(gram, gram.T, atol=1e-8):
 19         raise ValueError("Gramian must be symmetric.")
```

```
20
21      eigvals, eigvecs = np.linalg.eigh(gram)
22      order = np.argsort(eigvals)[::-1]
23      return eigvals[order], eigvecs[:, order]
24
25
26  def plot_gramian_eigenvectors(gram, state_labels=(r'Velocity, $V$', r'Angl
        e of Attack, $\alpha$', r'Pitch Rate, $q$', r'Pitch Angle, $\theta$'),
27                                scale_by_eigenvalue=True):
28      """
29      Plot eigenvectors of a 4x4 Gramian projected onto axes (0,1) and (2,3)
        .
30
31      If scale_by_eigenvalue, each eigenvector is drawn with length sqrt(eig
        envalue),
32      which is the semi-axis length of the reachable ellipsoid {x : x^T W^-1
         x <= 1}
33      along that eigenvector direction.
34      """
35      eigvals, eigvecs = gramian_eigs(gram)
36      n = len(eigvals)
37      colors = plt.cm.viridis(np.linspace(0, 0.85, n))
38
39      plt.rcParams.update({'font.size': 15})
40
41      # fig, axes = plt.subplots(1, 2, figsize=(10, 8))
42      # pairs = [(0, 1), (2, 3)]
43
44      lengths = np.sqrt(eigvals) if scale_by_eigenvalue else np.ones(n)
45
46      # Only plot one
47
48      i = 2
49      j = 3
50      fig, ax = plt.subplots()
51      for k in range(n):
52          v = eigvecs[:, k]
53          vx, vy = v[i] * lengths[k], v[j] * lengths[k]
54
55          ax.annotate('', xy=(vx, vy), xytext=(0, 0),
56                      arrowprops=dict(arrowstyle='-|>', color=colors[k], lw=3))
57
58          # uncomment this to plot arrows both directions
59          # ax.annotate('', xy=(-vx, -vy), xytext=(0, 0),
60          #             arrowprops=dict(arrowstyle='-|>', color=colors[k], l
        w=1, alpha=0.4))
61          ax.plot([], [], color=colors[k], label=fr'$\lambda_{k+1}$ = {eigva
        ls[k]:.3g}')
62
63          # annotate() arrows aren't picked up by autoscale, so set limits m
        anually
64          extent = np.max(np.abs(eigvecs[[i, j], :] * lengths)) * 1.2
65          ax.set_xlim(-extent, extent)
66          ax.set_ylim(-extent, extent)
67
68          ax.axhline(0, color='k', lw=0.5, alpha=0.3)
69          ax.axvline(0, color='k', lw=0.5, alpha=0.3)
70          ax.set_xlabel(state_labels[i])
```

```
        ax.set_ylabel(state_labels[j])
        ax.set_aspect('equal', adjustable='box')
        ax.grid(True, alpha=0.3)
        ax.legend(loc='best')

    fig.tight_layout()
    return fig, ax


if __name__ == '__main__':

    # opt_path = 'Output/aero_7dim_asym/N6_initial'
    # x_opt = extract_dataset_from_results(f'{opt_path}/opt_vars.hdf5', 'x
    ')
    # x_opt = np.asarray([5, 12])
    x_opt = np.asarray([5, 12, 35, 4.9, 10])
    gram = gramian(x_opt, desired_vel=200)

    print('Gramian:')
    print(gram)

    eigvals, eigvecs = gramian_eigs(gram)
    print('Eigenvalues:', eigvals)
    print('Eigenvectors (columns):')
    print(eigvecs)

    saveit = False
    scale_plot = False

    fig, axes = plot_gramian_eigenvectors(gram, scale_by_eigenvalue=scale_
    plot)

    if saveit:
        fig.savefig('Output/5_dec_vars/Gramian_comparison/gramian_eigenvec
    tors.png', dpi=300)
        path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive
    -Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures'
        plt.savefig(f'{path_to_paper}/gramian_eigenvectors.png',dpi=300)
    else:
        axes.set_title(f'Eigenvector projection')
        plt.show()
```

process_plot_data.py

```
import numpy as np
from utils.reach_set_functions import compute_hull, plot_hull_projection
from utils.reach_set_data_read import unpack_set_data
import matplotlib.pyplot as plt

''' This script is for processing some of the data and making some plots''
    '

def plot_the_sets(opt_path,init_csv_file,saveit,projection_line=True, scat
    teron=False):

```

```
    N_res = 6
    plt.rc('xtick', labelsize=14)
    plt.rc('ytick', labelsize=14)

    model_type = opt_path.split('/')[-1][3:]
    print(f'Showing {model_type}')

    angle = np.deg2rad(90)
    v = np.array([[0,0,np.cos(angle),np.sin(angle)]]).T
    # v = np.array([[np.cos(np.deg2rad(angle)),np.sin(np.deg2rad(angle)),0
    ,0]]).T

    opt_csv_file = f'{opt_path}/bwb_inf_data.csv'

    l10, l20, l30, l40, fp0 = unpack_set_data(init_csv_file,N_res)
    l11, l21, l31, l41, fp1 = unpack_set_data(opt_csv_file,N_res)

    hull0 = compute_hull(fp0.reshape(-1,4))
    hull1 = compute_hull(fp1.reshape(-1,4))
    # hull2 = compute_hull(fp2)

    hull0v = hull0.volume
    hull1v = hull1.volume
    diff_volume = hull1v - hull0v

    hull_points0 = hull0.points[hull0.vertices]
    projections0 = hull_points0 @ v

    # Compute min and max position of projections along v
    proj_min, proj_max = projections0.min(), projections0.max()

    # Compute the actual vectors in case of plot
    extent0 = proj_max - proj_min

    hull_points1 = hull1.points[hull1.vertices]
    projections1 = hull_points1 @ v
    proj_min, proj_max = projections1.min(), projections1.max()
    extent1 = proj_max - proj_min
    print(f"Extent along direction {v} changes from: {extent0} to {extent1}")
    print(f'Percent increase: {100*(extent1-extent0)/extent0}')

    print(f"Volume of Reachable Set: {hull0v} to {hull1v}")
    print(f"Percent increase: {100*diff_volume/hull0v} %")

    states = ["Airspeed (m/s)","Angle of Attack (rad)","Pitch Rate (rad/s)","P
    itch Angle (rad)"]
    states_abbrev = ["speed","aoa","pitch_rate","pitch"]
    fsize = 18

    col1 = 2
    col2 = 3

    plt.figure(figsize=(8,6))
    plot_hull_projection(hull0,col_x=col1,col_y=col2,hull_plot_label="Init
    ial Set",point_label="Initial Interior",hull_color="b", ls='dashed',p
    oints=scatteron)
    plot_hull_projection(hull1,col_x=col1,col_y=col2,hull_plot_label="Opti
    mized Set",point_label="Optimized Interior",hull_color="r", ls='so
```

```
    lid',points=scatteron)

    plt.xlabel(states[col1],fontsize=fsize)
    plt.ylabel(states[col2],fontsize=fsize)
    # plt.title(f"Projection of Reachable Set onto {states[col1]} and {sta
    tes[col2]}",fontsize=fsize)

    if projection_line:
        hull = hull1

        # Choose projection axis
        axis = v
        # axis = np.asarray([[0,0,0.5,-0.8]]).T
        axis = axis/np.linalg.norm(axis)

        # Get the vertices of convex hull and project onto axis
        hull_points = hull.points[hull.vertices]
        projections = hull_points @ axis

        # Compute min and max position of projections along axis
        proj_min, proj_max = projections.min(), projections.max()

        # Compute the actual vectors in case of plot
        line_points = np.concatenate((proj_min * axis, proj_max * axis),ax
    is=1) # Saves min and max as [[x_min, y_min],[x_max, y_max]]
        line_points = line_points.T
        plt.plot(line_points[:,col1], line_points[:,col2], 'r', lw=5, labe
    l='Optimized Set Projection')

        '''Do same for hull0'''
        hull = hull0
        hull_points = hull.points[hull.vertices]
        projections = hull_points @ axis

        # Compute min and max position of projections along axis
        proj_min, proj_max = projections.min(), projections.max()

        # Compute the actual vectors in case of plot
        line_points = np.concatenate((proj_min * axis, proj_max * axis),ax
    is=1) # Saves min and max as [[x_min, y_min],[x_max, y_max]]
        line_points = line_points.T
        plt.plot(line_points[:,col1], line_points[:,col2], 'b', linestyle='d
    ashed', lw=3, label='Initial Set Projection')

    if col1 == 2:
        plt.legend(fontsize=fsize*0.8,loc='upper left')

    plt.grid(True)
    plt.axis('auto')
    if saveit:
        plt.axis('equal')
        plt.savefig(f"{opt_path}/{model_type}_reach_proj_{states_abbrev[co
    l1]}_{states_abbrev[col2]}.png",dpi=300)
        path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive
    -Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures/Reach
     Sets'
        plt.savefig(f'{path_to_paper}/{model_type}_reach_proj_{states_abbr
    ev[col1]}_{states_abbrev[col2]}.png',dpi=300)
```

```
111
112     else:
113         plt.axis('equal')
114         plt.show()
115
116     col1 = 0
117     col2 = 1
118
119     plt.figure(figsize=(8,6))
120     plot_hull_projection(hull0,col_x=col1,col_y=col2,hull_plot_label="Init
        ial Set",point_label="Initial Interior",hull_color="b", ls='dashed', p
        oints=scatteron)
121     plot_hull_projection(hull1,col_x=col1,col_y=col2,hull_plot_label="Opti
        mized Set",point_label="Optimized Interior",hull_color="r", ls='so
        lid', points=scatteron)
122
123     plt.xlabel(states[col1],fontsize=fsize)
124     plt.ylabel(states[col2],fontsize=fsize)
125
126     if projection_line:
127         hull = hull1
128
129         # Choose projection axis
130         axis = v
131         axis = axis/np.linalg.norm(axis)
132
133         # Get the vertices of convex hull and project onto axis
134         hull_points = hull.points[hull.vertices]
135         projections = hull_points @ axis
136
137         # Compute min and max position of projections along axis
138         proj_min, proj_max = projections.min(), projections.max()
139
140         # Compute the actual vectors in case of plot
141         line_points = np.concatenate((proj_min * axis, proj_max * axis),ax
        is=1) # Saves min and max as [[x_min, y_min],[x_max, y_max]]
142         line_points = line_points.T
143         plt.plot(line_points[:,col1], line_points[:,col2], 'r', lw=5, labe
        l='Optimized Set Projection')
144
145         '''Do same for hull0'''
146         hull = hull0
147         hull_points = hull.points[hull.vertices]
148         projections = hull_points @ axis
149
150         # Compute min and max position of projections along axis
151         proj_min, proj_max = projections.min(), projections.max()
152
153         # Compute the actual vectors in case of plot
154         line_points = np.concatenate((proj_min * axis, proj_max * axis),ax
        is=1) # Saves min and max as [[x_min, y_min],[x_max, y_max]]
155         line_points = line_points.T
156         plt.plot(line_points[:,col1], line_points[:,col2], 'b', linestyle='d
        ashed', lw=3, label='Initial Set Projection')
157
158     if col1 == 2:
159         plt.legend(fontsize=fsize,location='bottom left')
160
```

```
    plt.grid(True)
    plt.axis('auto')
    if saveit:
        plt.axis('equal')
        plt.savefig(f"{opt_path}/{model_type}_reach_proj_{states_abbrev[col1]}_{states_abbrev[col2]}.png",dpi=300)
        path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive-Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures/Reach Sets'
        plt.savefig(f'{path_to_paper}/{model_type}_reach_proj_{states_abbrev[col1]}_{states_abbrev[col2]}.png',dpi=300)

    else:
        plt.axis('equal')
        plt.show()

if __name__ == "__main__":
    opt_path1 = "Output/5_dec_vars/N6_volmax"
    opt_path2 = "Output/5_dec_vars/N6_115max"
    opt_path3 = "Output/5_dec_vars/N6_volmax_115con"

    models = [opt_path1, opt_path2, opt_path3]

    init_file = "Output/5_dec_vars/N6_initial/bwb_inf_data.csv"

    saveit = False
    proj = True
    show_points = False
    for file in models:
        plot_the_sets(file,init_file,saveit,projection_line = proj, scatteron = show_points)
```

utils/compute_inf_reach_set.py

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.linalg import expm
from scipy.integrate import trapezoid
import pandas as pd
from utils.progress_bar import printProgressBar
from utils.reach_set_functions import compute_hull, plot_hull_projection
from utils.reach_set_data_read import unpack_set_data
import matplotlib.pyplot as plt

'''This script will sample costate initial conditions and then simulate trajectories
corresponding to controls of sufficiently low cost. The final states, control costs,
and costate initial condition grid will be output into a csv file.'''

def compute_inf_reach_set_sphere(A,B,tf,N_res,u_ub = [1,1], u_lb = [-1,-1],save_data=False,filename="Output/reachability_data.csv"):
    '''This function computes the reachable set of a continuous-time LTI system
    , where the control is chosen to satisfy some magnitude bound from [0,
```

```
    T].
18
19      Inputs:
20      A, B: System matrices
21      tf: Final time
22      N_res: Resolution along each axis for the grid of costate initial cond
        itions
23      u_ub,u_lb: arrays that contain upper and lower bounds for inputs thrus
        t and elevator. u_ub = [thrust_max, elev_max]. u_lb = [thrust_min,
         elev_min].
24      save_data: optionally choose to write out data to csv
25      filename: Name of file to which reachability data is saved
26
27      Outputs:
28      control_cost: Array of control costs corresponding to each initial con
        dition
29      final_positions: Array of final states corresponding to each initial c
        ondition'''
30
31      np.random.seed(50)
32
33
34      m = B.shape[1]  # Number of control inputs
35      n = A.shape[0]  # Number of states
36      x0 = np.zeros(n)  # Initial condition
37
38      # Function to simulate the continuous-time LTI system
39      def sys_sim(t, x, u, A, B):
40          x = x.reshape(-1,1)
41          u = u.reshape(-1,1)
42          x_prime = A @x + B @ u
43          return x_prime.reshape(-1)     # Requires reshaping x because solv
        e_ivp wants that...
44
45      def inf_norm_control(A,B,t,lambda_0):
46          b = -B.T@expm(-A.T*t)@lambda_0
47          u = np.zeros(m)
48          for i,bi in enumerate(b):
49              u[i] = u_ub[i] if np.sign(bi) >= 0 else u_lb[i]
50          return u
51
52      def sample_sphere_boundary(num_samples,n):
53          x = np.random.randn(num_samples, n)
54          x /= np.linalg.norm(x, axis=1, keepdims=True)
55          return x
56
57      sample_num = N_res**n
58      lx = sample_sphere_boundary(sample_num,n)
59
60      control_cost = np.zeros((sample_num,1))
61      final_positions = np.zeros((sample_num,n))
62
63      for i in range(sample_num):
64          lambda0 = np.array(lx[i,:])
65          control_cost[i] = 1 # placeholder
66          # Simulate the system using solve_ivp
67          def ode_func(t, x):
68              u = inf_norm_control(A, B, t, lambda0)
```

```
            return sys_sim(t, x, u, A, B)

        sol = solve_ivp(ode_func, [0, tf], x0, t_eval=[tf])  # This value of t_eval makes it so we only store final point of traj.
        final_positions[i,:] = sol.y[:, -1]
        printProgressBar(i, sample_num-1, prefix = 'Progress:', suffix = 'Complete', length = 50)

    if save_data:
        # Create dynamic column names based on state dimension
        data = {}

        # Add costate initial conditions
        for i in range(n):
            data[f"LX{i+1}"] = lx[:, i]

        # Add final positions
        for i in range(n):
            data[f"final_x{i+1}"] = final_positions[:, i]


        df = pd.DataFrame(data)

        # Save to CSV
        df.to_csv(filename, index=False)

        print(f"\nData saved to {filename}")

    return control_cost, final_positions

if __name__ == "__main__":

    A = np.array([[2, 3],[-1, 1]])
    B = np.array([[0],[1]])
    # A= np.array([[-0.00352121133,  5.87480073,  0, -9.81],
    #              [-0.000681553965, -0.520980253,  1, 0],
    #              [0, -2.63577136,  0,  0],
    #              [ 0,  0,  1,  0]])
    # B = np.array([[2.84071645, 0.32619862],
    #               [0.        , 0.06223328],
    #               [0.        , 1.75401816],
    #               [0.        ,0.        ]])

    tf = 2
    N_res = 20

    control_cost, final_positions = compute_inf_reach_set_sphere(A, B, tf, N_res, save_data=False,filename='2d_mag.csv')
    hull = compute_hull(control_cost, final_positions, 1)
    plt.figure()
    fsize = 18
    plot_hull_projection(hull, col_x = 0, col_y = 1,hull_plot_label='Convex Hull', point_label='Sampled Trajectories')
    plt.xlabel("$x_1$", fontsize=fsize)
    plt.ylabel('$x_2$', fontsize=fsize)
    plt.legend(fontsize=0.6*fsize,loc='best')
    plt.show()

```

```
    # plt.savefig("/Users/stevennguyen/Library/CloudStorage/OneDrive-Personal/UCSD/Research/ACC2026/Presentation/test_set.png",dpi=300)
    # print(f"Computed Volume of Reachable Set: {hull.volume}")
```

utils/gramian_metrics.py

```
from scipy import integrate
from scipy.linalg import expm
from utils.interpolatory_model import *
import numpy as np
import csv
import math
from utils.read_hdf5 import extract_dataset_from_results
from utils.trim_aircraft import compute_trim_eom
from utils.reach_set_functions import plot_projected_boundary, sample_ellipsoid_surface

def gramian(x,desired_vel,tf=1):
    # First construct A,B matrices

    print(f"Design variables: {x}")

    chs = x[0]
    whs = x[1]
    if x.size < 3:
        data_file = 'Aero_data/aero_7dim2.npz'
        alpha0, deltae0, deltaT0, pitch0, _ = compute_trim_eom(np.asarray([desired_vel,chs,whs]), data_file,climb_angle=0)
    else:
        data_file = 'Aero_data/aero_merged_cg_17.npz'     # MAKE SURE TO PASS THE CORRECT NUMBER OF FLIGHT_POINTS TO compute_trim_eom WHEN USING THIS
        wsl = x[2]
        thsl = x[3]
        wdl = x[4]

        alpha0, deltae0, deltaT0, pitch0, _ = compute_trim_eom(np.asarray([desired_vel,chs,whs,wsl,thsl,wdl]), data_file,climb_angle=0)

    interp_dict = aero_interps(data_file)

    # print(f'Trimmed alpha: {np.rad2deg(alpha0)}, deltae: {np.rad2deg(deltae0)}, deltaT: {deltaT0}, pitch: {np.rad2deg(pitch0)}')

    # Compute input bounds
    thrust_ub = 1-deltaT0
    thrust_lb = -deltaT0
    elev_ub = np.deg2rad(30) - deltae0
    elev_lb = np.deg2rad(-30) - deltae0

    u_upper = np.array([thrust_ub, elev_ub])
    u_lower = np.array([thrust_lb, elev_lb])
```

```
41
42      flight_point = np.array([desired_vel, alpha0, deltae0])
43
44      full_flight_point = np.concatenate((flight_point[0:2], x[0:2], flight_
        point[2:],x[2:]))
45      A,B = build_AB(interp_dict,full_flight_point)
46      # print(f'A: {A}')
47      # print(f'B: {B}')
48      # Use A, B matrices to compute the reach gramian
49      def gram_integrand(t,A,B,tf):
50          return expm(A * (tf-t)) @ B @ np.transpose(B) @ expm(np.transpose(A) * (t
        f-t))
51      gram, err = integrate.quad_vec(gram_integrand, 0, tf, args=(A,B,tf)) #
        Integrate the gramian integrand from 0 to 1
52      return gram
53
54  def gramian_hull(x,desired_vel, tf=1,seed=50,savepts=False,filename="Outpu
        t/gramian_set.csv"):
55      gram = gramian(x,desired_vel,tf)
56      myrng = np.random.default_rng(seed)
57      X = sample_ellipsoid_surface(np.linalg.inv(gram),gamma=1,N=1000,rng=my
        rng,savepts=savepts,filename=filename) # This pulls the convexhull
         associated with sampled reachableset for L2 norm.
58      return X
59
60
61  def gramian_eig_obj(x,desired_vel,eigs):
62      '''This objective will maximize the eigenvalues of reach gramian.
63      Input eigs gives a np array of the eigenvalues we care about.'''
64      gram = gramian(x,desired_vel,tf=1)
65
66      # eigenvalues are:
67      vals, vecs = np.linalg.eig(gram)
68
69      print(f'Eigenvalues: {vals}')
70      print(f'Eigenvectors: {vecs}')
71      # If we assume that the first 2 eigenvalues should be minimized and we
         ignore the last 2.
72      # This is because the last two eigenvectors align most with the pitch
        angle axis.
73      print(f'The sum: {np.sum(vals[eigs]) - np.sum(np.delete(vals,eigs))}')
74
75      return -np.sum(vals[eigs]) + np.sum(np.delete(vals,eigs))
76
77  def gramian_extent(x,v,desired_vel):
78      hull = gramian_hull(x,desired_vel,tf=1)
79      axis = v/np.linalg.norm(v)
80      hull_points = hull.points[hull.vertices]
81      projections = hull_points @ axis
82
83      # Compute min and max position of projections along axis
84      proj_min, proj_max = projections.min(), projections.max()
85
86      # Compute the actual vectors in case of plot
87      line_points = np.concatenate((proj_min * axis, proj_max * axis),axis=1) #
        Saves min and max as [x_min, x_max]
88      extent = proj_max - proj_min
89      print(f"Extent along direction {axis}: {extent}")
```

```
    return extent

def gramian_direction_obj(x,v,fp):
    print(f'Design variables: {x}')
    extent = gramian_extent(x,v,fp)
    return -extent
```

utils/interpolatory_model.py

```
import numpy as np
import matplotlib.pyplot as plt
from textwrap import wrap
from scipy.interpolate import RegularGridInterpolator
import math


#%% Build function to construct interpolators

def aero_interps(data_file):
    ''' This function will take in the name of the data .npz file and outp
    ut
    interpolators to query the tabulated data at various points'''

    aero_arrays = np.load(data_file)

    PL = aero_arrays['PL']
    WHSL = aero_arrays['WHSL']
    CHSL = aero_arrays['CHSL']
    # AL = aero_arrays['AL']
    ERL = aero_arrays['ERL']
    ML = aero_arrays['ML']
    if 'WSL' in aero_arrays:
        extra_dec_vars = True
    else:
        extra_dec_vars = False

    if extra_dec_vars:
        WSL = aero_arrays['WSL'] # wing sweep list
        THSL = aero_arrays['THSL'] # transition half span list
        WDL = aero_arrays['WDL'] # wing dihedral list

    X_v = aero_arrays['X_v']
    X_alpha = aero_arrays['X_alpha']
    X_deltae = aero_arrays['X_deltae']
    Z_v = aero_arrays['Z_v']
    Z_alpha = aero_arrays['Z_alpha']
    Z_deltae = aero_arrays['Z_deltae']
    M_v = aero_arrays['M_v']
    M_alpha = aero_arrays['M_alpha']
    M_deltae = aero_arrays['M_deltae']

    Fx = aero_arrays['Fx_mat']
    Fz = aero_arrays['Fz_mat']
    M = aero_arrays['M_mat']
    CL = aero_arrays['CL_mat']
```

```
46      Di = aero_arrays['Di']
47      L = aero_arrays['L']
48      Cdi = aero_arrays['Cdi_mat']
49
50      # Generate the 6 vectors passed into meshgrid.
51      shape = PL.shape
52      # print(shape)
53      nparams = len(shape)
54      sweep_resolution = shape[0] #Assume grid same resolution all axes
55      # print(sweep_resolution)
56      # print(extra_dec_vars)
57
58      if extra_dec_vars:
59          ''' Using additional 3 design variables, all have same resolution
        except for wing dihedral.'''
60          speed_list = np.linspace(ML[0,0,0,0,0,0,0,0],  ML[-1,0,0,0,0,0,0,0],s
        weep_resolution)
61          pitch_list = np.linspace(PL[0,0,0,0,0,0,0,0],  PL[0,-1,0,0,0,0,0,0],s
        weep_resolution)
62          chs_list = np.linspace(CHSL[0,0,0,0,0,0,0,0],CHSL[0,0,-1,0,0,0,0,0],s
        weep_resolution)
63          whs_list = np.linspace(WHSL[0,0,0,0,0,0,0,0],WHSL[0,0,0,-1,0,0,0,0],s
        weep_resolution)
64          er_list = np.linspace(  ERL[0,0,0,0,0,0,0,0], ERL[0,0,0,0,-1,0,0,0],s
        weep_resolution)
65          ws_list = np.linspace(  WSL[0,0,0,0,0,0,0,0], WSL[0,0,0,0,0,-1,0,0],s
        weep_resolution)
66          ths_list = np.linspace(THSL[0,0,0,0,0,0,0,0],THSL[0,0,0,0,0,0,-1,0],s
        weep_resolution)
67          wd_list = np.linspace(  WDL[0,0,0,0,0,0,0,0], WDL[0,0,0,0,0,0,0,-1],2)
68          interp_grid = (speed_list, pitch_list, chs_list, whs_list, er_list, w
        s_list, ths_list, wd_list)
69
70      else:
71          ''' Below is the approach used for 2 design variables in the AIAA
        paper.'''
72          speed_list = np.linspace(ML[0,0,0,0,0],ML[-1,0,0,0,0],sweep_resolu
        tion)
73          pitch_list = np.linspace(PL[0,0,0,0,0],PL[0,-1,0,0,0],sweep_resolu
        tion)
74          chs_list = np.linspace(CHSL[0,0,0,0,0],CHSL[0,0,-1,0,0],sweep_reso
        lution)
75          whs_list = np.linspace(WHSL[0,0,0,0,0],WHSL[0,0,0,-1,0],sweep_reso
        lution)
76          er_list = np.linspace(ERL[0,0,0,0,0],  ERL[0,0,0,0,-1],sweep_resol
        ution)
77          interp_grid = (speed_list, pitch_list, chs_list, whs_list, er_list)
78      # print(interp_grid)
79      xv_interp = RegularGridInterpolator(interp_grid, X_v)
80      xalpha_interp = RegularGridInterpolator(interp_grid, X_alpha)
81      xdeltae_interp = RegularGridInterpolator(interp_grid, X_deltae)
82      zv_interp = RegularGridInterpolator(interp_grid, Z_v)
83      zalpha_interp = RegularGridInterpolator(interp_grid, Z_alpha)
84      zdeltae_interp = RegularGridInterpolator(interp_grid, Z_deltae)
85      mv_interp = RegularGridInterpolator(interp_grid, M_v)
86      malpha_interp = RegularGridInterpolator(interp_grid, M_alpha)
87      mdeltae_interp = RegularGridInterpolator(interp_grid, M_deltae)
88
```

```
    Fx_interp = RegularGridInterpolator(interp_grid, Fx)
    Fz_interp = RegularGridInterpolator(interp_grid, Fz)
    M_interp = RegularGridInterpolator(interp_grid, M)
    CL_interp = RegularGridInterpolator(interp_grid, CL)
    Di_interp = RegularGridInterpolator(interp_grid, Di)
    L_interp = RegularGridInterpolator(interp_grid, L)
    Cdi_interp = RegularGridInterpolator(interp_grid, Cdi)

    interp_dict = {'xv':xv_interp, 'xalpha':xalpha_interp, 'xdeltae':xdeltae_interp,
                   'zv':zv_interp, 'zalpha':zalpha_interp, 'zdeltae':zdeltae_interp,
                   'mv':mv_interp, 'malpha':malpha_interp, 'mdeltae':mdeltae_interp,

                   'Fx':Fx_interp, 'Fz':Fz_interp, 'M':M_interp, 'CL':CL_interp,
                   'Di':Di_interp, 'L':L_interp, 'Cdi':Cdi_interp}

    return interp_dict

def build_AB(interpolators, flight_point):
    ''' This function will take in the interpolators and return the A,B matrices
    in our linear-longitudinal model.

    Inputs: interpolators is a dictionary of the interpolators created in deriv_interps
            flight_point is an array of the 6 state/control variables at the flight point.

            More specifically, flight_point gives [V, alpha, chs, whs, er]

            In a lot of my examples, we assume 0 flight path angle, so alpha = pitch. But really,
            The angle of attack is what the panel method code cares about, and that's what should be passed
            into the interpolators.

    Outputs: A,B matrices at the selected point'''

    X_v = interpolators['xv'](flight_point)
    X_alpha = interpolators['xalpha'](flight_point)
    X_deltae = interpolators['xdeltae'](flight_point)
    Z_v = interpolators['zv'](flight_point)
    Z_alpha = interpolators['zalpha'](flight_point)
    Z_deltae = interpolators['zdeltae'](flight_point)
    M_v = interpolators['mv'](flight_point)
    M_alpha = interpolators['malpha'](flight_point)
    M_deltae = interpolators['mdeltae'](flight_point)

    Z_q = 0 # cannot take derivs w.r.t. pitch rate
    M_q = 0

    A = np.zeros((4, 4))
    B = np.zeros((4, 2)) # two inputs: thrust and elevator deflection

    g = 9.81  # m/s^2
```

```
    gamma_0 = 0  # Flight path angle at trim (radians), we assume level fl
    ight
    V_0 = flight_point[0]  # Trim airspeed (m/s)

    A[0, 0] = X_v
    A[0, 1] = X_alpha
    A[0, 2] = 0
    A[0, 3] = -g * math.cos(gamma_0)

    A[1, 0] = Z_v / V_0
    A[1, 1] = Z_alpha/ V_0
    A[1, 2] = 1 + Z_q/ V_0
    A[1,3] = -g * math.sin(gamma_0) / V_0

    A[2, 0] = M_v
    A[2, 1] = M_alpha
    A[2, 2] = M_q
    A[2, 3] = 0

    A[3,:] = np.array([0, 0, 1, 0])

    thrust_force = 96000 # lb
    thrust_force = thrust_force * 4.44822 # convert to N
    weight = 331522 # lb
    weight = weight * 4.44822 # convert to N
    mass = weight / g  # kg

    B[0,0] = thrust_force/mass # for thrust input from 0 to 1, 1 is maximu
    m acceleration forward.
    B[1:,0] = np.zeros((3,))
    B[0, 1] = X_deltae
    B[1, 1] = Z_deltae / V_0
    B[2, 1] = M_deltae
    B[3, 1] = 0

    return A,B


if __name__ == "__main__":
    data_file = 'Aero_data/aero_7dim2.npz'
    interp_dict = aero_interps(data_file)
    print(interp_dict)


```

---

utils/my_rk4.py

```
import numpy as np
import matplotlib.pyplot as plt

def rk4_step(f, t, y, h):
    """One step of classical RK4 for y' = f(t, y).

    Parameters
    ----------
```

```
    f : callable
        RHS f(t, y) -> ndarray of same shape as y.
    t : float
        Current time.
    y : ndarray
        Current state.
    h : float
        Step size.

    Returns
    -------
    y_next : ndarray
        State at t + h.
    """
    k1 = f(t,           y)
    k2 = f(t + 0.5*h,   y + 0.5*h*k1)
    k3 = f(t + 0.5*h,   y + 0.5*h*k2)
    k4 = f(t + h,       y + h*k3)
    return y + (h / 6.0) * (k1 + 2.0*k2 + 2.0*k3 + k4)


def rk4_integrate(f, t_span, y0, h):
    """Integrate y' = f(t, y) on t_span = (t0, tf) with fixed step h.

    The final step is shortened if (tf - t0) is not a multiple of h, so th
    e
    last grid point lands exactly at tf.

    Returns
    -------
    t : (N,) ndarray
        Time grid.
    Y : (N, d) ndarray
        State trajectory, Y[k] = y(t[k]).
    """
    t0, tf = t_span
    if h <= 0 or tf <= t0:
        raise ValueError("Require h > 0 and tf > t0.")

    y0 = np.atleast_1d(np.asarray(y0, dtype=float))
    n_full = int(np.floor((tf - t0) / h))
    h_last = (tf - t0) - n_full * h
    n_steps = n_full + (1 if h_last > 0 else 0)

    t = np.empty(n_steps + 1)
    Y = np.empty((n_steps + 1, y0.size))
    t[0], Y[0] = t0, y0

    for k in range(n_full):
        Y[k+1] = rk4_step(f, t[k], Y[k], h)
        t[k+1] = t[k] + h
    if h_last > 0:
        Y[-1] = rk4_step(f, t[-2], Y[-2], h_last)
        t[-1] = tf

    return t, Y

if __name__ == "__main__":
```

```
    def myode(t,x):
        A = np.array([[0, 1],[-5, -2]])
        return A@x
    tf = 5
    x0 = np.array([2, 5])
    t2, X = rk4_integrate(myode, (0.0, tf), x0, h=1e-2)

    plt.plot(X[:,0],X[:,1])
    plt.grid()
    plt.xlabel('x1')
    plt.ylabel('x2')
    plt.show()
```

utils/nonlinear_model.py

```
''' This script will be a nonlinear model from Stevens/Lewis'''
from utils.interpolatory_model import *
import numpy as np
from utils.trim_aircraft import compute_trim_eom
import control as ct
from scipy.integrate import solve_ivp
import matplotlib.pyplot as plt
from scipy.integrate import trapezoid
from utils.read_hdf5 import extract_dataset_from_results

# def xref(t):
#     '''This function will return a reference trajectory to be followed.
#     Did sims for reference of 204 m/s, an increase from 200 m/s initial.

#     Works for up to 208 m/s. At 209, some states go out of bound (I think AOA...)'''
#     v_ref = 204  # m/s
#     return v_ref

# def run_nonlinear(fp,data_file,tf):
#     ''' This code will simulate nonlinear steady level flight. However, the input is not coded to actually
#     do any special control. It simply holds the trimmed inputs. Not very useful...
#     Inputs: fp,             [velocity, chs, whs] flight point
#             data_file,      file from which to take aero data interpolators
#             tf,             final simulation time
#     Outputs: t,x,u_hist'''

#     # Using given flight point, compute trimmed angle of attack and inputs.
#     # alpha0, deltae0, deltaT0 = compute_trim_forces(fp, data_file)
#     # pitch0 = alpha0

#     alpha0, deltae0, deltaT0, pitch0 = compute_trim_eom(fp,data_file,climb_angle=0)

#     # this input simply holds the inputs at their trimmed values.
#     u = np.array([deltaT0,deltae0])
```

```
34 |
35 | #     # Trim point aka initial condition of simulation
36 | #     x0 = np.array([v0, alpha0, 0, pitch0])     # initial state, with int
       egrator state appended
37 |
38 | #     # some constants
39 | #     g = 9.81  # m/s^2
40 | #     weight = 331522 # lb
41 | #     weight = weight * 4.44822 # convert to N
42 | #     mass = weight / g  # kg
43 | #     Iy = 3759014 # slug ft^2
44 | #     Iy = Iy * 14.5939 * 0.3048**2 # kg m^2
45 | #     thrust_force = 96000 # lb
46 | #     maxT = thrust_force * 4.44822 * 1.5 # convert to N
47 |
48 | #     interp_dict = aero_interps(data_file)
49 | #     M_interp = interp_dict['M']
50 | #     Di_interp = interp_dict['Di']
51 | #     L_interp = interp_dict['L']
52 |
53 | #     def controller(t,x):
54 | #         # if t > 2:
55 | #         #     return u + np.array([0.5,0])
56 | #         return u
57 |
58 | #     def nonlinear_model(t, x,u):
59 | #         v = x[0]
60 | #         alpha = x[1]
61 | #         q = x[2]
62 | #         theta = x[3]
63 |
64 | #         deltae = u[1] # if t < 2 else u[1] + np.deg2rad(2) # add 5 deg e
       levator deflection after 2 seconds
65 | #         deltaT = u[0] # if t < 2 else u[0] + 0.5
66 |
67 | #         flight_point = np.asarray([v, alpha, chs, whs, deltae])
68 |
69 | #         M = M_interp(flight_point)[0]
70 | #         Di = Di_interp(flight_point)[0]
71 | #         L = L_interp(flight_point)[0]
72 |
73 | #         Ft = deltaT * maxT
74 |
75 | #         x_dot = np.zeros_like(x)
76 | #         x_dot[0] = Ft/mass * np.cos(alpha) - Di/mass - g * np.sin(theta
       - alpha)
77 | #         x_dot[1] = -Ft/(mass * v) * np.sin(alpha) - L/(mass * v) + g/v *
        np.cos(theta - alpha) + q
78 | #         x_dot[2] = M / Iy
79 | #         x_dot[3] = q
80 |
81 | #         return x_dot
82 |
83 | #     sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x,controller(t,x))
       , (0.0, tf), x0, method='BDF')
84 | #     if not sol_fwd.success:
85 | #         raise RuntimeError("Forward integration failed: " + str(sol_fwd.
       message))
```

```
 86 |
 87 | #     t = sol_fwd.t
 88 | #     x = sol_fwd.y
 89 |
 90 | #     u_hist = np.zeros((2,len(t)))
 91 |
 92 | #     u_hist = np.asarray([controller(ti,x[:,i]) for i,ti in enumerate(t)]
    |     )
 93 | #     u_hist = u_hist.T
 94 |
 95 | #     return t,x,u_hist
 96 |
 97 | # def run_linear(fp,data_file,tf):
 98 | #     ''' Run steady level flight.'''
 99 | #     v0 = fp[0]
100 | #     chs = fp[1]
101 | #     whs = fp[2]
102 | #     alpha0, deltae0, deltaT0, pitch0 = compute_trim_eom(fp, data_file,cl
    |     imb_angle=10)
103 | #     full_flight_point = np.array([v0,alpha0,chs,whs,deltae0])
104 | #     trim_point = np.array([v0,alpha0,0,pitch0])
105 | #     trim_inputs = np.array([deltaT0,deltae0])
106 | #     interp_dict = aero_interps(data_file)
107 | #     A,B = build_AB(interp_dict,full_flight_point)
108 |
109 | #     def linear_model(t,x,control):
110 | #         x = x-trim_point
111 | #         u = control - trim_inputs
112 | #         x_dot = np.zeros((4,1))
113 | #         x_dot = A@x + B@u
114 | #         return x_dot
115 |
116 | #     u = np.array([deltaT0,deltae0])
117 | #     sol = solve_ivp(lambda t,x: linear_model(t,x,u)  , (0.0, tf), trim_p
    |     oint)
118 | #     t = sol.t
119 | #     x = sol.y
120 | #     u_hist = np.zeros((2,len(t)))
121 | #     u_hist[0,:] = np.ones(len(t)) * deltaT0
122 | #     u_hist[1,:] = np.ones(len(t)) * np.rad2deg(deltae0)
123 | #     xr_hist = np.zeros((4,len(t)))
124 |
125 | #     return t,x,u_hist, xr_hist
126 |
127 | # def linear_track(fp,data_file,tf):
128 | #     '''This function will do linearized integral tracking control in vel
    |     ocity.
129 | #     This code is not as robust as the other one for comparing performanc
    |     e, but use this to compare with the nonlinear model.
130 | #     The input bounding is more correct here and we apply the linear mode
    |     l more "corectly" since we treat it as a perturbation
131 | #     model.'''
132 |
133 | #     v0 = fp[0]
134 | #     chs = fp[1]
135 | #     whs = fp[2]
136 | #     alpha0, deltae0, deltaT0, pitch0 = compute_trim_eom(fp, data_file,cl
    |     imb_angle=0)
```

```
137 #     full_flight_point = np.array([v0,alpha0,chs,whs,deltae0])
138 #     trim_point = np.array([v0,alpha0,0,pitch0])
139 #     print(trim_point)
140 #     trim_inputs = np.array([deltaT0,deltae0])
141 #     interp_dict = aero_interps(data_file)
142 #     A,B = build_AB(interp_dict,full_flight_point)
143 #     C = np.array([[1,0,0,0]])
144 #
145 #     # Take integral reference tracking code but modify slightly.
146 #
147 #     k = C.shape[0]
148 #     n = A.shape[0]
149 #     m = B.shape[1]
150 #
151 #     Abar = np.concatenate((np.concatenate((A,np.zeros((n,k))),axis=1),np
    .concatenate((C,np.zeros((k,k))),axis=1)),axis=0)
152 #     Bbar = np.concatenate((B,np.zeros((k,m))),axis=0)
153 #
154 #     Q = np.diag([1,1,1,1,1])
155 #     R = np.diag([0.1,1000])
156 #
157 #     K = ct.lqr(Abar,Bbar,Q,R)[0]
158 #     def control_signal(x):
159 #         x = x.reshape((n+1,1))
160 #         u = -K@x
161 #
162 #         # In order to make sure we have the right input bounds, we add t
    he value of trimmed inputs
163 #         # apply the bounds, then remove the trimmed value again to get t
    he perturbed inputs.
164 #
165 #         u_actual = u + trim_inputs.reshape(2,-1)
166 #         u_actual[0] = np.clip(u_actual[0],0,1)
167 #         u_actual[1] = np.clip(u_actual[1],np.deg2rad(-30),np.deg2rad(30)
    )
168 #         u = u_actual - trim_inputs.reshape(2,-1)
169 #
170 #         return u
171 #
172 #     t_eval_fwd = np.linspace(0.0, tf, 1001)
173 #
174 #     def closed_loop_forward(t, x_flat):
175 #         x = x_flat.reshape((n+1,1))
176 #         u = control_signal(x)
177 #         # xr = xref(t) - trim_point[0]
178 #         xr = 0
179 #         dx = Abar @ x + Bbar @ u - np.concatenate((np.zeros((4,1)),np.as
    array([[xr]])),axis=0)
180 #
181 #         if x[4] > 50:      # prevent wind up, if integrator state too la
    rge, keep constant...
182 #             dx[4] = min(dx[4],0)
183 #         elif x[4] < -50:
184 #             dx[4] = max(dx[4],0)
185 #         return dx.flatten()
186 #
187 #     x0 = np.zeros((n+1,1)).flatten()
188 #     x0 = x0 + np.array([-10,0,0,0,0])
```

```
#     sol_fwd = solve_ivp(closed_loop_forward, (0.0, tf), x0, t_eval=t_eva
    l_fwd)

#     t = sol_fwd.t
#     y = sol_fwd.y
#     x = y[:n,:]
#     # x[0,:] = x[0,:] + np.ones(y.shape[1])*trim_point[0]
#     # x[1,:] = x[1,:] + np.ones(y.shape[1])*trim_point[1]
#     # x[2,:] = x[2,:] + np.ones(y.shape[1])*trim_point[2]
#     # x[3,:] = x[3,:] + np.ones(y.shape[1])*trim_point[3]

#     u_hist = np.zeros((m,t.size))
#     for i, ti in enumerate(t):
#         u = control_signal(y[:,i].reshape((n+1,1)))
#         u_hist[:,i] = u.flatten() + trim_inputs
#     print('here')

#     return t,y, u_hist

def plotit(t,x,u,ref=None,bound=False,save=False,model=None):
    plt.figure(figsize=(12, 12))  # Adjust figure size to accommodate more
     subplots (width, height)
    plt.rcParams.update({'font.size': 15})
    plt.rcParams['lines.linewidth'] = 4
    color_ref = 'orange'

    # Plot 1: Airspeed
    plt.subplot(5, 1, 1)
    plt.plot(t, x[0, :], label='System trajectory')
    plt.ylabel('Airspeed [m/s]')
    plt.legend(loc='best')  # Set legend location to center right
    if bound:
        plt.ylim((220,260))
    if ref is not None:
        plt.plot(t, ref[0,:], label='Reference trajectory', linestyle='dot
    ted', color=color_ref)
        plt.legend(loc='best')  # Set legend location to center right
    plt.grid(True)

    # Plot 2: Angle of Attack
    plt.subplot(5,1,2)
    plt.plot(t, np.rad2deg(x[1, :]))
    if bound:
        plt.ylim((-5,15))
    if ref is not None:
        plt.plot(t, ref[1,:], label='Reference trajectory', linestyle='dot
    ted', color=color_ref)
    plt.ylabel('Angle of Attack [deg]')
    plt.grid(True)

    # Plot 3: Pitch Rate
    plt.subplot(5, 1, 3)
    plt.plot(t, np.rad2deg(x[2, :]))
    if bound:
        plt.ylim((-2,2))
    if ref is not None:
        plt.plot(t, ref[2,:], label='Reference trajectory', linestyle='dot
    ted', color=color_ref)
```

```
242     plt.ylabel('Pitch Rate [deg/s]')
243     plt.grid(True)
244 
245     # Plot 4: Pitch angle
246     plt.subplot(5, 1, 4)
247     plt.plot(t, np.rad2deg(x[3, :]))
248     if bound:
249         plt.ylim((0,20))
250     if ref is not None:
251         plt.plot(t, ref[3,:], label='Reference trajectory', linestyle='dot
    ted', color=color_ref)
252     plt.ylabel('Pitch angle [deg]')
253     plt.grid(True)
254 
255     # Plot 5: Control input
256     # only one input...
257     plt.subplot(5, 1, 5)
258     plt.plot(t, u[0, :], label='thrust', color='red')
259     plt.plot(t, u[1, :], label='elevator [rad]', color='purple')
260 
261     # plt.plot(u_list.t_list, u_hist[0, :], label='thrust', color='red')
262     # plt.plot(u_list.t_list, u_hist[1, :], label='elevator [deg]', color=
    'purple')
263     plt.xlabel('time [ts]')
264     plt.ylabel('Control')
265     # plt.title('Control')
266     plt.legend(loc='center right')  # Set legend location to center right
267     plt.grid(True)
268 
269 
270     error = x[0:4,:] - ref
271     vel_err =   np.sqrt(trapezoid(error[0,:]**2,t))
272     aoa_err =   np.sqrt(trapezoid(error[1,:]**2,t))
273     pr_err =    np.sqrt(trapezoid(error[2,:]**2,t))
274     pitch_err = np.sqrt(trapezoid(error[3,:]**2,t))
275 
276     error_norm = np.sqrt(trapezoid(np.sum(error**2,0),t))
277     print(f'Total error: {error_norm}')
278     print(f'Velocity error: {vel_err}, AOA error: {aoa_err}, Pitch rate er
    ror: {pr_err}, Pitch angle error: {pitch_err}')
279 
280     # Adjust layout and show the figure
281     plt.tight_layout()
282     # plt.text(0.5, -0.5, f'L2 norm of error is {error_norm:.3f}')
283     if save:
284         plt.savefig(f'Output/aero_7dim_asym/{model}_nonlinear_track.png', d
    pi=300)
285     else:
286         plt.show()
287 
288 # def niceplot(sim_dictionary,save=False,model=None):
289 #     ''' This function makes a nicer looking plot for the simulation traj
    ectories with all models
290 #     and also plots error bars to compare'''
291 
292 #     plt.rcParams.update({'font.size': 15})
293 #     plt.rcParams['lines.linewidth'] = 4
294 #     plt.rcParams["legend.handlelength"] = 3.5
```

```
295 
296 #     t_init = sim_dictionary['initial']['t']
297 #     x_init = sim_dictionary['initial']['x']
298 #     u_init = sim_dictionary['initial']['u']
299 #     tp_init = sim_dictionary['initial']['tp']
300 #     style_init = sim_dictionary['initial']['style']
301 #     color_init = sim_dictionary['initial']['color']
302 
303 #     t_ref = sim_dictionary['ref']['t']
304 #     x_ref = sim_dictionary['ref']['x']
305 #     style_ref = sim_dictionary['ref']['style']
306 #     color_ref = sim_dictionary['ref']['color']
307 
308 #     t_vm = sim_dictionary['vm']['t']
309 #     x_vm = sim_dictionary['vm']['x']
310 #     u_vm = sim_dictionary['vm']['u']
311 #     tp_vm = sim_dictionary['vm']['tp']
312 #     style_vm = sim_dictionary['vm']['style']
313 #     color_vm = sim_dictionary['vm']['color']
314 
315 #     t_dm = sim_dictionary['dm']['t']
316 #     x_dm = sim_dictionary['dm']['x']
317 #     u_dm = sim_dictionary['dm']['u']
318 #     tp_dm = sim_dictionary['dm']['tp']
319 #     style_dm = sim_dictionary['dm']['style']
320 #     color_dm = sim_dictionary['dm']['color']
321 
322 #     t_vmdc = sim_dictionary['vmdc']['t']
323 #     x_vmdc = sim_dictionary['vmdc']['x']
324 #     u_vmdc = sim_dictionary['vmdc']['u']
325 #     tp_vmdc = sim_dictionary['vmdc']['tp']
326 #     style_vmdc = sim_dictionary['vmdc']['style']
327 #     color_vmdc = sim_dictionary['vmdc']['color']
328 
329 #     # Plot 1: Airspeed
330 #     plt.subplot(5, 1, 1)
331 #     plt.plot(t_init, x_init[0, :], label='Initial', color=color_init, li
    nestyle=style_init)
332 #     plt.plot(t_vm, x_vm[0, :], label='VM', color = color_vm, linestyle=s
    tyle_vm)
333 #     plt.plot(t_dm, x_dm[0, :], label='DM', color=color_dm, linestyle=sty
    le_dm)
334 #     plt.plot(t_vmdc, x_vmdc[0, :], label='VMDC', color=color_vmdc, lines
    tyle=style_vmdc)
335 #     plt.legend(loc='best')  # Set legend location to center right
336 #     plt.plot(t_ref, x_ref[0,:], label='Reference trajectory', linestyle=
    style_ref, color=color_ref)
337 #     plt.legend(loc='best')  # Set legend location to center right
338 #     plt.grid(True)
339 #     plt.ylabel('Airspeed [m/s]')
340 
341 #     # Plot 2: Angle of Attack
342 #     plt.subplot(5,1,2)
343 #     plt.plot(t_init, np.rad2deg(x_init[1, :]), label='Initial', color=co
    lor_init, linestyle=style_init)
344 #     plt.plot(t_vm,   np.rad2deg(x_vm[1, :]), label='VM', color = color_v
    m, linestyle=style_vm)
345 #     plt.plot(t_dm,   np.rad2deg(x_dm[1, :]), label='DM', color=color_dm,
```

```
     linestyle=style_dm)
#     plt.plot(t_vmdc, np.rad2deg(x_vmdc[1, :]), label='VMDC', color=color_vmdc, linestyle=style_vmdc)
#     plt.plot(t_ref,  x_ref[1,:], label='Reference trajectory', linestyle=style_ref, color=color_ref)
#     plt.grid(True)
#     plt.ylabel('Angle of Attack [deg]')
#
#     # Plot 3: Pitch Rate
#     plt.subplot(5, 1, 3)
#     plt.plot(t_init, np.rad2deg(x_init[2, :]), label='Initial', color=color_init, linestyle=style_init)
#     plt.plot(t_vm,   np.rad2deg(x_vm[2, :]), label='VM', color = color_vm, linestyle=style_vm)
#     plt.plot(t_dm,   np.rad2deg(x_dm[2, :]), label='DM', color=color_dm,
     linestyle=style_dm)
#     plt.plot(t_vmdc, np.rad2deg(x_vmdc[2, :]), label='VMDC', color=color_vmdc, linestyle=style_vmdc)
#     plt.plot(t_ref,  x_ref[2,:], label='Reference trajectory', linestyle=style_ref, color=color_ref)
#     plt.grid(True)
#     plt.ylabel('Pitch Rate [deg/s]')
#
#     # Plot 4: Pitch angle
#     plt.subplot(5, 1, 4)
#     plt.plot(t_init, np.rad2deg(x_init[3, :]), label='Initial', color=color_init, linestyle=style_init)
#     plt.plot(t_vm,   np.rad2deg(x_vm[3, :]), label='VM', color = color_vm, linestyle=style_vm)
#     plt.plot(t_dm,   np.rad2deg(x_dm[3, :]), label='DM', color=color_dm,
     linestyle=style_dm)
#     plt.plot(t_vmdc, np.rad2deg(x_vmdc[3, :]), label='VMDC', color=color_vmdc, linestyle=style_vmdc)
#     plt.plot(t_ref,  x_ref[3,:], label='Reference trajectory', linestyle=style_ref, color=color_ref)
#     plt.ylabel('Pitch angle [deg]')
#     plt.grid(True)
#
#     # Plot 5: Control input
#     # only one input...
#     plt.subplot(5, 1, 5)
#     plt.plot(t_init, u_init[0, :], label='thrust', color=color_init, linestyle=style_init)
#     plt.plot(t_init, u_init[1, :], label='elevator [rad]', color=color_init, linestyle=style_init)
#     plt.plot(t_vm, u_vm[0, :], label='thrust', color=color_vm, linestyle=style_vm)
#     plt.plot(t_vm, u_vm[1, :], label='elevator [rad]', color=color_vm, linestyle=style_vm)
#     plt.plot(t_dm, u_dm[0, :], label='thrust', color=color_dm, linestyle=style_dm)
#     plt.plot(t_dm, u_dm[1, :], label='elevator [rad]', color=color_dm, linestyle=style_dm)
#     plt.plot(t_vmdc, u_vmdc[0, :], label='thrust', color=color_vmdc, linestyle=style_vmdc)
#     plt.plot(t_vmdc, u_vmdc[1, :], label='elevator [rad]', color=color_vmdc, linestyle=style_vmdc)
#     plt.xlabel('time [s]')
```

```
383 #     plt.ylabel('Control')
384 #     plt.legend(loc='center right')  # Set legend location to center righ
    t
385 #     plt.grid(True)
386 
387 #     # Adjust layout and show the figure
388 #     plt.tight_layout()
389 
390 #     if save:
391 #         plt.savefig(f'Output/aero_7dim_fixedmoment/{model}_nonlinear_tra
    ck.png', dpi=300)
392 #     else:
393 #         plt.show()
394 
395 #     trajectories = {
396 #         'initial': {'t': t_init, 'x': x_init, 'color': color_init, 'ref'
    :tp_init},
397 #         'vm': {'t': t_vm, 'x': x_vm, 'color': color_vm, 'ref':tp_vm},
398 #         'dm': {'t': t_dm, 'x': x_dm, 'color': color_dm, 'ref':tp_dm},
399 #         'vmdc': {'t': t_vmdc, 'x': x_vmdc, 'color': color_vmdc, 'ref':tp
    _vmdc}
400 #     }
401 #     # Error computation for all trajectories
402 #     print("\n" + "="*70)
403 #     print("TRACKING ERROR ANALYSIS")
404 #     print("="*70)
405 
406 #     error_data = {}
407 #     for traj_name, traj_data in trajectories.items():
408 #         t = traj_data['t']
409 #         x = traj_data['x']
410 #         tp = traj_data['ref']
411 
412 #         # # Interpolate reference trajectory to match current trajectory
    's time points
413 #         # x_ref_interp = np.zeros_like(x)
414 #         # for i in range(4):
415 #         #     x_ref_interp[i, :] = np.interp(t, t_ref, x_ref[i, :])
416 
417 #         # Compute errors
418 #         error = x[0:4, :] - tp
419 #         vel_err =   np.sqrt(trapezoid(error[0, :]**2, t))
420 #         aoa_err =   np.sqrt(trapezoid(error[1, :]**2, t))
421 #         pr_err =    np.sqrt(trapezoid(error[2, :]**2, t))
422 #         pitch_err = np.sqrt(trapezoid(error[3, :]**2, t))
423 #         error_norm = np.sqrt(trapezoid(np.sum(error**2, 0), t))
424 
425 #         error_data[traj_name] = {
426 #             'vel_err': vel_err,
427 #             'aoa_err': aoa_err,
428 #             'pr_err': pr_err,
429 #             'pitch_err': pitch_err,
430 #             'error_norm': error_norm
431 #         }
432 
433 #         print(f"\n{traj_name.upper()} Model:")
434 #         print(f"  Total error (L2 norm): {error_norm:.6f}")
435 #         print(f"  Velocity error:        {vel_err:.6f}")
```

```
436 #         print(f"  AOA error:             {aoa_err:.6f}")
437 #         print(f"  Pitch rate error:      {pr_err:.6f}")
438 #         print(f"  Pitch angle error:     {pitch_err:.6f}")
439 
440 #     print("="*70 + "\n")
441 
442 #     # Start new figure for error plot
443 #     plt.figure()
444 
445 #     models_list = list(trajectories.keys())
446 #     error_types = ['vel_err', 'aoa_err', 'pr_err', 'pitch_err', 'error_n
    orm']
447 #     error_labels = ['Velocity', 'AOA', 'Pitch Rate', 'Pitch Angle', 'Tot
    al']
448 
449 #     # Define bar patterns/hatches
450 #     hatches = ['///', '\\\\\\', '|||', '---', 'xxx']
451 
452 #     # Set up bar positions
453 #     x_pos = np.arange(len(models_list))
454 #     bar_width = 0.15
455 
456 #     # Plot bars for each error type
457 #     for i, (error_type, error_label, hatch) in enumerate(zip(error_types
    , error_labels, hatches)):
458 #         offset = (i - 2) * bar_width
459 #         values = [error_data[model][error_type] for model in models_list
    ]
460 #         plt.bar(x_pos + offset, values, bar_width, label=error_label, ha
    tch=hatch, edgecolor='black', linewidth=1.5)
461 
462 #     # Customize the plot
463 #     plt.xlabel('Model Type', fontsize=12)
464 #     plt.ylabel('Error Magnitude', fontsize=12)
465 #     plt.title('Tracking Error Comparison Across Models', fontsize=14, fo
    ntweight='bold')
466 #     plt.xticks(x_pos, [m.upper() for m in models_list])
467 #     plt.legend(loc='upper right', fontsize=10)
468 #     plt.grid(True, axis='y', alpha=0.3)
469 #     plt.show()
470 
471 def nonlinear_track_velocity(design,fp1,fp2,data_file,tf,switch_time=1e6):
472     '''This code will simulate the nonlinear tracking problem. It construc
    ts a linear controller that uses LQR + integrator
473     to track the reference.
474     THE INTEGRATOR TRACKS VELOCITY
475     Inputs:
476         design is an array [chs, whs] that gives the design parameters we
    are testing
477         fp1,fp2 are arrays [v0,climb_ang] that give the first and second d
    esired velocities and climb angles
478         data_file is the aero data file we are using for all aerodynamic d
    ata
479         tf is final time of simulation
480         switch_time is the time where we command aircraft to switch from t
    racking fp1 to fp2. By default, this is very large to prevent swit
    ching.
481 
```

```
482     Outputs:
483         time, trajectory, control history, reference trajectory'''
484 
485     chs = design[0]
486     whs = design[1]
487 
488     v01 = fp1[0]
489     climb_ang1 = fp1[1]
490 
491     # Using given flight point, compute trimmed angle of attack and inputs
        .
492     alpha01, deltae01, deltaT01, pitch01 = compute_trim_eom(np.array([v01,c
        hs,whs]), data_file,climb_angle=climb_ang1)
493 
494     full_flight_point = np.array([v01,alpha01,chs,whs,deltae01])
495     trim_point1 = np.array([v01,alpha01,0,pitch01])
496     trim_inputs1 = np.array([deltaT01,deltae01])
497     tp1 = trim_point1.copy()
498     tp1[1] = np.rad2deg(tp1[1])
499     tp1[2] = np.rad2deg(tp1[2])
500     tp1[3] = np.rad2deg(tp1[3])
501     # print(f'trim point 1:{tp1}')
502     interp_dict = aero_interps(data_file)
503     A1,B1 = build_AB(interp_dict,full_flight_point)
504     C = np.array([[1,0,0,0]])
505     C1 = C
506 
507     k = C1.shape[0]
508     n = A1.shape[0]
509     m = B1.shape[1]
510 
511     Abar1 = np.concatenate((np.concatenate((A1,np.zeros((n,k))),axis=1),np.c
        oncatenate((C1,np.zeros((k,k))),axis=1)),axis=0)
512     Bbar1 = np.concatenate((B1,np.zeros((k,m))),axis=0)
513 
514     Q = np.diag([1, 1, 1, 100, 1])
515     R = np.diag([1,100]) # penalize elevator usage more to avoid too much
        variation in AOA.
516 
517     # This is the control gain we want to use, we need to create an integr
        ator state in the nonlinear model to use this.
518     # Then, with each application, we simply subtract from the nonlinear s
        tate the trim state and multiply with K...
519     K1 = ct.lqr(Abar1,Bbar1,Q,R)[0]
520 
521     v02 = fp2[0]
522     climb_ang2 = fp2[1]
523 
524     alpha02, deltae02, deltaT02, pitch02 = compute_trim_eom(np.array([v02,c
        hs,whs]), data_file,climb_angle=climb_ang2)
525 
526     full_flight_point = np.array([v02,alpha02,chs,whs,deltae02])
527     trim_point2 = np.array([v02,alpha02,0,pitch02])
528     tp2 = trim_point2.copy()
529     tp2[1] = np.rad2deg(tp2[1])
530     tp2[2] = np.rad2deg(tp2[2])
531     tp2[3] = np.rad2deg(tp2[3])
532     # print(f'trim point 2: {tp2}')
```

```
533     trim_inputs2 = np.array([deltaT02,deltae02])
534     A2,B2 = build_AB(interp_dict,full_flight_point)
535     C2 = C
536 
537     k = C2.shape[0]
538     n = A2.shape[0]
539     m = B2.shape[1]
540 
541     Abar2 = np.concatenate((np.concatenate((A2,np.zeros((n,k))),axis=1),np.c
        oncatenate((C2,np.zeros((k,k))),axis=1)),axis=0)
542     Bbar2 = np.concatenate((B2,np.zeros((k,m))),axis=0)
543 
544     K2 = ct.lqr(Abar2,Bbar2,Q,R)[0]
545 
546     # Now we have 2 controllers, K1 and K2...
547 
548     # some constants
549     g = 9.81  # m/s^2
550     weight = 331522 # lb
551     weight = weight * 4.44822 # convert to N
552     mass = weight / g  # kg
553     Iy = 3759014 # slug ft^2
554     Iy = Iy * 14.5939 * 0.3048**2 # kg m^2
555     thrust_force = 96000 # lb
556     maxT = thrust_force * 4.44822 * 1.5 # convert to N
557 
558     M_interp = interp_dict['M']
559     Di_interp = interp_dict['Di']
560     L_interp = interp_dict['L']
561 
562     def linear_controller1(t,x):
563         # take current state, subtract from it the trimmed state and multi
        ply with gain K.
564         # send back the resulting controller after applying proper bounds.
565 
566         x_bar = x - np.concatenate((trim_point1.reshape(4,-1),np.array([[0]])),a
        xis=0).flatten()
567         u = -K1 @ x_bar
568         u = u + trim_inputs1
569         u[0] = np.clip(u[0],0,1)
570         u[1] = np.clip(u[1],np.deg2rad(-30),np.deg2rad(30))
571 
572         return u
573 
574     def linear_controller2(t,x):
575         x_bar = x - np.concatenate((trim_point2.reshape(4,-1),np.array([[0]])),a
        xis=0).flatten()
576         u = -K2 @ x_bar
577         u = u + trim_inputs2
578         u[0] = np.clip(u[0],0,1)
579         u[1] = np.clip(u[1],np.deg2rad(-30),np.deg2rad(30))
580         return u
581 
582     def nonlinear_model(t, x):
583         # with an integrator
584 
585         v = x[0]
```

```
586         alpha = x[1]
587         q = x[2]
588         theta = x[3]
589 
590         # solve for control using integral controller
591 
592         # switch based on fixed time
593         if t < switch_time:
594             u = linear_controller1(t,x)
595         else:
596             u = linear_controller2(t,x)
597         xr = v01 if t < switch_time else v02
598 
599         deltaT = u[0]
600         deltae = u[1]
601 
602         flight_point = np.asarray([v, alpha, chs, whs, deltae])
603         # print(flight_point)
604 
605         M = M_interp(flight_point)[0]
606         Di = Di_interp(flight_point)[0]
607         L = L_interp(flight_point)[0]
608 
609         Ft = deltaT * maxT
610 
611         x_dot = np.zeros_like(x)
612         x_dot[0] = Ft/mass * np.cos(alpha) - Di/mass - g * np.sin(theta - a
    lpha)
613         x_dot[1] = -Ft/(mass * v) * np.sin(alpha) - L/(mass * v) + g/v * n
    p.cos(theta - alpha) + q
614         x_dot[2] = M / Iy
615         x_dot[3] = q
616         x_dot[4] = v - xr           # integrator state
617 
618         threshold = 5
619         if x[4] > threshold:      # prevent wind up, if integrator state t
    oo large, keep constant...
620             x_dot[4] = min(x_dot[4],0)
621         elif x[4] < -threshold:
622             x_dot[4] = max(x_dot[4],0)
623 
624         # print(f'derivative: {x_dot}')
625 
626         return x_dot
627 
628     # Build initial condition using trim point around slower state
629     fp0 = np.asarray([v01 - 10, chs, whs])
630     alpha0, deltae0, deltaT0, pitch0 = compute_trim_eom(fp0, data_file,cli
    mb_angle=0)
631     trim_point0 = np.array([fp0[0],alpha0,0,pitch0])
632 
633     x0 = np.concatenate((trim_point0.reshape(4,-1),np.array([[0]])),axis=0)
634     x0 = x0.flatten()
635 
636     sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x)  , (0.0, tf), x0, m
    ethod='BDF')
637     if not sol_fwd.success:
638         raise RuntimeError("Forward integration failed: " + str(sol_fwd.me
```

```
    ssage))

    t = sol_fwd.t
    x = sol_fwd.y

    u_hist = np.zeros((m,t.size))
    tp_hist = np.zeros((n,t.size))

    for i, ti in enumerate(t):
        if ti <= switch_time:
            u = linear_controller1(ti,x[:,i])
            tp_hist[:,i] = trim_point1
        else:
            u = linear_controller2(ti,x[:,i])
            tp_hist[:,i] = trim_point2
        u_hist[:,i] = u.flatten()

    return t,x, u_hist, tp_hist

def nonlinear_track_pitch(design,fp1,fp2,data_file,tf,switch_time=100):
    '''This code will simulate the nonlinear tracking problem. It construc
    ts a linear controller that uses LQR + integrator
    to track the reference.
    THE INTEGRATOR TRACKS PITCH ANGLE
    Inputs:
        design is an array [chs, whs] that gives the design parameters we
    are testing
        fp1,fp2 are arrays [v0,climb_ang] that give the first and second d
    esired velocities and climb angles
        data_file is the aero data file we are using for all aerodynamic d
    ata
        tf is final time of simulation
        switch_time is the time where we command aircraft to switch from t
    racking fp1 to fp2. By default, this is very large to prevent swit
    ching.

    Outputs:
        time, trajectory, control history, trim point trajectory'''

    chs = design[0]
    whs = design[1]

    v01 = fp1[0]
    climb_ang1 = fp1[1]

    # Using given flight point, compute trimmed angle of attack and inputs
    .
    if design.size < 3:
        data_file = 'Aero_data/aero_7dim2.npz'
        alpha01, deltae01, deltaT01, pitch01, _cost = compute_trim_eom(np.a
    sarray([v01,chs,whs]), data_file,climb_angle=climb_ang1)
        full_flight_point = np.array([v01,alpha01,chs,whs,deltae01])
    else:
        data_file = 'Aero_data/aero_merged_cg_17.npz'     # MAKE SURE TO P
    ASS THE CORRECT NUMBER OF FLIGHT_POINTS TO compute_trim_eom WHEN U
    SING THIS
        ws = design[2]
        ths= design[3]
```

```
686         wd = design[4]
687 
688         alpha01, deltae01, deltaT01, pitch01, _cost = compute_trim_eom(np.a
    sarray([v01,chs,whs,ws,ths,wd]), data_file,climb_angle=climb_ang1)
689         full_flight_point = np.array([v01,alpha01,chs,whs,deltae01,ws,ths,w
    d])
690 
691 
692     trim_point1 = np.array([v01,alpha01,0,pitch01])
693     trim_inputs1 = np.array([deltaT01,deltae01])
694     tp1 = trim_point1.copy()
695     tp1[1] = np.rad2deg(tp1[1])
696     tp1[2] = np.rad2deg(tp1[2])
697     tp1[3] = np.rad2deg(tp1[3])
698     print(f'trim point 1:{tp1}')
699     # print(f'trim inputs 1: {trim_inputs1}')
700     interp_dict = aero_interps(data_file)
701     A1,B1 = build_AB(interp_dict,full_flight_point)
702     C = np.array([[0,0,0,1]])
703     C1 = C
704 
705     k = C1.shape[0]
706     n = A1.shape[0]
707     m = B1.shape[1]
708 
709     Abar1 = np.concatenate((np.concatenate((A1,np.zeros((n,k))),axis=1),np.c
    oncatenate((C1,np.zeros((k,k))),axis=1)),axis=0)
710     Bbar1 = np.concatenate((B1,np.zeros((k,m))),axis=0)
711 
712     # I think this is main
713     Q = np.diag([1,100,1,100,50])
714     R = np.diag([0.1,1000]) # penalize elevator usage more to avoid too mu
    ch variation in AOA.
715 
716     # This is the control gain we want to use, we need to create an integr
    ator state in the nonlinear model to use this.
717     # Then, with each application, we simply subtract from the nonlinear s
    tate the trim state and multiply with K...
718     K1 = ct.lqr(Abar1,Bbar1,Q,R)[0]
719 
720     v02 = fp2[0]
721     climb_ang2 = fp2[1]
722 
723     if design.size < 3:
724         data_file = 'Aero_data/aero_7dim2.npz'
725         alpha02, deltae02, deltaT02, pitch02, _cost = compute_trim_eom(np.a
    sarray([v02,chs,whs]), data_file,climb_angle=climb_ang2)
726         full_flight_point = np.array([v02,alpha02,chs,whs,deltae02])
727     else:
728         data_file = 'Aero_data/aero_merged_cg_17.npz'     # MAKE SURE TO P
    ASS THE CORRECT NUMBER OF FLIGHT_POINTS TO compute_trim_eom WHEN U
    SING THIS
729         ws = design[2]
730         ths= design[3]
731         wd = design[4]
732 
733         alpha02, deltae02, deltaT02, pitch02, _cost = compute_trim_eom(np.a
    sarray([v02,chs,whs,ws,ths,wd]), data_file,climb_angle=climb_ang2, v
```

```
    erbose=False)
734         full_flight_point = np.array([v02,alpha02,chs,whs,deltae02,ws,ths,w
    d])
735
736     trim_point2 = np.array([v02,alpha02,0,pitch02])
737     tp2 = trim_point2.copy()
738     tp2[1] = np.rad2deg(tp2[1])
739     tp2[2] = np.rad2deg(tp2[2])
740     tp2[3] = np.rad2deg(tp2[3])
741     print(f'trim point 2: {tp2}')
742     # print(f'Inputs: {deltaT02} and {deltae02}')
743     trim_inputs2 = np.array([deltaT02,deltae02])
744     A2,B2 = build_AB(interp_dict,full_flight_point)
745
746     # eigvals = np.linalg.eigvals(A2)
747     # print("Eigenvalues:", eigvals)
748     # print("Real parts:", eigvals.real)
749     # stiffness_ratio = np.max(np.abs(eigvals.real)) / np.min(np.abs(eigva
    ls.real[eigvals.real != 0]))
750     # print("Stiffness ratio:", stiffness_ratio)
751     C2 = C
752
753     k = C2.shape[0]
754     n = A2.shape[0]
755     m = B2.shape[1]
756
757     Abar2 = np.concatenate((np.concatenate((A2,np.zeros((n,k))),axis=1),np.c
    oncatenate((C2,np.zeros((k,k))),axis=1)),axis=0)
758     Bbar2 = np.concatenate((B2,np.zeros((k,m))),axis=0)
759
760     K2 = ct.lqr(Abar2,Bbar2,Q,R)[0]
761
762     # Now we have 2 controllers, K1 and K2...
763
764     # some constants
765     g = 9.81  # m/s^2
766     weight = 331522 # lb
767     weight = weight * 4.44822 # convert to N
768     mass = weight / g  # kg
769     Iy = 3759014 # slug ft^2
770     Iy = Iy * 14.5939 * 0.3048**2 # kg m^2
771     thrust_force = 96000 # lb
772     maxT = thrust_force * 4.44822 * 1.5 # convert to N
773
774     M_interp = interp_dict['M']
775     Di_interp = interp_dict['Di']
776     L_interp = interp_dict['L']
777
778     def linear_controller1(t,x):
779         # take current state, subtract from it the trimmed state and multi
    ply with gain K.
780         # send back the resulting controller after applying proper bounds.
781
782         x_bar = x - np.concatenate((trim_point1.reshape(4,-1),np.array([[0]])),a
    xis=0).flatten()
783         u = -K1 @ x_bar
784         u = u + trim_inputs1
```

```
785         u[0] = np.clip(u[0],0,1)
786         u[1] = np.clip(u[1],np.deg2rad(-30),np.deg2rad(30))
787 
788         return u
789 
790     def linear_controller2(t,x):
791         x_bar = x - np.concatenate((trim_point2.reshape(4,-1),np.array([[0]])),a
    xis=0).flatten()
792         u = -K2 @ x_bar
793         u = u + trim_inputs2
794         u[0] = np.clip(u[0],0,1)
795         u[1] = np.clip(u[1],np.deg2rad(-30),np.deg2rad(30))
796         return u
797 
798     def nonlinear_model(t, x, mode):
799         # with an integrator
800 
801         v = x[0]
802         alpha = x[1]
803         q = x[2]
804         theta = x[3]
805 
806         # solve for control using integral controller
807 
808         # switch based on fixed time
809         if mode == 1:
810             u = linear_controller1(t,x)
811             xr = pitch01
812         elif mode == 2:
813             u = linear_controller2(t,x)
814             xr = pitch02
815 
816         deltaT = u[0]
817         deltae = u[1]
818 
819         if design.size < 3:
820             flight_point = np.asarray([v, alpha, chs, whs, deltae])
821         else:
822             flight_point = np.asarray([v,alpha,chs,whs,deltae,ws,ths,wd])
823 
824         M = M_interp(flight_point)[0]
825         Di = Di_interp(flight_point)[0]
826         L = L_interp(flight_point)[0]
827 
828         Ft = deltaT * maxT
829 
830         x_dot = np.zeros_like(x)
831         x_dot[0] = Ft/mass * np.cos(alpha) - Di/mass - g * np.sin(theta - a
    lpha)
832         x_dot[1] = -Ft/(mass * v) * np.sin(alpha) - L/(mass * v) + g/v * n
    p.cos(theta - alpha) + q
833         x_dot[2] = M / Iy
834         x_dot[3] = q
835         x_dot[4] = theta - xr           # integrator state
836 
837         threshold = 0.3
838         if x[4] > threshold:      # prevent wind up, if integrator state t
    oo large, keep constant...
```

```
839             x_dot[4] = min(x_dot[4],0)
840         elif x[4] < -threshold:
841             x_dot[4] = max(x_dot[4],0)
842 
843         # print(f'derivative: {x_dot}')
844 
845         return x_dot
846 
847     # Build initial condition using trim point around slower state
848     fp0 = np.asarray([v01 - 5, chs, whs])
849     if design.size < 3:
850         data_file = 'Aero_data/aero_7dim2.npz'
851         alpha0, deltae0, deltaT0, pitch0, _cost = compute_trim_eom(np.asar
    ray([v01-5,chs,whs]), data_file,climb_angle=climb_ang2)
852         full_flight_point = np.array([v02,alpha02,chs,whs,deltae02])
853     else:
854         data_file = 'Aero_data/aero_merged_cg_17.npz'
855         ws = design[2]
856         ths= design[3]
857         wd = design[4]
858 
859         alpha0, deltae0, deltaT0, pitch0, _cost = compute_trim_eom(np.asar
    ray([v01-5,chs,whs,ws,ths,wd]), data_file,climb_angle=climb_ang1 - 5)
860         full_flight_point = np.array([v02,alpha02,chs,whs,deltae02,ws,ths,w
    d])
861     trim_point0 = np.array([fp0[0],alpha0,0,pitch0])
862 
863     tp0 = trim_point0.copy()
864     tp0[1] = np.rad2deg(tp0[1])
865     tp0[2] = np.rad2deg(tp0[2])
866     tp0[3] = np.rad2deg(tp0[3])
867     print(f'trim point 0: {tp0}')
868 
869     x0 = np.concatenate((trim_point0.reshape(4,-1),np.array([[0]])),axis=0)
870     x0 = x0.flatten()
871 
872     #--------
873 
874     t_eval1 = np.linspace(0,switch_time,600)
875     sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x, 1)  , (0.0, switc
    h_time), x0, method='BDF', t_eval = t_eval1)
876     # sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x, 1)  , (0.0, swi
    tch_time), x0, method='BDF')
877     t1 = sol_fwd.t
878     x1 = sol_fwd.y
879     # Take new initial condition at end of previous and set integrator sta
    te to 0
880     new_init = x1[:,-1]
881     new_init[4] = 0
882     t_eval2 = np.linspace(switch_time, tf, 600)
883     sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x, 2)  , (switch_tim
    e, tf), new_init, method='BDF', t_eval = t_eval2)
884     # sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x, 2)  , (switch_t
    ime, tf), new_init, method='BDF')
885     t2 = sol_fwd.t
886     x2 = sol_fwd.y
887 
888     t = np.concatenate((t1,t2),axis=0)
```

```
    x = np.concatenate((x1,x2),axis=1)


    #---------

    # sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x)  , (0.0, tf), x
    0, method='BDF')
    # if not sol_fwd.success:
    #     raise RuntimeError("Forward integration failed: " + str(sol_fwd.
    message))

    # t = sol_fwd.t
    # x = sol_fwd.y

    u_hist = np.zeros((m,t.size))
    tp_hist = np.zeros((n,t.size))

    for i, ti in enumerate(t):
        if ti <= switch_time:
            u = linear_controller1(ti,x[:,i])
            tp_hist[:,i] = trim_point1
        else:
            u = linear_controller2(ti,x[:,i])
            tp_hist[:,i] = trim_point2
        u_hist[:,i] = u.flatten()

    return t,x, u_hist, tp_hist


def nonlinear_track_pitch_air_force(design,fp1,fp2,data_file,tf,switch_tim
    e=100):
    '''This code will simulate the nonlinear tracking problem. It construc
    ts a linear controller that uses LQR + integrator
    to track the reference.
    THE INTEGRATOR TRACKS PITCH ANGLE
    Inputs:
        design is an array [chs, whs] that gives the design parameters we
    are testing
        fp1,fp2 are arrays [v0,climb_ang] that give the first and second d
    esired velocities and climb angles
        data_file is the aero data file we are using for all aerodynamic d
    ata
        tf is final time of simulation
        switch_time is the time where we command aircraft to switch from t
    racking fp1 to fp2. By default, this is very large to prevent swit
    ching.

    Outputs:
        time, trajectory, control history, trim point trajectory'''

    chs = design[0]
    whs = design[1]

    v01 = fp1[0]
    climb_ang1 = fp1[1]

    # Using given flight point, compute trimmed angle of attack and inputs
    .
```

```
937     alpha01, deltae01, deltaT01, pitch01 = compute_trim_eom(np.array([v01,c
        hs,whs]), data_file,climb_angle=climb_ang1)
938
939     full_flight_point = np.array([v01,alpha01,chs,whs,deltae01])
940     trim_point1 = np.array([v01,alpha01,0,pitch01])
941     trim_inputs1 = np.array([deltaT01,deltae01])
942     tp1 = trim_point1.copy()
943     tp1[1] = np.rad2deg(tp1[1])
944     tp1[2] = np.rad2deg(tp1[2])
945     tp1[3] = np.rad2deg(tp1[3])
946     print(f'trim point 1:{tp1}')
947     interp_dict = aero_interps(data_file)
948     A1,B1 = build_AB(interp_dict,full_flight_point)
949     C = np.array([[0,0,0,1]])
950     C1 = C
951
952     k = C1.shape[0]
953     n = A1.shape[0]
954     m = B1.shape[1]
955
956     Abar1 = np.concatenate((np.concatenate((A1,np.zeros((n,k))),axis=1),np.c
        oncatenate((C1,np.zeros((k,k))),axis=1)),axis=0)
957     Bbar1 = np.concatenate((B1,np.zeros((k,m))),axis=0)
958
959     # I think this is main
960     # Q = np.diag([1,100,1,100,100])
961     # R = np.diag([0.1,1000]) # penalize elevator usage more to avoid too
        much variation in AOA.
962
963     # Trying from frequency analysis
964     Q = np.diag([100,1,1,1000,10])
965     R = np.diag([0.1,10])
966
967     # Another one
968     # Q = np.diag([100, 1, 1, 1000, 10])
969     # R = np.diag([0.1, 10])
970
971     # This is the control gain we want to use, we need to create an integr
        ator state in the nonlinear model to use this.
972     # Then, with each application, we simply subtract from the nonlinear s
        tate the trim state and multiply with K...
973     K1 = ct.lqr(Abar1,Bbar1,Q,R)[0]
974
975     v02 = fp2[0]
976     climb_ang2 = fp2[1]
977
978     alpha02, deltae02, deltaT02, pitch02 = compute_trim_eom(np.array([v02,c
        hs,whs]), data_file,climb_angle=climb_ang2)
979     # print(f'Trimmed alpha: {np.rad2deg(alpha02)}, deltae: {np.rad2deg(de
        ltae02)}, deltaT: {deltaT02}, pitch: {np.rad2deg(pitch02)}')
980
981     full_flight_point = np.array([v02,alpha02,chs,whs,deltae02])
982     trim_point2 = np.array([v02,alpha02,0,pitch02])
983     tp2 = trim_point2.copy()
984     tp2[1] = np.rad2deg(tp2[1])
985     tp2[2] = np.rad2deg(tp2[2])
986     tp2[3] = np.rad2deg(tp2[3])
987     print(f'trim point 2: {tp2}')
```

```
    trim_inputs2 = np.array([deltaT02,deltae02])
    A2,B2 = build_AB(interp_dict,full_flight_point)

    # eigvals = np.linalg.eigvals(A2)
    # print("Eigenvalues:", eigvals)
    # print("Real parts:", eigvals.real)
    # stiffness_ratio = np.max(np.abs(eigvals.real)) / np.min(np.abs(eigva
    ls.real[eigvals.real != 0]))
    # print("Stiffness ratio:", stiffness_ratio)
    C2 = C

    k = C2.shape[0]
    n = A2.shape[0]
    m = B2.shape[1]

    Abar2 = np.concatenate((np.concatenate((A2,np.zeros((n,k))),axis=1),np.c
    oncatenate((C2,np.zeros((k,k))),axis=1)),axis=0)
    Bbar2 = np.concatenate((B2,np.zeros((k,m))),axis=0)

    K2 = ct.lqr(Abar2,Bbar2,Q,R)[0]

    # Now we have 2 controllers, K1 and K2...

    # some constants
    g = 9.81  # m/s^2
    weight = 331522 # lb
    weight = weight * 4.44822 # convert to N
    mass = weight / g  # kg
    Iy = 3759014 # slug ft^2
    Iy = Iy * 14.5939 * 0.3048**2 # kg m^2
    thrust_force = 96000 # lb
    maxT = thrust_force * 4.44822 * 1.5 # convert to N

    M_interp = interp_dict['M']
    Di_interp = interp_dict['Di']
    L_interp = interp_dict['L']

    def linear_controller1(t,x):
        # take current state, subtract from it the trimmed state and multi
    ply with gain K.
        # send back the resulting controller after applying proper bounds.


        x_bar = x - np.concatenate((trim_point1.reshape(4,-1),np.array([[0]])),a
    xis=0).flatten()
        u = -K1 @ x_bar
        u = u + trim_inputs1
        u[0] = np.clip(u[0],0,1)
        u[1] = np.clip(u[1],np.deg2rad(-30),np.deg2rad(30))

        return u

    def linear_controller2(t,x):
        x_bar = x - np.concatenate((trim_point2.reshape(4,-1),np.array([[0]])),a
    xis=0).flatten()
        u = -K2 @ x_bar
        u = u + trim_inputs2
        u[0] = np.clip(u[0],0,1)
```

```
        u[1] = np.clip(u[1],np.deg2rad(-30),np.deg2rad(30))
        return u

    def nonlinear_model(t, x, mode):
        # with an integrator

        v = x[0]
        alpha = x[1]
        q = x[2]
        theta = x[3]

        # solve for control using integral controller

        # switch based on fixed time
        if mode == 1:
            u = linear_controller1(t,x)
            xr = pitch01
        elif mode == 2:
            u = linear_controller2(t,x)
            xr = pitch02

        deltaT = u[0]
        deltae = u[1]

        flight_point = np.asarray([v, alpha, chs, whs, deltae])
        # print(x)
        # print(flight_point)

        M = M_interp(flight_point)[0]
        Di = Di_interp(flight_point)[0]
        L = L_interp(flight_point)[0]

        Ft = deltaT * maxT

        x_dot = np.zeros_like(x)
        x_dot[0] = Ft/mass * np.cos(alpha) - Di/mass - g * np.sin(theta - a
    lpha)
        x_dot[1] = -Ft/(mass * v) * np.sin(alpha) - L/(mass * v) + g/v * n
    p.cos(theta - alpha) + q
        x_dot[2] = M / Iy
        x_dot[3] = q
        x_dot[4] = theta - xr           # integrator state

        threshold = 0.3
        if x[4] > threshold:      # prevent wind up, if integrator state t
    oo large, keep constant...
            x_dot[4] = min(x_dot[4],0)
        elif x[4] < -threshold:
            x_dot[4] = max(x_dot[4],0)

        # print(f'derivative: {x_dot}')

        return x_dot

    # Build initial condition using trim point around slower state
    fp0 = np.asarray([v01, chs, whs])
    alpha0, deltae0, deltaT0, pitch0 = compute_trim_eom(fp0, data_file,cli
    mb_angle=climb_ang1)
```

```
1094      # trim_point0 = np.array([fp0[0] - 5,alpha0,0,pitch0 - np.deg2rad(5)])
1095      trim_point0 = np.array([fp0[0],alpha0,0,pitch0])
1096
1097      tp0 = trim_point0.copy()
1098      tp0[1] = np.rad2deg(tp0[1])
1099      tp0[2] = np.rad2deg(tp0[2])
1100      tp0[3] = np.rad2deg(tp0[3])
1101      print(f'trim point 0: {tp0}')
1102
1103      x0 = np.concatenate((trim_point0.reshape(4,-1),np.array([[0]])),axis=0)
1104      x0 = x0.flatten()
1105
1106      #--------
1107      sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x, 1)  , (0.0, switc
          h_time), x0, method='BDF')
1108      t1 = sol_fwd.t
1109      x1 = sol_fwd.y
1110      # Take new initial condition at end of previous and set integrator sta
          te to 0
1111      new_init = x1[:,-1]
1112      new_init[4] = 0
1113      sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x, 2)  , (switch_tim
          e, tf), new_init, method='BDF')
1114      t2 = sol_fwd.t
1115      x2 = sol_fwd.y
1116
1117      t = np.concatenate((t1,t2),axis=0)
1118      x = np.concatenate((x1,x2),axis=1)
1119
1120
1121      #---------
1122
1123      # sol_fwd = solve_ivp(lambda t,x: nonlinear_model(t,x)  , (0.0, tf), x
          0, method='BDF')
1124      # if not sol_fwd.success:
1125      #     raise RuntimeError("Forward integration failed: " + str(sol_fwd.
          message))
1126
1127      # t = sol_fwd.t
1128      # x = sol_fwd.y
1129
1130      u_hist = np.zeros((m,t.size))
1131      tp_hist = np.zeros((n,t.size))
1132
1133      for i, ti in enumerate(t):
1134          if ti <= switch_time:
1135              u = linear_controller1(ti,x[:,i])
1136              tp_hist[:,i] = trim_point1
1137          else:
1138              u = linear_controller2(ti,x[:,i])
1139              tp_hist[:,i] = trim_point2
1140          u_hist[:,i] = u.flatten()
1141
1142      return t,x, u_hist, tp_hist
1143
1144
1145
```

```
1146 if __name__ == "__main__":
1147     # Test the nonlinear model by integrating it forward in time.
1148     v0 = 235 # speed in m/s
1149 
1150     # Initial
1151     chs = 5 # center halfspan in m
1152     whs = 12 # wing halfspan in m
1153     model_type = 'initial'
1154 
1155     opt_path3 = 'Output/aero_7dim_asym/N6_volmax_110con'
1156     opt_path1 = 'Output/aero_7dim_asym/N6_volmax'
1157     opt_path2 = 'Output/aero_7dim_asym/N6_110max'
1158     models = [opt_path1, opt_path2, opt_path3]
1159 
1160     speed1 = 190
1161     climb_angle1 = 10
1162     speed2 = 210
1163     climb_angle2 = 0
1164 
1165     data_file = 'Aero_data/aero_7dim2.npz'
1166     flight_point = np.array([v0,chs,whs])
1167     tf = 40 # seconds
1168     st = 20 # switching time
1169 
1170     savefig = False
1171 
1172 
1173     #%% Velocity integrator tracker for initial and 3 models
1174 
1175     model_type = 'initial'
1176     t,x,u_hist,tp_hist = nonlinear_track_velocity(np.array([chs,whs]),np.a
         rray([speed1,climb_angle1]),
1177                                                   np.array([speed2,climb_angle2]),d
         ata_file, tf,switch_time=st)
1178     plotit(t,x,u_hist,ref=tp_hist,bound=False,model=model_type, save=savef
         ig)
1179 
1180     for opt_path in models:
1181 
1182         model_type = opt_path.split('/')[-1][3:]
1183         print(model_type)
1184         output_hdf5_file = f"{opt_path}/opt_vars.hdf5"
1185         x = extract_dataset_from_results(output_hdf5_file, "x")
1186         chs = x[0]
1187         whs = x[1]
1188 
1189         t,x,u_hist,tp_hist = nonlinear_track_velocity(np.array([chs,whs]),n
     p.array([speed1,climb_angle1]),
1190                                                   np.array([speed2,climb_angle2]),d
         ata_file, tf,switch_time=st)
1191         plotit(t,x,u_hist,ref=tp_hist,bound=False,model=model_type,save=sa
         vefig)
1192 
1193     #%%
1194 
1195 
1196     # t,x,u_hist = run_nonlinear(flight_point,data_file,tf)
1197 
```

```
    # t,x,u_hist, tp_hist = run_linear(flight_point,data_file,tf)
    # print(x[:,-1])

    # t,x,u_hist = linear_track(flight_point,data_file,tf)

    # t,x,u_hist = nonlinear_track(flight_point,data_file,tf)

    # print out final state for convenience.
    y = x.copy()
    y[1,:] = np.rad2deg(y[1,:])
    y[2,:] = np.rad2deg(y[2,:])
    y[3,:] = np.rad2deg(y[3,:])
    # print(f'final state: {y[:,-1]}')

    #%% Plot all
    # plotit(t,x,u_hist,ref=tp_hist,bound=False)

    # Show integrator state
    plt.plot(t,x[4,:])
    plt.show()

```

utils/plotters.py

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import trapezoid

def niceplot_one_at_a_time(sim_dictionary,model,save=False):
    ''' This function makes a nicer looking plot for the simulation trajectories
    between the initial and an optimized model that is fed in.
    No error bar plots since we only look at one model at a time.'''
    plt.figure(figsize=(12,12))
    plt.rcParams.update({'font.size': 15})
    plt.rcParams['lines.linewidth'] = 4
    plt.rcParams["legend.handlelength"] = 3.5
    plt.rcParams['lines.markersize'] = 12

    t_init = sim_dictionary['initial']['t']
    x_initial = sim_dictionary['initial']['x']
    u_init = sim_dictionary['initial']['u']
    tp_init = sim_dictionary['initial']['tp']
    style_init = sim_dictionary['initial']['style']
    color_init = sim_dictionary['initial']['color']

    # t_ref = sim_dictionary['ref']['t']
    # x_ref = sim_dictionary['ref']['x']
    # style_ref = sim_dictionary['ref']['style']
    # color_ref = sim_dictionary['ref']['color']

    t_opt = sim_dictionary[model]['t']
    x_opt = sim_dictionary[model]['x']
    u_opt = sim_dictionary[model]['u']
    tp_opt = sim_dictionary[model]['tp']
```

```
    style_opt = sim_dictionary[model]['style']
    color_opt = sim_dictionary[model]['color']

    name = sim_dictionary[model]['name']

    x_opt[0:4,:] = x_opt[0:4,:] - tp_opt
    x_init = x_initial.copy()[0:4,:] - tp_init

    # Plot 1: Airspeed
    plt.subplot(5, 1, 1)
    plt.plot(t_init, x_init[0, :], label='Initial', color=color_init, linestyle=style_init)
    plt.plot(t_opt, x_opt[0, :], label=name.upper(), color = color_opt, linestyle=style_opt)
    # plt.plot(t_ref, x_ref[0,:], label='Reference', linestyle=style_ref, color=color_ref)
    plt.legend(loc='best')  # Set legend location to center right
    plt.grid(True)
    plt.ylabel('Perturbed Airspeed\n [m/s] ($V - V_0$)')

    # Plot 2: Angle of Attack
    plt.subplot(5,1,2)
    plt.plot(t_init, np.rad2deg(x_init[1, :]), label='Initial', color=color_init, linestyle=style_init)
    plt.plot(t_opt,   np.rad2deg(x_opt[1, :]), label=name.upper(), color = color_opt, linestyle=style_opt)
    # plt.plot(t_ref,  np.rad2deg(x_ref[1,:]), label='Reference', linestyle=style_ref, color=color_ref)
    plt.grid(True)
    plt.ylabel('Perturbed Angle of\n Attack [deg]\n ($\\alpha - \\alpha_0$)')

    # Plot 3: Pitch Rate
    plt.subplot(5, 1, 3)
    plt.plot(t_init, np.rad2deg(x_init[2, :]), label='Initial', color=color_init, linestyle=style_init)
    plt.plot(t_opt,   np.rad2deg(x_opt[2, :]), label=name.upper(), color = color_opt, linestyle=style_opt)
    # plt.plot(t_ref,  np.rad2deg(x_ref[2,:]), label='Reference', linestyle=style_ref, color=color_ref)
    plt.grid(True)
    plt.ylabel('Perturbed Pitch Rate\n [deg/s] ($\dot{\\theta} - \dot{\\theta}_0$)')

    # Plot 4: Pitch angle
    plt.subplot(5, 1, 4)
    plt.plot(t_init, np.rad2deg(x_init[3, :]), label='Initial', color=color_init, linestyle=style_init)
    plt.plot(t_opt,   np.rad2deg(x_opt[3, :]), label=name.upper(), color = color_opt, linestyle=style_opt)
    # plt.plot(t_ref,  np.rad2deg(x_ref[3,:]), label='Reference', linestyle=style_ref, color=color_ref)
    plt.ylabel('Perturbed Pitch angle\n [deg] ($\\theta - \\theta_0$)')
    plt.grid(True)

    # Plot 5: Control input
    # only one input...
    plt.subplot(5, 1, 5)
```

```
75      plt.plot(t_init, u_init[0, :], label='$\delta_{th}$ Initial', color=co
        lor_init, linestyle='dashdot'         )#,marker='^',markevery=0.1)

76      plt.plot(t_init, u_init[1, :], label='$\delta_e$ Initial [rad]', color=c
        olor_init, linestyle='solid',marker='o',markevery=0.1  )#,marker='
        o',markevery=0.1)
77      plt.plot(t_opt, u_opt[0, :], label='$\delta_{th}$ Optimized', color=co
        lor_opt, linestyle='dashdot'          )#,marker='^',markevery=0.1)

78      plt.plot(t_opt, u_opt[1, :], label='$\delta_e$ Optimized [rad]', color=c
        olor_opt, linestyle='solid',marker='o',markevery=0.1   )#,marker='
        o',markevery=0.1)
79      plt.xlabel('Time [s]')
80      plt.ylabel('Control', wrap=True)
81      plt.legend(loc='center right')  # Set legend location to center right
82      # plt.legend(loc='outside lower center', ncol=2, frameon=False)
83      plt.grid(True)
84
85      # Adjust layout and show the figure
86      plt.tight_layout()
87
88      if save:
89          figname = f'{model}_nonlinear_track'
90          plt.savefig(f'Output/aero_7dim_asym/Nonlinear_Pitch_Integrator/{fi
        gname}.png', dpi=300)
91          path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive
        -Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures/Track
        ing'
92          plt.savefig(f'{path_to_paper}/{figname}')
93      else:
94          plt.show()
95
96  def niceplot_3_models_alt(sim_dictionary,models,save=False,savename='nonli
        near_track', compare=False):
97      ''' This function makes a nicer looking plot for the simulation trajec
        tories
98      between the initial and all optimized models. The 4 states are laid ou
        t in
99      a 2x2 grid, with the control subplot spanning a third row underneath.'
        ''
100     plt.rcParams.update({'font.size': 15})
101     plt.rcParams['lines.linewidth'] = 4
102     plt.rcParams["legend.handlelength"] = 3.5
103     plt.rcParams['lines.markersize'] = 12
104
105     mosaic = [
106         ['V', 'alpha'],
107         ['q', 'theta'],
108         ['u', 'u'],
109     ]
110     fig, axd = plt.subplot_mosaic(mosaic, figsize=(15, 8), layout='constra
        ined')
111
112     for i, m in enumerate(models):
113
114         t = sim_dictionary[m]['t']
115         x =     sim_dictionary[m]['x']
116         u =     sim_dictionary[m]['u']
```

```
117         tp =    sim_dictionary[m]['tp']
118         style = sim_dictionary[m]['style']
119         color = sim_dictionary[m]['color']
120 
121         track_error = x.copy()[0:4,:] - tp
122         name = sim_dictionary[m]['name']
123 
124         if name == 'inf_set':
125             name = 'Reach Proj'
126         elif name == 'gramian_eigs':
127             name = 'Gram Eigs'
128             style = 'dashdot'
129         else:
130             name = name.upper()
131 
132         if i == 0:
133             style = 'solid' if compare else 'dashed'
134         elif i == 1:
135             style = 'dotted'
136         elif i == 2:
137             style = 'dashdot'
138         elif i ==3 :
139             style = 'solid'
140 
141         # Airspeed
142         axd['V'].plot(t, track_error[0, :], label=name, color=color, lines
    tyle=style)
143         axd['V'].legend(loc='best')  # Set legend location to center right
144         axd['V'].grid(True)
145         axd['V'].set_ylabel('Perturbed Airspeed\n [m/s] ($V - V_0$)')
146 
147         # Angle of Attack
148         axd['alpha'].plot(t, np.rad2deg(track_error[1, :]), label=name, co
    lor=color, linestyle=style)
149         axd['alpha'].grid(True)
150         axd['alpha'].set_ylabel('Perturbed Angle of\n Attack [deg] ($\\alp
    ha - \\alpha_0$)')
151 
152         # Pitch Rate
153         axd['q'].plot(t,   np.rad2deg(track_error[2, :]), label=name, colo
    r = color, linestyle=style)
154         axd['q'].grid(True)
155         axd['q'].set_ylabel('Perturbed Pitch Rate\n [deg/s] ($\dot{\\theta
    } - \dot{\\theta}_0$)')
156         axd['q'].set_xlabel('Time (s)')
157 
158         # Pitch angle
159         axd['theta'].plot(t,   np.rad2deg(track_error[3, :]), label=name, c
    olor = color, linestyle=style)
160         axd['theta'].set_ylabel('Perturbed Pitch angle\n [deg] ($\\theta -
     \\theta_0$)')
161         axd['theta'].grid(True)
162         axd['theta'].set_xlabel('Time (s)')
163 
164         # Control input (spans both columns)
165         # only one input...
166         # axd['u'].plot(t, u[0, :], label='$\delta_{th}$'+f' {name}', colo
```

```
    r=color, linestyle='dashdot'         )#,marker='^',markevery=0.1)
        axd['u'].plot(t, np.rad2deg(u[1, :]), label=f'$\delta_e$ {name} [rad]', color=color, linestyle=style  )#,marker='o',markevery=0.1)
        axd['u'].set_xlabel('Time [s]')
        axd['u'].set_ylabel('Elevator Control\n Surface [deg] ($\delta_e$)', wrap=True)
        # axd['u'].legend(loc='center right')  # Set legend location to center right

        axd['u'].grid(True)

    if save:
        if not compare:
            plt.savefig(f'Output/5_dec_vars/Nonlinear_Pitch_Integrator/{savename}.png', dpi=300)
            path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive-Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures/Tracking'
            plt.savefig(f'{path_to_paper}/{savename}.png', dpi=300)
        else:
            plt.savefig(f'Output/5_dec_vars/Gramian_comparison/{savename}.png', dpi=300)
            path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive-Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures/Tracking'
            plt.savefig(f'{path_to_paper}/{savename}.png', dpi=300)

    else:
        plt.show()


def niceplot_compare_models(sim_dictionary,models,save=False):
    ''' This function makes a nicer looking plot for the simulation trajectories
    between the initial and an optimized model that is fed in.
    No error bar plots since we only look at one model at a time.'''
    fig = plt.figure(figsize=(12,12), layout='constrained')
    plt.rcParams.update({'font.size': 15})
    plt.rcParams['lines.linewidth'] = 4
    plt.rcParams["legend.handlelength"] = 3.5
    plt.rcParams['lines.markersize'] = 12

    for i, m in enumerate(models):


        t = sim_dictionary[m]['t']
        x =     sim_dictionary[m]['x']
        u =     sim_dictionary[m]['u']
        tp =    sim_dictionary[m]['tp']
        style = sim_dictionary[m]['style']
        color = sim_dictionary[m]['color']

        track_error = x.copy()[0:4,:] - tp
        name = sim_dictionary[m]['name']

        if name == 'inf_set':
            name = 'Reach Proj'
        elif name == 'gramian_eigs':
```

```
            name = 'Gram Eigs'
            style = 'dashdot'
        else:
            name = name.upper()

        # Plot 1: Airspeed
        plt.subplot(5, 1, 1)
        plt.plot(t, track_error[0, :], label=name, color=color, linestyle=s
    tyle)
        plt.legend(loc='best')  # Set legend location to center right
        plt.grid(True)
        plt.ylabel('Perturbed Airspeed\n [m/s] ($V - V_0$)')

        # Plot 2: Angle of Attack
        plt.subplot(5,1,2)
        plt.plot(t, np.rad2deg(track_error[1, :]), label=name, color=color, l
    inestyle=style)
        plt.grid(True)
        plt.ylabel('Perturbed Angle of\n Attack [deg]\n ($\\alpha - \\alph
    a_0$)')

        # Plot 3: Pitch Rate
        plt.subplot(5, 1, 3)
        plt.plot(t,   np.rad2deg(track_error[2, :]), label=name, color = c
    olor, linestyle=style)
        plt.grid(True)
        plt.ylabel('Perturbed Pitch Rate\n [deg/s] ($\dot{\\theta} - \dot{\
    \theta}_0$)')

        # Plot 4: Pitch angle
        plt.subplot(5, 1, 4)
        plt.plot(t,   np.rad2deg(track_error[3, :]), label=name, color = c
    olor, linestyle=style)
        plt.ylabel('Perturbed Pitch angle\n [deg] ($\\theta - \\theta_0$)')
        plt.grid(True)

        # Plot 5: Control input
        # only one input...
        plt.subplot(5, 1, 5)
        plt.plot(t, u[0, :], label='$\delta_{th}$'+f' {name}', color=color, l
    inestyle='dashdot'          )#,marker='^',markevery=0.1)
        plt.plot(t, u[1, :], label=f'$\delta_e$ {name} [rad]', color=color, l
    inestyle='solid',marker='o',markevery=0.1  )#,marker='o',markevery
    =0.1)
        plt.xlabel('Time [s]')
        plt.ylabel('Control', wrap=True)
        plt.legend(loc='center right')  # Set legend location to center ri
    ght

        plt.grid(True)

        # Adjust layout and show the figure
        plt.tight_layout()

    if save:
        figname = "nonlinear_track_compare"
        plt.savefig(f'Output/aero_7dim_asym/Gramian_comparison/Pitch_angle
    /Nonlinear_Pitch_Integrator/{figname}.png', dpi=300)
```

```
261         # path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDri
    ve-Personal/UCSD/Research/AIAA 2026/Reachability Paper'
262         # plt.savefig(f'{path_to_paper}/Figures/Tracking/gramian_comp/{fig
    name}.png',dpi=300)
263 
264     else:
265         plt.show()
266 
267 def error_bars(sim_dictionary,save=False):
268     ''' This function makes error bars to compare the models'''
269 
270     plt.figure(figsize=(8,6))
271     plt.rcParams.update({'font.size': 15})
272     plt.rcParams['lines.linewidth'] = 4
273     plt.rcParams['lines.markersize'] = 12
274 
275     # Error computation for all trajectories
276     print("\n" + "="*70)
277     print("TRACKING ERROR ANALYSIS")
278     print("="*70)
279 
280     for model, data in sim_dictionary.items():
281         if model == 'ref':
282             continue
283         t = data['t']
284         x = data['x']
285         tp = data['tp']
286         u = data['u']
287 
288         # Compute errors
289         error = x[0:4, :] - tp
290         vel_err =   np.sqrt(trapezoid(error[0, :]**2, t))
291         aoa_err =   np.sqrt(trapezoid(error[1, :]**2, t))
292         pr_err =    np.sqrt(trapezoid(error[2, :]**2, t))
293         pitch_err = np.sqrt(trapezoid(error[3, :]**2, t))
294         error_norm = np.sqrt(trapezoid(np.sum(error**2, 0), t))
295         u_norm = np.sqrt(trapezoid(np.sum(u**2,axis=0),t))
296 
297         data['error'] = {
298                 'vel_err': vel_err,
299                 'aoa_err': aoa_err,
300                 'pr_err': pr_err,
301                 'pitch_err': pitch_err,
302                 'error_norm': error_norm,
303                 'u_norm':u_norm
304         }
305         print(f"\n{model.upper()} Model:")
306         print(f"  Total error (L2 norm): {error_norm:.6f}")
307         print(f"  Velocity error:        {vel_err:.6f}")
308         print(f"  AOA error:             {aoa_err:.6f}")
309         print(f"  Pitch rate error:      {pr_err:.6f}")
310         print(f"  Pitch angle error:     {pitch_err:.6f}")
311         print(f"  Input L2 norm:         {u_norm:.6f}")
312 
313         if model == 'initial':
314             init_vel = vel_err
315             init_aoa = aoa_err
316             init_pr = pr_err
```

```
            init_pitch = pitch_err
            init_total = error_norm
            init_u = u_norm
            continue   # This skips the rest of the for loop and starts again.

        vel_rel_err = (init_vel - vel_err)/init_vel
        aoa_rel_err = (init_aoa - aoa_err)/init_aoa
        pr_rel_err = (init_pr - pr_err)/init_pr
        pitch_rel_err = (init_pitch - pitch_err)/init_pitch
        total_rel_err = (init_total - error_norm)/init_total
        u_rel_norm = (init_u - u_norm)/init_u

        print(f"  Velocity relative error:        {vel_rel_err*100:.2f}")
        print(f"  AOA relative error:             {aoa_rel_err*100:.2f}")
        print(f"  Pitch rate relative error:      {pr_rel_err*100:.2f}")
        print(f"  Pitch angle relative error:     {pitch_rel_err*100:.2f}")
        print(f"  Input relative L2 norm:         {u_rel_norm*100:.2f}")


        data['error']['relative'] = {
            'vel':vel_rel_err     * 100 ,
            'aoa':aoa_rel_err     * 100 ,
            'pr':pr_rel_err       * 100 ,
            'pitch':pitch_rel_err * 100 ,
            'total':total_rel_err * 100 ,
            'u':u_rel_norm
        }

    print("="*70 + "\n")

    models_list_label = ['vm','dm','vmdc']
    models_list_actual = [key for key in sim_dictionary.keys() if key not in ['initial', 'ref']]
    error_types = ['vel', 'aoa', 'pr', 'pitch']
    error_labels = ['Velocity', 'AOA', 'Pitch Rate', 'Pitch Angle']

    # Define bar patterns/hatches
    hatches = ['//', '\\', 'o', '*', 'xxx']

    # Set up bar positions
    x_pos = np.arange(len(models_list_actual))
    bar_width = 0.15

    # Plot bars for each error type
    for i, (error_type, error_label, hatch) in enumerate(zip(error_types, error_labels, hatches)):
        offset = (i - 2) * bar_width
        values = [sim_dictionary[model]['error']['relative'][error_type] for model in models_list_actual]
        plt.bar(x_pos + offset, values, bar_width, label=error_label, hatch=hatch, edgecolor='black', linewidth=1.5)

    # Customize the plot
    plt.xlabel('Model Type')
    plt.ylabel('Decrease in Error (%)')
    # plt.title('Tracking Error Comparison Across Models')
    plt.xticks(x_pos, [m.upper() for m in models_list_label])
```

```
370     plt.legend(loc='best')
371     plt.grid(True, axis='y')
372     if save:
373         figname = 'error_nonlinear'
374         plt.savefig(f'Output/5_dec_vars/Nonlinear_Pitch_Integrator/{fignam
        e}.png', dpi=300)
375         path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive
        -Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures/Track
        ing'
376         plt.savefig(f'{path_to_paper}/{figname}')
377     else:
378         plt.show()
379 
380 def linear_sim_plots(sim_dictionary,models,case,save=False):
381     plt.rcParams.update({'font.size': 15})
382     plt.rcParams['lines.linewidth'] = 4
383     plt.rcParams["legend.handlelength"] = 3.5
384     plt.rcParams['lines.markersize'] = 12
385 
386     if case == 'pitch':
387         mosaic = [
388             ['V'],
389             ['Elevator'],
390         ]
391         figsize = (9, 6)
392     elif case == 'velocity':
393         mosaic = [
394             ['V'],
395             ['Thrust'],
396             ['Elevator'],
397         ]
398         figsize = (11,8)
399     else:
400         figsize = (15,8)
401     fig, axd = plt.subplot_mosaic(mosaic, figsize=figsize, layout='constra
        ined')
402 
403     for i, m in enumerate(models):
404 
405         t =     sim_dictionary[f'{m}_{case}']['t']
406         x =     sim_dictionary[f'{m}_{case}']['x']
407         u =     sim_dictionary[f'{m}_{case}']['u']
408         tp =    sim_dictionary[f'{m}_{case}']['tp']
409         style = sim_dictionary[f'{m}_{case}']['style']
410         color = sim_dictionary[f'{m}_{case}']['color']
411 
412         name = sim_dictionary[f'{m}_{case}']['name']
413 
414         if name == 'inf_set':
415             name = 'Reach Proj'
416         elif name == 'gramian_eigs':
417             name = 'Gram Eigs'
418             style = 'dashdot'
419         else:
420             name = name.upper()
421 
422         if i == 0:
423             style = 'dashed'
```

```
        elif i == 1:
            style = 'dotted'
        elif i == 2:
            style = 'dashdot'
        elif i ==3 :
            style = 'solid'
        else:
            style = 'dashed'

        if case == 'velocity':
            state = 0
            ylabel = 'Perturbed Airspeed\n [m/s] ($V - V_0$)'
        elif case == 'pitch':
            state = 3
            ylabel = 'Perturbed Pitch angle\n [deg] ($\\theta - \\theta_0$)'

        # Airspeed
        axd['V'].plot(t, x[state, :], label=name, color=color, linestyle=style)
        axd['V'].legend(loc='best')  # Set legend location to center right

        axd['V'].grid(True)
        axd['V'].set_ylabel(ylabel)

        if case == 'velocity':
            # Thrust control
            axd['Thrust'].plot(t, u[0, :], label='$\delta_{th}$'+f' {name}', color=color, linestyle=style         )#,marker='^',markevery=0.1)
            axd['Thrust'].grid(True)
            axd['Thrust'].set_ylabel('Thrust Input ($\delta_{th}$)')

        # Elevator Control
        axd['Elevator'].plot(t, np.rad2deg(u[1, :]), label=f'$\delta_e$ {name} [rad]', color=color, linestyle=style  )#,marker='o',markevery=0.1)
        axd['Elevator'].set_xlabel('Time [s]')
        axd['Elevator'].set_ylabel('Elevator Control\n Surface [deg] ($\delta_e$)', wrap=True)

        axd['Elevator'].grid(True)

    if save:
        figname = f'linear_track_{case}'
        plt.savefig(f'Output/5_dec_vars/Opt_LQ/{figname}.png', dpi=300)
        path_to_paper = '/Users/stevennguyen/Library/CloudStorage/OneDrive-Personal/UCSD/Research/AIAA 2026/Reachability Paper/Figures/Tracking'
        plt.savefig(f'{path_to_paper}/{figname}.png', dpi=300)

    else:
        plt.show()
```

utils/progress_bar.py

```
1 # Print iterations progress
```

```python
import time

def printProgressBar (iteration, total, prefix = '', suffix = '', decimals = 1, length = 100, fill = '■  ', printEnd = "\r"):
    """
    Call in a loop to create terminal progress bar
    @params:
        iteration   - Required  : current iteration (Int)
        total       - Required  : total iterations (Int)
        prefix      - Optional  : prefix string (Str)
        suffix      - Optional  : suffix string (Str)
        decimals    - Optional  : positive number of decimals in percent complete (Int)
        length      - Optional  : character length of bar (Int)
        fill        - Optional  : bar fill character (Str)
        printEnd    - Optional  : end character (e.g. "\r", "\r\n") (Str)
    """
    percent = ("{0:." + str(decimals) + "f}").format(100 * (iteration / float(total)))  # This is the text displayed at end of bar
    filledLength = int(length * iteration // total)   # This is the length of the bar that is filled.
                                                      # iteration // total gives percent completion, then multiply by length of completed bar
    bar = fill * filledLength + '-' * (length - filledLength)   # This is the bar itself, a bunch of blocks and dashes
    print(f'\r{prefix} |{bar}| {percent}% {suffix}', end = printEnd)
    # Print New Line on Complete
    if iteration == total:
        print()

if __name__ == '__main__':
    for i in range(10):
        printProgressBar(i, 9, prefix = 'Progress:', suffix = 'Complete', length = 50)
        time.sleep(0.1)

```

utils/reach_set_data_read.py

```python
import pandas as pd
import numpy as np

def unpack_set_data(csv_input,N_res):
    '''This function reads the CSV file containing the reachability data and
    outputs: [LX1, LX2, LX3, LX4, control_cost, final_positions]. final_positions[...,i]
    gives the i-th component of the final state corresponding to that initial costate'''

    # Load the data from the CSV file
    df = pd.read_csv(csv_input)

    # Extract the number of resolutions (N_res) from the data
```

```
    # N_res = int(round(len(df["lx1"]) ** (1 / 4)))  # Assuming a 4D grid

    # Reshape the LX arrays
    LX1 = df["LX1"].values.reshape(N_res, N_res, N_res, N_res)
    LX2 = df["LX2"].values.reshape(N_res, N_res, N_res, N_res)
    LX3 = df["LX3"].values.reshape(N_res, N_res, N_res, N_res)
    LX4 = df["LX4"].values.reshape(N_res, N_res, N_res, N_res)

    # Reshape the final_positions array
    final_positions = np.zeros((N_res, N_res, N_res, N_res, 4))
    final_positions[..., 0] = df["final_x1"].values.reshape(N_res, N_res, N_res, N_res)
    final_positions[..., 1] = df["final_x2"].values.reshape(N_res, N_res, N_res, N_res)
    final_positions[..., 2] = df["final_x3"].values.reshape(N_res, N_res, N_res, N_res)
    final_positions[..., 3] = df["final_x4"].values.reshape(N_res, N_res, N_res, N_res)

    # final_positions = final_positions.reshape(N_res,N_res,N_res,N_res,4)


    return LX1, LX2, LX3, LX4, final_positions


if __name__ == "__main__":
    N_res = 5
    [LX1,LX2,LX3,LX4,fp] = unpack_set_data("reachability_data.csv",N_res)
    print(LX1)
    print(fp)
```

utils/reach_set_functions.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.spatial import ConvexHull
from utils.reach_set_data_read import unpack_set_data
import pandas as pd

def compute_hull(final_positions, cost_threshold = 1, control_cost = 1):
    ''' This function computes convex hull of our final positions, only considering points associated with control cost under cost_threshold

    INPUT: control_cost, final_positions, max value of control: cost_threshold
    OUTPUT: volume of convex hull'''

    if final_positions.shape == (6,6,6,6,4):
        filtered_positions = final_positions[control_cost <= cost_threshold]
    else:
        filtered_positions = final_positions.reshape((-1,4))

    if len(filtered_positions) < 4:
        print("Not enough points to compute a convex hull. Must recompute
```

```
    p-norm reach set.")
19          return 0
20
21      # Compute the convex hull of the filtered positions
22      hull = ConvexHull(filtered_positions)
23
24      # Return the volume of the convex hull
25      return hull
26
27  def plot_hull_projection(hull, col_x, col_y, hull_plot_label="Convex Hull", p
        oint_label = "Projected Points", hull_color = "r",
28                           x_label=None, y_label=None, title=None, hatchstyl
        e=None, ls=None, points = False):
29      """
30      Plots the projection of a 4D convex hull onto two specified dimensions
        .
31
32      Parameters:
33      hull: A ConvexHull object representing the 4D convex hull.
34      col_x: Index of the dimension to plot on the x-axis.
35      col_y: Index of the dimension to plot on the y-axis.
36      """
37      # Extract the vertices of the convex hull
38      vertices = hull.points[hull.vertices]
39
40      # Project the vertices onto the specified dimensions
41      x = vertices[:, col_x]
42      y = vertices[:, col_y]
43
44      # Create a 2D convex hull of the projected points
45      # This is useful so that we can pull out its vertices and draw boundar
        ies of the projection.
46      # If we only plotted (x,y), we would get messy zig zag since some poin
        ts are only vertices in higher dimensions.
47      projected_hull = ConvexHull(np.column_stack((x, y)))
48
49      # Extract the vertices of the projected hull and close the hull by add
        ing the first vertex to the end
50      hull_vertices = np.append(projected_hull.vertices, projected_hull.vert
        ices[0])
51
52      # Plot the projected convex hull
53      # plt.figure(figsize=(8, 6))
54      plt.plot(x[hull_vertices], y[hull_vertices], f'{hull_color}', label=f"{h
        ull_plot_label}", linestyle=ls)
55      plt.fill(x[hull_vertices], y[hull_vertices], f'{hull_color}', alpha=0.2, h
        atch=hatchstyle)
56      if points:
57          plt.scatter(x, y, s=35, label=f"{point_label}")
58
59      plt.xlabel(x_label if x_label else f"Dimension {col_x}")
60      plt.ylabel(y_label if y_label else f"Dimension {col_y}")
61      # plt.title(title if title else f"Projection of Convex Hull onto Dimen
        sions {col_x} and {col_y}")
62      plt.title(title if title else None)
63
64      # plt.legend()
65      plt.grid()
```

```
    plt.axis('equal')
    # plt.show()


def sample_ellipsoid_surface(A, gamma, N, rng,savepts=False, filename="Outputs/gramian_set.csv"):
    """
    Draw N points on the ellipsoid surface {x in R^n : x^T A x = gamma}.
    Map the unit sphere through x = sqrt(gamma) * R^{-1} u, A = R^T R (Cholesky),
    ||u|| = 1  =>  x^T A x = gamma.
    """
    n = A.shape[0]
    R = np.linalg.cholesky(A).T                         # A = R^T R
    g = rng.standard_normal((N, n))
    u = g / np.linalg.norm(g, axis=1, keepdims=True)  # uniform on unit sphere
    X =  np.sqrt(gamma) * np.linalg.solve(R, u.T).T  # (N, n) on the surface
    if savepts:
        # save the points
        # Create dynamic column names based on state dimension
        data = {}

        # Add final positions
        for i in range(n):
            data[f"final_x{i+1}"] = X[:, i]

        df = pd.DataFrame(data)

        # Save to CSV
        df.to_csv(filename, index=False)

        print(f"\nData saved to {filename}")
        pass
    return ConvexHull(X)


def plot_projected_boundary(A, gamma, label=None, color="C0", ax=None,
                            N=20000, seed=0, draw_analytic=True):
    """
    Sample the 4D ellipsoid SURFACE, project onto (x1,x2) and (x3,x4).
    The convex hull of the projected surface points is the shadow's boundary.
    Optionally overlay the exact analytic shadow ellipse.
    """
    A = np.asarray(A, dtype=float)
    if not np.allclose(A, A.T, atol=1e-10):
        raise ValueError("A must be symmetric.")
    if np.min(np.linalg.eigvalsh(A)) <= 0:
        raise ValueError("A must be positive definite.")
    if gamma <= 0:
        raise ValueError("gamma must be positive.")

    if ax is None:
        fig, axes = plt.subplots(1, 2, figsize=(9, 4.2))
    else:
        axes = np.atleast_1d(ax).ravel()
```

```
        fig = axes[0].figure

    rng = np.random.default_rng(seed)
    X = sample_ellipsoid_surface(A, gamma, N, rng)

    Ainv = np.linalg.inv(A)
    theta = np.linspace(0, 2 * np.pi, 400)
    circle = np.vstack([np.cos(theta), np.sin(theta)])
    pairs = [(0, 1), (2, 3)]

    for k, (ax_k, (i, j)) in enumerate(zip(axes, pairs)):
        Y = X[:, [i, j]]                              # projected surface
    points

        # outline = convex hull of the projected points (the shadow bounda
    ry)
        from scipy.spatial import ConvexHull
        h = ConvexHull(Y)
        ring = np.append(h.vertices, h.vertices[0])   # close the loop
        ax_k.plot(Y[ring, 0], Y[ring, 1], color=color, lw=1.5,
                  label=label if k == 0 else None)

        if draw_analytic:
            P = Ainv[np.ix_([i, j], [i, j])]
            L = np.linalg.cholesky(P)
            pts = np.sqrt(gamma) * (L @ circle)
            ax_k.plot(pts[0], pts[1], color="k", lw=1.0, ls="--")

        ax_k.axhline(0, color="k", lw=0.5, alpha=0.3)
        ax_k.axvline(0, color="k", lw=0.5, alpha=0.3)
        ax_k.set_xlabel(f"$x_{i+1}$")
        ax_k.set_ylabel(f"$x_{j+1}$")
        ax_k.set_title(f"proj onto ($x_{i+1}$, $x_{j+1}$)")
        # ax_k.set_aspect("equal", adjustable="datalim")
        ax_k.grid(True, alpha=0.3)

    fig.tight_layout()
    return fig, axes



if __name__ == "__main__":
    N_res = 6
    [LX1, LX2, LX3, LX4, control_cost, final_positions] = unpack_set_data("O
    utput/aero_7dim_asym/N6_initial.csv",N_res)
    cost_threshold = 1  # Define your cost threshold here
    hull = compute_hull(control_cost, final_positions, cost_threshold)

    # set_fill = compute_percentage_under(control_cost.reshape(-1,1),1)
    print(f"Computed Volume of Reachable Set: {hull.volume}")
    # print(f"Set coverage: {set_fill}%")

    # Assuming `hull` is the 4D convex hull computed earlier
    plot_hull_projection(hull, col_x=0, col_y=1)  # Project onto dimension
    s 0 and 1
    plot_hull_projection(hull, col_x=2, col_y=3)  # Project onto dimension
    s 2 and 3
```

utils/read_hdf5.py

```
import h5py
```

```python

def print_hdf5_contents(filename):
    """Reads an HDF5 file and prints its contents."""
    def recursively_print_group(name, obj):
        """Recursively prints the contents of a group or dataset."""
        if isinstance(obj, h5py.Group):
            print(f"Group: {name}")
        elif isinstance(obj, h5py.Dataset):
            print(f"Dataset: {name}, Shape: {obj.shape}, Data: {obj[()]}")

    with h5py.File(filename, "r") as f:
        print(f"Reading HDF5 file: {filename}")
        f.visititems(recursively_print_group)

def print_results_group(filename):
    """Reads an HDF5 file and prints the contents of the 'results' group."
    ""
    with h5py.File(filename, "r") as f:
        if 'results' in f:
            print("Reading 'results' group:")
            results_group = f['results']
            for name, obj in results_group.items():
                if isinstance(obj, h5py.Dataset):
                    print(f"Dataset: {name}, Shape: {obj.shape}, Data: {ob
    j[()]}")
                elif isinstance(obj, h5py.Group):
                    print(f"Nested Group: {name}")
        else:
            print("'results' group not found in the HDF5 file.")

def extract_dataset_from_results(filename, dataset_name):
    """
    Extracts a NumPy array from a specific dataset in the 'results' group
    of an HDF5 file.

    Args:
        filename (str): Path to the HDF5 file.
        dataset_name (str): Name of the dataset to extract.

    Returns:
        numpy.ndarray: The data stored in the specified dataset.
    """
    with h5py.File(filename, "r") as f:
        if 'results' in f:
            results_group = f['results']
            if dataset_name in results_group:
                dataset = results_group[dataset_name]
                return dataset[()]  # Extract and return as a NumPy array
            else:
                raise KeyError(f"Dataset '{dataset_name}' not found in the
     'results' group.")
        else:
            raise KeyError("'results' group not found in the HDF5 file.")

if __name__ == "__main__":
    # Example usage

    print('sm0')
```

```
    filename = "Output/5_dec_vars/10-23-26/N6_volmax_sm0/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-23-26/N6_115max_sm0/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    # ------------------------------------------------------
    print('------------------------------------------------------')
    print('sm005')
    filename = "Output/5_dec_vars/10-22-26/N6_volmax_sm005/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-22-26/N6_115max_sm005/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    # ------------------------------------------------------
    print('------------------------------------------------------')
    print('sm1')
    filename = "Output/5_dec_vars/10-21-26/N6_volmax_sm1/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-21-26/N6_115max_sm1/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")


    # ------------------------------------------------------
    print('------------------------------------------------------')
    print('sm015')
    filename = "Output/5_dec_vars/10-22-26/N6_volmax_sm015/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-22-26/N6_115max_sm015/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

```

```
    filename = "Output/5_dec_vars/10-22-26/N6_90max_sm015/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")


    # ----------------------------------------------------
    print('----------------------------------------------------')
    print('sm2')
    filename = "Output/5_dec_vars/10-23-26/N6_volmax_sm2/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-23-26/N6_115max_sm2/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-23-26/N6_90max_sm2/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    # ----------------------------------------------------
    print('----------------------------------------------------')
    print('sm025')
    filename = "Output/5_dec_vars/10-20-26/N6_volmax_sm025/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-20-26/N6_115max_sm025/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")

    filename = "Output/5_dec_vars/10-22-26/N6_90max_sm025/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")


    # ----------------------------------------------------
    print('----------------------------------------------------')
    print('sm4')
    filename = "Output/5_dec_vars/10-22-26/N6_volmax_sm4/opt_vars.hdf5"
    dataset_name = "x"  # Replace with the name of the dataset you want to extract
    data = extract_dataset_from_results(filename, dataset_name)
    print(f"Extracted data from dataset '{dataset_name}':\n{data}")
```

```
156
157     filename = "Output/5_dec_vars/10-22-26/N6_90max_sm4/opt_vars.hdf5"
158     dataset_name = "x"  # Replace with the name of the dataset you want to
         extract
159     data = extract_dataset_from_results(filename, dataset_name)
160     print(f"Extracted data from dataset '{dataset_name}':\n{data}")
161
162     filename = "Output/5_dec_vars/10-29-26/N6_volmax_90con_sm4/opt_vars.hd
        f5"
163     dataset_name = "x"  # Replace with the name of the dataset you want to
         extract
164     data = extract_dataset_from_results(filename, dataset_name)
165     print(f"Extracted data from dataset '{dataset_name}':\n{data}")
166
167     # ------------------------------------------------------
168     print('------------------------------------------------------')
169     print('final')
170     filename = "Output/5_dec_vars/N6_volmax/opt_vars.hdf5"
171     dataset_name = "x"  # Replace with the name of the dataset you want to
         extract
172     data = extract_dataset_from_results(filename, dataset_name)
173     print(f"Extracted data from dataset '{dataset_name}':\n{data}")
174
175     filename = "Output/5_dec_vars/N6_115max/opt_vars.hdf5"
176     dataset_name = "x"  # Replace with the name of the dataset you want to
         extract
177     data = extract_dataset_from_results(filename, dataset_name)
178     print(f"Extracted data from dataset '{dataset_name}':\n{data}")
179
180     filename = "Output/5_dec_vars/N6_volmax_115con/opt_vars.hdf5"
181     dataset_name = "x"  # Replace with the name of the dataset you want to
         extract
182     data = extract_dataset_from_results(filename, dataset_name)
183     print(f"Extracted data from dataset '{dataset_name}':\n{data}")
184
185     # ------------------------------------------------------
186     print('------------------------------------------------------')
187     print('gram comp')
188     print('mine')
189     filename = "Output/5_dec_vars/Gramian_comparison/inf_set/opt_vars.hdf5"
190     dataset_name = "x"  # Replace with the name of the dataset you want to
         extract
191     data = extract_dataset_from_results(filename, dataset_name)
192     print(f"Extracted data from dataset '{dataset_name}':\n{data}")
193
194     print('theirs')
195     filename = "Output/5_dec_vars/Gramian_comparison/gramian_eigs/opt_vars
        .hdf5"
196     dataset_name = "x"  # Replace with the name of the dataset you want to
         extract
197     data = extract_dataset_from_results(filename, dataset_name)
198     print(f"Extracted data from dataset '{dataset_name}':\n{data}")
```

utils/ref_trackers.py

```
1 import numpy as np
```

```python
from scipy.integrate import solve_ivp, trapezoid
from scipy.linalg import inv
from scipy.interpolate import RegularGridInterpolator
from utils.solve_rde import solve_rde
import control as ct

''' This file will contain a function that takes an A,B matrix, an instruction on the type
of reference tracking to do, and then just return the states, reference, and control.'''

def lqr_ref_track(A,B,case,Tf=5.0, u_ub = [1, 1], u_lb = [-1, -1], do_meas_noise=False, do_proc_noise=False):

    if case == 'velocity':
        C = np.array([[1,0,0,0]])
        x1 = 4
    elif case == 'pitch':
        C = np.array([[0,0,0,1]])
        x1 = 0.5
    elif case == 'aoa':
        C = np.array([[0,1,0,0]])
        x1 = 0.5

    n = A.shape[0]
    m = B.shape[1]

    # LQ Tracking Cost weights
    Q = np.diag(1000 * C.flatten())
    if case == 'velocity':
        R = 1000 * np.identity(m)
    elif case == 'pitch':
        R = 100 * np.identity(m)
    elif case == 'aoa':
        R = 10 * np.identity(m)

    Rinv = inv(R)

    # Horizon
    T_f = Tf

    # Terminal cost
    P_f = np.zeros((n, n))

    #%% Reference trajectory (state-level reference x_ref(t))

    def x_reference(t):
        # Is a constant reference.
        L = C.T @ np.linalg.inv(C @ C.T)

        yref = np.array([[x1]])
        xref = L @ yref.T
        return xref

    #%% Solve Riccati equations backward in time

    # Backward-time ODE for P(t) and eta(t)
    # equations:
```

```
57      #   -dot P = A^T P + P A - P B R^{-1} B^T P + Q
58      #   -dot eta = (A^T - P B R^{-1} B^T) eta - Q x_ref(t)
59      # Terminal: P(T_f) = P_f, eta(T_f) = -P_f x_ref(T_f)
60
61      t_P_rev, P_t, eta_t = solve_rde(A,B,Q,R,T_f,P_f,x_reference)
62
63      def interp_P_eta(t_query):
64          # simple linear interpolation (clamped)
65          if t_query <= 0.0:
66              return P_t[0], eta_t[0]
67          if t_query >= T_f:
68              return P_t[-1], eta_t[-1]
69          idx = np.searchsorted(t_P_rev, t_query) - 1
70          if idx < 0:
71              idx = 0
72          t0, t1 = t_P_rev[idx], t_P_rev[idx+1]
73          alpha = (t_query - t0) / (t1 - t0)
74          P_interp = (1-alpha) * P_t[idx] + alpha * P_t[idx+1]
75          eta_interp = (1-alpha) * eta_t[idx] + alpha * eta_t[idx+1]
76          return P_interp, eta_interp
77
78      #%% Simulate forward in time with the optimal control law
79      def control_signal(Rinv,B,P,eta,x):
80          u = -Rinv @ B.T @ (P @ x + eta)
81          # u[0] = np.clip(u[0],0,1)
82          # u[1] = np.clip(u[1],-1,1)
83          u[0] = np.clip(u[0],u_lb[0],u_ub[0])
84          u[1] = np.clip(u[1],u_lb[1],u_ub[1])
85          # u = np.clip(u,-1,1)
86          return u
87
88      t_eval_fwd = np.linspace(0.0, T_f, 1001)
89      np.random.seed(30)
90      w = np.random.normal(0,2,size=(len(t_eval_fwd)))
91      w_interp = RegularGridInterpolator([t_eval_fwd],w)
92
93
94      def closed_loop_forward(t, x_flat):
95          x = x_flat.reshape((n,1))
96          P_now, eta_now = interp_P_eta(t)
97          noise = np.asarray([[1],[1],[1],[1]])*w_interp([t])     # Noise af
        fecting measurement
98
99          if do_meas_noise:
100             u = control_signal(Rinv,B,P_now,eta_now,x + noise)
101         else:
102             u = control_signal(Rinv,B,P_now,eta_now,x)
103
104         # u = min_energy_control(A,B,t)
105
106         if do_proc_noise:
107             dx = A@x+ B@u + noise
108         else:
109             dx = A @ x + B @ u
110         return dx.flatten()
111
112     x0 = np.zeros((n,1)).flatten()
113     sol_fwd = solve_ivp(closed_loop_forward, (0.0, T_f), x0, t_eval=t_eval
```

```
    _fwd, rtol=1e-7, atol=1e-9)

    if not sol_fwd.success:
        raise RuntimeError("Forward integration failed: " + str(sol_fwd.message))

    t = sol_fwd.t
    x = sol_fwd.y

    # compute reference and control histories
    xref_hist = np.zeros_like(x)
    u_hist = np.zeros((m,t.size))
    for i, ti in enumerate(t):
        # xr = ref_traj(ti)
        xr = x_reference(ti)

        P_now, eta_now = interp_P_eta(ti)
        noise = np.asarray([[1],[1],[1],[1]]) * w_interp([ti])
        if do_meas_noise:
            u = control_signal(Rinv,B,P_now,eta_now,x[:, i].reshape((n,1)) + noise)
        else:
            u = control_signal(Rinv,B,P_now,eta_now,x[:, i].reshape((n,1)))

        xref_hist[:, i] = xr.flatten()
        u_hist[:,i] = u.flatten()

    error = x - xref_hist

    if case == 'pitch':
        error_norm = np.sqrt(trapezoid(error[3,:]**2,t))
    elif case == 'velocity':
        error_norm = np.sqrt(trapezoid(error[0,:]**2,t))
    return t, x, xref_hist, u_hist, error_norm

def integral_ref_track(A,B,case,Tf=5.0, u_ub = [1,1], u_lb=[-1,-1], do_meas_noise = False, do_proc_noise = False):
    n = A.shape[0]
    m = B.shape[1]

    if case == 'velocity':
        C = np.array([[1,0,0,0]])
        x1 = 4
    elif case == 'pitch':
        C = np.array([[0,0,0,1]])
        x1 = 0.5

    k = C.shape[0] # outputs
    T_f = Tf
    # Add integrator state:
    Abar = np.concatenate((np.concatenate((A,np.zeros((n,k))),axis=1),np.concatenate((C,np.zeros((k,k))),axis=1)),axis=0)
    Bbar = np.concatenate((B,np.zeros((k,m))),axis=0)

    # print(f'Abar: {Abar}')
    # print(f'Bbar: {Bbar}')

    # LQR Cost weights
```

```
167     if case == 'pitch':
168         Q = np.diag([0,1,0,1,100])
169         R = np.diag([0.1,10]) # penalize elevator usage more to avoid too
    much variation in AOA.
170         # Q = np.diag([1, 100, 1, 100, 100])
171         # R = np.diag([0.1, 1000])
172     elif case == 'velocity':
173         Q = np.diag([1, 1, 1, 1, 1])
174         R = np.diag([0.1,0.1])
175         # Q = np.diag([1, 1, 1, 100, 1])
176         # R = np.diag([1, 100])
177 
178     #%% Reference trajectory (state-level reference x_ref(t))
179 
180     def x_reference(t):
181         # x1 = 4
182         return x1
183 
184     #%% Simulate forward in time with the control
185 
186     K = ct.lqr(Abar,Bbar,Q,R)[0]
187     def control_signal(x):
188         x = x.reshape((n+1,1))
189         u = -K@x
190         u[0] = np.clip(u[0],u_lb[0],u_ub[0])
191         u[1] = np.clip(u[1],u_lb[1],u_ub[1])
192         # u = np.clip(u,-1,1)
193         return u
194 
195     t_eval_fwd = np.linspace(0.0, T_f, 1001)
196     np.random.seed(30)
197     w = np.random.normal(0,0.5,size=(len(t_eval_fwd)))
198     w_interp = RegularGridInterpolator([t_eval_fwd],w)
199 
200     def closed_loop_forward(t, x_flat):
201         x = x_flat.reshape((n+1,1))
202         noise = np.asarray([[1],[1],[1],[1],[1]])*w_interp([t])     # Nois
    e affecting measurement
203 
204         if do_meas_noise:
205             u = control_signal(x + noise)
206         else:
207             u = control_signal(x)
208 
209         if do_proc_noise:
210             dx = Abar @ x + Bbar @ u - np.concatenate((np.zeros((4,1)),np.a
    sarray([[x_reference(t)]])),axis=0) + noise
211         else:
212             dx = Abar @ x + Bbar @ u - np.concatenate((np.zeros((4,1)),np.a
    sarray([[x_reference(t)]])),axis=0)
213 
214         if case=='velocity':
215             threshold=5
216         elif case=='pitch':
217             threshold=1
218         if x[4] > threshold:      # prevent wind up, if integrator state t
    oo large, keep constant...
219             dx[4] = min(dx[4],0)
```

```
        elif x[4] < -threshold:
            dx[4] = max(dx[4],0)

        return dx.flatten()

    x0 = np.zeros((n+1,1)).flatten()
    sol_fwd = solve_ivp(closed_loop_forward, (0.0, T_f), x0, t_eval=t_eval_fwd, rtol=1e-7, atol=1e-9)

    if not sol_fwd.success:
        raise RuntimeError("Forward integration failed: " + str(sol_fwd.message))

    t = sol_fwd.t
    y = sol_fwd.y
    x = y[:n,:]
    integrator = y[n:,:]

    # compute reference and control histories
    xref_hist = np.zeros(t.size) # Reference is only one state...
    u_hist = np.zeros((m,t.size))
    for i, ti in enumerate(t):
        xr = x_reference(ti)
        noise = np.asarray([[1],[1],[1],[1],[1]])*w_interp([ti])
        if do_meas_noise:
            u = control_signal(y[:,i].reshape((n+1,1)) + noise)
        else:
            u = control_signal(y[:,i].reshape((n+1,1)))
        xref_hist[i] = xr
        u_hist[:,i] = u.flatten()

    error = C@x - xref_hist
    error_norm = np.sqrt(trapezoid(error**2,t))

    return t, y, xref_hist, u_hist, error_norm[0]
```

utils/solve_rde.py

```
import numpy as np
from scipy.integrate import solve_ivp
from scipy.linalg import inv
import matplotlib.pyplot as plt
from functools import partial

def solve_rde(A, B, Q, R, T_f, P_f=None, ref_traj = None, t_eval=None, method='RK45'):
    """
    Solves the matrix Riccati differential equation backward in time:
        -dot P = A^T P + P A - P B R^{-1} B^T P + Q,   P(T_f)=P_f
    If n-dimensional reference trajectory provided, also solves:
        -dot eta = (A^T - P B R^{-1} B^T) eta - Q x_ref(t), eta(T_f) = -P_f * ref_traj(T_f)
    Returns:
        t_grid: increasing times from 0..T_f (numpy array)
        P_t: array shape (len(t_grid), n, n) with P(t) on that grid
```

```
    Inputs:
        A, B, Q, R: numpy arrays with compatible dimensions
        T_f: final time (float)
        P_f: terminal matrix (if None, defaults to zero matrix)
        ref_traj: Function of time that returns a STATE (n-dim) reference trajectory for all t in [0,T_f]. If not included,
                  it will be assumed that there is no pre-specified trajectory. If included, this function will also
                  solve for the value of eta that yields optimal control for tracking...
        t_eval: optional increasing time grid (0..T_f). If provided, will be used (must be increasing).
                Internally we integrate backward, so we create the reversed grid for solve_ivp.
        method: ODE solver to use (default 'RK45'; use 'Radau' or 'BDF' if stiff)

    Outputs:
        t_grid: increasing times from 0..T_f (numpy array)
        P_t: array shape (len(t_grid), n, n) with P_t(i,:) = P(t_grid(i))
    """
    A = np.atleast_2d(A)
    B = np.atleast_2d(B)
    Q = np.atleast_2d(Q)
    R = np.atleast_2d(R)
    n = A.shape[0]

    if P_f is None:
        P_f = np.zeros((n, n))
    Rinv = inv(R)

    def backward_ode_full_args(t, y_flat, ref_traj):
        if not ref_traj:
            P = y_flat.reshape((n, n))
            term = A.T @ P + P @ A - P @ B @ Rinv @ B.T @ P + Q
            dotP = -term
            return dotP.reshape(-1)
        else:
            P_flat = y_flat[:n*n]
            eta_flat = y_flat[n*n:]
            P = P_flat.reshape((n,n))
            eta = eta_flat.reshape((n,1))

            term = A.T @ P + P @ A - P @ B @ Rinv @ B.T @ P + Q
            dotP = -term
            xref = np.asarray(ref_traj(t)).reshape((n,1))
            RHS_eta = (A.T - P @ B @ Rinv @ B.T) @ eta - Q @ xref
            doteta = -RHS_eta

            dy = np.concatenate([dotP.reshape(-1), doteta.reshape(-1)])
            return dy

    backward_ode = partial(backward_ode_full_args,ref_traj=ref_traj)

    # prepare t_eval for solve_ivp (it wants the mesh in the direction we integrate)
    if t_eval is None:
        t_eval_back = np.linspace(T_f, 0.0, 1000)  # backward mesh
```

```
    else:
        # ensure it's increasing 0..T_f, then reverse for backward integra
    tion
        t_eval = np.array(t_eval)
        if np.min(t_eval) < 0 or np.max(t_eval) > T_f:
            raise ValueError("t_eval must be within [0, T_f].")
        if not np.all(np.diff(t_eval) >= 0):
            raise ValueError("t_eval must be increasing.")
        t_eval_back = t_eval[::-1]  # solve from T_f->0

    if ref_traj is not None:
        xref_f = np.asarray(ref_traj(T_f)).reshape((n,1))
        eta_f = -P_f @ xref_f
        yT = np.concatenate([P_f.reshape(-1), eta_f.reshape(-1)])
        do_eta = True
    else:
        yT = P_f.reshape(-1)
        do_eta = False

    sol_back = solve_ivp(backward_ode, (T_f, 0.0), yT, t_eval=t_eval_back, m
    ethod=method, rtol=1e-7, atol=1e-9)

    if not sol_back.success:
        raise RuntimeError("Riccati backward integration failed: " + str(s
    ol_back.message))

    # reverse to increasing time
    t_grid = sol_back.t[::-1].copy()
    y_rev = sol_back.y[:, ::-1].copy()

    nt = t_grid.size
    P_t = np.zeros((nt, n, n))
    if ref_traj is not None:
        eta_t = np.zeros((nt,n,1))
        for i in range(nt):
            P_t[i] = y_rev[:n*n, i].reshape((n,n))
            eta_t[i] = y_rev[n*n:,i].reshape((n,1))
        return t_grid, P_t, eta_t
    else:
        for i in range(nt):
            P_t[i] = y_rev[:, i].reshape((n, n))
        return t_grid, P_t

# -------------------------
# Example usage
# -------------------------
if __name__ == "__main__":

    A = np.array([[0., 1.],
                  [0., 0.]])
    B = np.array([[0.],
                  [1.]])
    Q = np.diag([50., 1.])
    R = np.array([[0.1]])
    T_f = 1

    def xref(t):
        return [np.sin(t),np.cos(t)]
```

```
122
123     t_grid, P_t, eta_t= solve_rde(A, B, Q, R, T_f, method='RK45', ref_traj=x
        ref)
124
125     # plot one element of P(t) as a sanity check
126     plt.plot(t_grid, P_t[:,0,0])
127     plt.plot(t_grid,eta_t[:,1])
128     plt.plot(t_grid,eta_t[:,0])
129     plt.xlabel('time [s]')
130     plt.ylabel('P11(t)')
131     plt.title('Riccati solution P_{11}(t)')
132     plt.grid(True)
133     plt.show()
```

utils/transient_errors.py

```
  1 import numpy as np
  2 from scipy.integrate import trapezoid
  3 from scipy.signal import find_peaks
  4
  5 def max_peak(t, x, prominence=None):
  6     maxima, _ = find_peaks(x, prominence=prominence)
  7     minima, _ = find_peaks(-x, prominence=prominence)
  8     ext = np.concatenate((maxima, minima))
  9     if ext.size == 0:
 10         print('issue computing rise/settle/os')
 11         return None, None, None
 12     i = ext[np.argmax(np.abs(x[ext]))]
 13     return t[i], x[i], i
 14
 15 def rise_settle(t,state_signal,trim_signal,initial_time,threshold=0.1,stat
        e_track=3):
 16     # This will be a specialized function to compute rise time for my nonl
        inear reference tracking sims.
 17     # It will be calculated by checking all the times at which a given sig
        nal crosses 10% of the initial value
 18     # This will yield
 19
 20     '''
 21     Inputs:
 22     t - time signal
 23     x - state signal
 24     initial_time - Time that we treat as the "initial" time for computing
        rise time. We want to know when the state reaches (threshold * 100
        )% of its value at the initial time.
 25     threshold - How close the signal needs to be to equilibrium before we
        determine it has finished rising
 26
 27     Outputs:
 28     rise time'''
 29
 30     new_t = t[t > initial_time]
 31     x = state_signal[:-1,:] - trim_signal
 32     x = x[:,t > initial_time]
 33
```

```
    t = new_t

    x_init = x[state_track,0]

    bar = np.abs(threshold * x_init)
    time_below = t[np.abs(x[state_track,:]) <= bar]
    rise = time_below[0] - initial_time

    # settle
    time_above = t[np.abs(x[state_track,:]) >= bar]
    settle = time_above[-1] - initial_time

    overshoot_time, overshoot_peak, _idx = max_peak(t,x[state_track,:])
    overshoot_pct = np.abs(overshoot_peak) / np.abs(x[state_track,0])
    # print(f'Rise time: {rise}, Settle time: {settle}, Overshoot: (Peak)
    {overshoot_peak}, (PCT) {overshoot_pct}%')

    return rise, settle, overshoot_pct

def transient_errors(sim_dictionary, switch_time = 20):
    print("TRACKING ERROR ANALYSIS")
    print("="*70)

    for model, data in sim_dictionary.items():
        if model == 'ref':
            continue
        t = data['t']
        x = data['x']
        tp = data['tp']
        u = data['u']

        # print('here again')
        # print(u[:,599:605])

        rise, settle, os = rise_settle(t,x,tp,switch_time,threshold=0.05)


        # Compute errors
        error = x[0:4, :] - tp
        vel_err =   np.sqrt(trapezoid(error[0, :]**2, t))
        aoa_err =   np.sqrt(trapezoid(error[1, :]**2, t))
        pr_err =    np.sqrt(trapezoid(error[2, :]**2, t))
        pitch_err = np.sqrt(trapezoid(error[3, :]**2, t))
        error_norm = np.sqrt(trapezoid(np.sum(error**2, 0), t))
        u_norm = np.sqrt(trapezoid(np.sum(u**2,axis=0),t))
        elevator_norm = np.sqrt(trapezoid(u[1,:]**2,t))

        data['error'] = {
                'vel_err': vel_err,
                'aoa_err': aoa_err,
                'pr_err': pr_err,
                'pitch_err': pitch_err,
                'error_norm': error_norm,
                'u_norm':u_norm,
                'elev_norm':elevator_norm
        }
        print(f"\n{model.upper()} Model:")
        print(f"  Total error (L2 norm): {error_norm:.6f}")
```

```
91          print(f"  Velocity error:         {vel_err:.6f}")
92          print(f"  AOA error:              {aoa_err:.6f}")
93          print(f"  Pitch rate error:       {pr_err:.6f}")
94          print(f"  Pitch angle error:      {pitch_err:.6f}")
95          print(f"  Input L2 norm:          {u_norm:.6f}")
96          print(f"  Elevator L2 norm:       {elevator_norm:.6f}")
97
98          print(f"  Rise time:              {rise}")
99          print(f"  Settle time:            {settle}")
100         print(f"  Overshoot:              {100*os}%")
101
102         if model == 'initial':
103             init_vel = vel_err
104             init_aoa = aoa_err
105             init_pr = pr_err
106             init_pitch = pitch_err
107             init_total = error_norm
108             init_u = u_norm
109
110             init_rise = rise
111             init_settle = settle
112             init_overshoot = os
113
114             init_elev = elevator_norm
115             continue   # This skips the rest of the for loop and starts ag
    ain.
116
117         vel_rel_err = (init_vel - vel_err)/init_vel
118         aoa_rel_err = (init_aoa - aoa_err)/init_aoa
119         pr_rel_err = (init_pr - pr_err)/init_pr
120         pitch_rel_err = (init_pitch - pitch_err)/init_pitch
121         total_rel_err = (init_total - error_norm)/init_total
122         u_rel_norm = (init_u - u_norm)/init_u
123         elev_rel_norm = (init_elev - elevator_norm)/init_elev
124
125         rise_rel_norm = (rise - init_rise)/init_rise
126         settle_rel_norm = (init_settle - settle)/init_settle
127         os_rel_norm = (init_overshoot - os)/init_overshoot
128
129         print(f"  Velocity relative error:        {vel_rel_err*100:.2f}")
130         print(f"  AOA relative error:             {aoa_rel_err*100:.2f}")
131         print(f"  Pitch rate relative error:      {pr_rel_err*100:.2f}")
132         print(f"  Pitch angle relative error:     {pitch_rel_err*100:.2f}")
133         print(f"  Elevator relative L2 norm:         {elev_rel_norm*100:.2
    f}")
134
135         print(f"  Rise relative:                  {rise_rel_norm*100:.2f}")
136         print(f"  Settle relative:                {settle_rel_norm*100:.2f}")
137         print(f"  OS relative:                    {os_rel_norm*100:.2f}")
138
139
140         data['error']['relative'] = {
141             'vel':vel_rel_err     * 100 ,
142             'aoa':aoa_rel_err     * 100 ,
143             'pr':pr_rel_err       * 100 ,
144             'pitch':pitch_rel_err * 100 ,
145             'total':total_rel_err * 100 ,
146             'u':u_rel_norm
```

```
        }

    print("="*70 + "\n")
    pass
```

utils/trim_aircraft.py

```
''' This script will compute trim '''

import numpy as np
from utils.interpolatory_model import aero_interps
import math
from functools import partial
from scipy import optimize


# def compute_trim_forces(flight_point,data_file, verbose=False):
#     '''This function computes the trim condition for a given flight point.
#     As an input, the flight_point will contain velocity, center halfspan, and wing halfspan.
#     It will also take the aero data file as an input.

#     It will output the pitch angle, elevator deflection, and thrust throttle for trim.'''

#     # data_file = 'Aero_data/aero_7dim2.npz'
#     interp_dict = aero_interps(data_file)

#     Fx = interp_dict['Fx']
#     Fz = interp_dict['Fz']
#     M = interp_dict['M']

#     thrust_force = 96000 # lb
#     maxT = thrust_force * 4.44822 * 1.5 # convert to N
#     weight = 331522 # lb
#     weight = weight * 4.44822 # convert to N
#     mass = weight / 9.81  # kg

#     x0 = np.asarray([np.deg2rad(6),np.deg2rad(10),0.3]) # initial guess: 6 deg pitch, 10 deg elev, 30% thrust

#     # Each of these force/moment functions will take x = [pitch, elev, thrust] as inputs...
#     def x_con(x):
#         deltaT = x[2]
#         full_flight_point = np.concatenate((flight_point[0:1],x[0:1],flight_point[1:],x[1:2]))
#         return (Fx(full_flight_point) + deltaT * maxT * math.cos(x[0])) * 1e-5

#     def z_con(x):
#         deltaT = x[2]
#         full_flight_point = np.concatenate((flight_point[0:1],x[0:1],flight_point[1:],x[1:2]))
```

```
41 #         return (-Fz(full_flight_point) - weight + deltaT * maxT * math.s
       in(x[0])) * 1e-4
42
43 #     def m_con(x):
44 #         full_flight_point = np.concatenate((flight_point[0:1],x[0:1],fli
       ght_point[1:],x[1:2]))
45 #         return  M(full_flight_point) * 1e-5
46
47 #     def full_obj(x):
48 #         forces = np.asarray([x_con(x), z_con(x), m_con(x)])
49 #         return forces.flatten()
50
51 #     sol = optimize.fsolve(func = full_obj, x0 = x0)
52
53 #     if np.max(np.abs(full_obj(sol))) > 1e-2:
54 #         print('Warning: Trim solution may not have converged well enough
       !')
55
56 #     if verbose:
57 #         print(f'Forces in x: {x_con(sol) * 1e5} N')
58 #         print(f'Forces in z: {z_con(sol) * 1e4} N')
59 #         print(f'Pitching moment: {m_con(sol) * 1e5} Nm')
60 #         y = [np.rad2deg(sol[0]), np.rad2deg(sol[1]), sol[2]]
61 #         print(f'Trim solution: pitch = {y[0]:.2f} deg, elev = {y[1]:.2f}
        deg, throttle = {y[2]:.2f}')
62
63 #     return sol
64
65
66 def compute_trim_eom(flight_point,data_file,climb_angle,verbose=False,warn
       _trim = True):
67     ''' This function computes trim using nonlinear EOM. It sets V_dot, al
       pha_dot, q_dot to 0
68     and also constrains the flight path angle. Trim variables are aoa, ele
       vator, thrust, pitch.
69     Note that bounds on the values of these quantities are not set.
70     Inputs: flight_point is [velocity, chs, whs, ws (?), ths (?), wd (?)]
71             data_file is the aero data file
72             climb_angle is the desired climb angle in degrees (positive fo
       r climb, negative for descent)
73             verbose is for printing out constraint values
74     Outputs: trimmed [aoa, elev, thrust, pitch]'''
75     interp_dict = aero_interps(data_file)
76     V0 = flight_point[0]
77     M = interp_dict['M']
78     Di = interp_dict['Di']
79     L = interp_dict['L']
80
81     # h0 = 10      # alt in km
82     # full flight point will modify fp argument so it's [vel, aoa, chs, wh
       s, elevator]
83
84     thrust_force = 96000 # lb
85     maxT = thrust_force * 4.44822 * 1.5 # convert to N
86     weight = 331522 # lb
87     weight = weight * 4.44822 # convert to N
88     mass = weight / 9.81  # kg
89     g = 9.81
```

```
 90
 91      x0 = np.asarray([np.deg2rad(5),np.deg2rad(5),0.5, np.deg2rad(5)]) # in
         itial guess: 5 deg aoa, 5 deg elev, half thrust, 5 deg pitch
 92
 93      # Constraints based on EOM instead of force/moment balance.
 94      def vdot_con(x):
 95          # x is [alpha, elev, thrust, pitch]
 96          lb = np.array([np.deg2rad(-5),np.deg2rad(-30),0,np.deg2rad(-20)])
 97          ub = np.array([np.deg2rad(15),np.deg2rad(30),1,np.deg2rad(60)])
 98          x = np.clip(x,lb,ub)
 99
100          deltaT = x[2]
101          Ft = deltaT * maxT
102          full_flight_point = np.concatenate((flight_point[0:1],x[0:1],fligh
         t_point[1:3],x[1:2],flight_point[3:]))
103          return ( Ft/mass * np.cos(x[0]) - Di(full_flight_point)/mass - g * n
         p.sin(x[3] - x[0]) ) * 1
104
105      def alpha_con(x):
106
107          lb = np.array([np.deg2rad(-5),np.deg2rad(-30),0,np.deg2rad(-20)])
108          ub = np.array([np.deg2rad(15),np.deg2rad(30),1,np.deg2rad(60)])
109          x = np.clip(x,lb,ub)
110
111          deltaT = x[2]
112          alpha = x[0]
113          theta = x[3]
114          Ft = deltaT * maxT
115          full_flight_point = np.concatenate((flight_point[0:1],x[0:1],fligh
         t_point[1:3],x[1:2],flight_point[3:]))
116          return (-Ft/(mass * V0) * np.sin(alpha) - L(full_flight_point)/(ma
         ss * V0) + g/V0 * np.cos(theta - alpha) ) * 1e3
117
118      def q_con(x):
119          lb = np.array([np.deg2rad(-5),np.deg2rad(-30),0,np.deg2rad(-20)])
120          ub = np.array([np.deg2rad(15),np.deg2rad(30),1,np.deg2rad(60)])
121          x = np.clip(x,lb,ub)
122
123          deltaT = x[2]
124          alpha = x[0]
125          theta = x[3]
126          Ft = deltaT * maxT
127          full_flight_point = np.concatenate((flight_point[0:1],x[0:1],fligh
         t_point[1:3],x[1:2],flight_point[3:]))
128          return M(full_flight_point) * 1e-5
129
130      def pathangle_con(x):
131          lb = np.array([np.deg2rad(-5),np.deg2rad(-30),0,np.deg2rad(-20)])
132          ub = np.array([np.deg2rad(15),np.deg2rad(30),1,np.deg2rad(60)])
133          x = np.clip(x,lb,ub)
134
135          return np.asarray([x[3] - x[0] - np.deg2rad(climb_angle)])
136
137      def full_obj(x):
138          forces = np.asarray([vdot_con(x), alpha_con(x), q_con(x), pathangl
         e_con(x)])
139          return forces.flatten()
140
```

```
141     bnds = ((np.deg2rad(-5),np.deg2rad(15)), # AOA bounds
142         (np.deg2rad(-30),np.deg2rad(30)), # elev bounds
143         (0.0,1.0),                        # thrust bounds
144         (np.deg2rad(-20),np.deg2rad(60)))  # pitch bounds
145 
146     cons = ({'type':'eq','fun':vdot_con},
147         {'type':'eq','fun':alpha_con},
148         {'type':'eq','fun':q_con},
149         {'type':'eq','fun':pathangle_con})
150 
151     def penalty_obj(x):
152         return vdot_con(x)**2 + alpha_con(x)**2 + q_con(x)**2 + pathangle_
    con(x)**2
153 
154     best = {"x": None, "cost": np.inf}
155 
156     def callback(x):
157         cost = penalty_obj(x)
158         if cost < best["cost"]:
159             best["cost"] = cost
160             best["x"] = x.copy()
161 
162     ops = {'maxiter':200}
163     sol = optimize.minimize(fun=penalty_obj, x0=x0, constraints=cons, boun
    ds=bnds, options=ops, callback=callback)
164     if not sol.success and warn_trim == True:
165         print('Warning, failed to trim!')
166         print(f'Velocity is: {flight_point[0]}')
167         print(best)
168 
169     if verbose:
170         y = [np.rad2deg(sol.x[0]), np.rad2deg(sol.x[1]), sol.x[2], np.rad2
    deg(sol.x[3])]
171         print(f'y:{y}')
172         print(f'V constraint: {vdot_con(sol.x)}')
173         print(f'Alpha constraint: {alpha_con(sol.x)}')
174         print(f'Moment constraint: {q_con(sol.x)}')
175         print(f'Gamma constraint: {pathangle_con(sol.x)}')
176 
177     # Solving with fsolve
178 
179     # sol = optimize.fsolve(func = full_obj, x0 = x0)
180 
181 
182     return sol.x[0],sol.x[1],sol.x[2],sol.x[3], sol.fun
183 
184 if __name__ == "__main__":
185     v0 = 200 # speed in m/s
186     more_vars = True
187     chs = 5 # center halfspan in m
188     whs = 12 # wing halfspan in m
189     if more_vars:
190 
191         ws = 35 # wing sweep in rad
192         ths = 5 # trans half span in m
193         wd = 10 # wing dihedral in rad
194         flight_point = np.asarray([v0,chs,whs,ws,ths,wd])
195         data_file = 'Aero_data/aero_merged_cg_17.npz'
```

```
    else:
        flight_point = np.asarray([v0,chs,whs])
        data_file = 'Aero_data/aero_7dim2.npz'
    trimx = compute_trim_eom(flight_point, data_file, 0, verbose=True)
    print(trimx)
```

utils/volume_smoothness/volume_smoothness.py

```
''' Plot the behavior of the volume of the reachable set in a neighborhood
     of one of the optimal points'''

from design_optimization import set_hull
import numpy as np
import matplotlib.pyplot as plt
import time
# pick a point

x = np.array([6, 11])
dx = np.array([1, 1])
V0 = 200
num_pts = 15

a = np.linspace(x[0]-dx[0],x[0]+dx[0],num_pts)
b = np.linspace(x[1]-dx[1],x[1]+dx[1],num_pts)

A,B = np.meshgrid(a,b,indexing='ij')
# volumes = np.zeros((num_pts,num_pts))
# start_time = time.perf_counter()
# for i in range(num_pts):
#     for j in range(num_pts):
#         x = np.array([A[i,j],B[i,j]])
#         print(x)
#         hull = set_hull(x,V0)
#         vol = hull.volume
#         volumes[i,j] = vol

# end_time = time.perf_counter()
# data = {'volumes':volumes}
# volumes_data = np.savez_compressed('volume_smoothness/volume_data2',**da
    ta)
# print(end_time-start_time)

data = np.load('volume_smoothness/volume_data2.npz')
volumes = data['volumes']

# Creating figure
plt.rcParams.update({'font.size': 15})
plt.rcParams['lines.linewidth'] = 4
plt.rcParams["legend.handlelength"] = 3.5
plt.rcParams['lines.markersize'] = 12

fig = plt.figure(figsize =(14, 9))
ax = plt.axes(projection ='3d')


```

```
surf = ax.plot_surface(A,B,volumes, cmap="viridis")

fig.colorbar(surf, ax=ax, shrink=0.5, label="Volume")
ax.set_title('Volume of reachable set for varying design parameters')
ax.set_xlabel("Center half-span", labelpad=15)
ax.set_ylabel("Wing half-span", labelpad=15)
ax.set_zlabel("Volume", labelpad=10)

ax.set_yticks(np.arange(10, 12.2, 0.5))
ax.set_xticks(np.arange(5,7.2, 0.5))

ax.view_init(elev=10, azim=-131, roll=0)
plt.savefig('volume_smoothness/volume_smoothness.png',dpi=300)
plt.show()
print(A)
print(B)
```